

iOS SDK 1.10.0.0 for Xcode User Guide



This document is intended only for authorized individuals of AppSealing customer.
Unauthorized distribution of any parts of this document to other parties is prohibited.

The software referenced herein, this User Guide, and any associated documentation is provided to you pursuant to the agreement between your company, governmental body or other entity (“you”) and INKA Entworks Corporation (“AppSealing”) under which you have received a copy of AppSealing Licensed Technology and this User Guide (such agreement, the “Agreement”). Defined terms not defined herein shall have the meanings ascribed to them in the Agreement. In the event of conflict between the terms of this User Guide and the terms of the Agreement, the terms of the Agreement shall prevail. Without limiting the generality of the remainder of this paragraph, (a) this User Guide is provided to you for informational purposes only, (b) your right to access, view, use, and copy this User Guide is limited to the rights and subject to the applicable requirements and limitations set forth in the Agreement, and (c) all of the content of this User Guide constitutes “Confidential Information” of AppSealing (or the equivalent term used in the Agreement) and is subject to all of the limitations and requirements pertinent to the use, disclosure and safeguarding of such information. Permitting anyone who is not directly involved in the authorized use of AppSealing Licensed Technology by your company or other entity to gain any access to this User Guide shall violate the Agreement and subject your company or other entity to liability therefore.

Copyright Information

Copyright © 2000-2025 INKA Entworks Corporation. All rights reserved.

AppSealing® is a trademark of INKA Entworks Corporation in the South Korea and/or other countries.

macOS™ and Xcode® are trademarks of Apple Inc., registered in the United States and other countries.

All other trademarks are the property of their respective owners.

Disclaimer

The remainder of this User Guide notwithstanding, this User Guide is provided “as is”, without any warranty whatsoever (including that it is error-free or complete). This User Guide contains no express or implied warranties, covenants or grants of rights or licenses, and does not modify or supplement any express warranty, covenant or grant of rights or licenses that is set forth in the Agreement. This User Guide is current as of the date set forth in the header that appears above on this page, but may be modified at any time without prior notice. Future revisions and updates of this User Guide shall be distributed as part of AppSealing SDK new releases. INKA Entworks shall under no circumstances bear any responsibility for your failure to operate AppSealing Licensed Technology in compliance with the then-current version of this User Guide. Your remedies with respect to your use of this User Guide, and INKA Entworks' liability for your use of this User Guide (including for any errors or inaccuracies that appear in this User Guide) are limited to those remedies expressly authorized by the Agreement (if any).

Contact Information

Address: [06109] 1F 608, Nonhyeon-ro, Gangnam-gu, Seoul, South Korea

Technical support: <https://helpcenter.appsealing.com>

Website: www.appsealing.com

목차

요구 사항	4
-------------	---

Part 1. AppSealing SDK의 위치

1-1 AppSealing SDK 프로젝트 폴더에 복사	5
1-2 AppSealing SDK 압축 해제	6
1-3 프로젝트 Bundle ID 와 SDK에 등록된 Bundle ID 비교	7

Part 2. 프로젝트에 AppSealing 파일 추가

2-1 Xcode project 열기	8
2-2 'File >> Add Files to "TestApp_Swift"...'를 이용하여 파일 추가	8
2-3 Swift 프로젝트에 Bridging header 추가	10
2-4 Bridging header file에 AppSealing header 추가 (Swift 프로젝트)	11
2-5 PrivacyInfo.xcprivacy 파일에 항목 추가 (App Store upload).	12

Part 3. 보안 위협 안내를 위한 GUI 추가

3-1 앱에서 UIAlertController 출력	14
------------------------------------	----

Part 4. 프로젝트 "Build Settings" 수정

4-1 프로젝트 메인 target 선택	18
4-2 Build Settings "Other Linker Flags" 설정	18
4-3 Build Settings "Architectures – Excluded Architectures" 설정	21
4-4 앱 빌드 모드에 따른 주의 사항	24
4-5 앱 무결성 및 인증서 검증 정보 생성	25
4-6 메소드 스위칭/메소드 후킹 감지 기능 제어	31
4-7 재서명 앱 App Store Connect 업로드	33

Part 5. AppSealing 기기 고유 ID 획득

5-1 기기 고유 ID 확인	38
-----------------------	----

Part 6. 앱 서버를 이용한 강화된 탈옥 감지 적용 방법

6-1 강화된 탈옥 감지 방안의 개요 및 필요성	40
6-2 iOS 앱 코드 작업	41
6-3 서버 측 검증 작업	42

Part 7. Xcode Cloud에서 재서명 및 업로드

7-1 Xcode Cloud 설정	57
7-2 ci_scripts 준비 사항	59
7-3 빌드 및 업로드 과정 확인	67

Part 8. 소스 코드 문자열 암호화 적용

8-1 프로젝트 Build Phase 설정	72
8-2 문자열 암호화 태그 추가	76
8-3 Swift 문자열 암호화 주의 사항	82
8-4 컴파일 오류 대응: 암호화 된 문자열 복원	83

Part 9. Fastlane 프로젝트에 적용

9-1 프로젝트 Fastfile 설정	85
9-2 Github Action과 Fastlane 동시 적용	89

Part 10. 문제 해결

10-1 Xcode Build error (1) Use of undeclared type 'AppSealingInterface'	99
10-2 Xcode Build error (2) Undefined symbols for architecture arm64: "_OBJC_CLASS_\$_AppSealingInterface"	100
10-3 Xcode Build error (3) ld: library not found for -lStaticAppSec_Debug	102
10-4 Xcode Build error (4) Undefined symbols for architecture arm64: " ObjC_IsAbnormalEnvironmentDetected()", "Appsealing()"	104
10-5 Xcode Build error (5) Undefined symbols for architecture arm64: " ObjC_IsAbnormalEnvironmentDetected()" (Objective-C 프로젝트)	106
10-6 Execution error (1) 앱이 실행되고 곧바로 종료됩니다	107
10-7 Execution error (2) 앱이 실행 중 갑자기 종료됩니다	108
10-8 무결성/인증서 스냅샷 생성 스크립트가 실행되지 않습니다	109

Prerequisite

- Xcode 12.5.1 이상 (macOS 11 이상)
- iOS 9.0 이상 기기

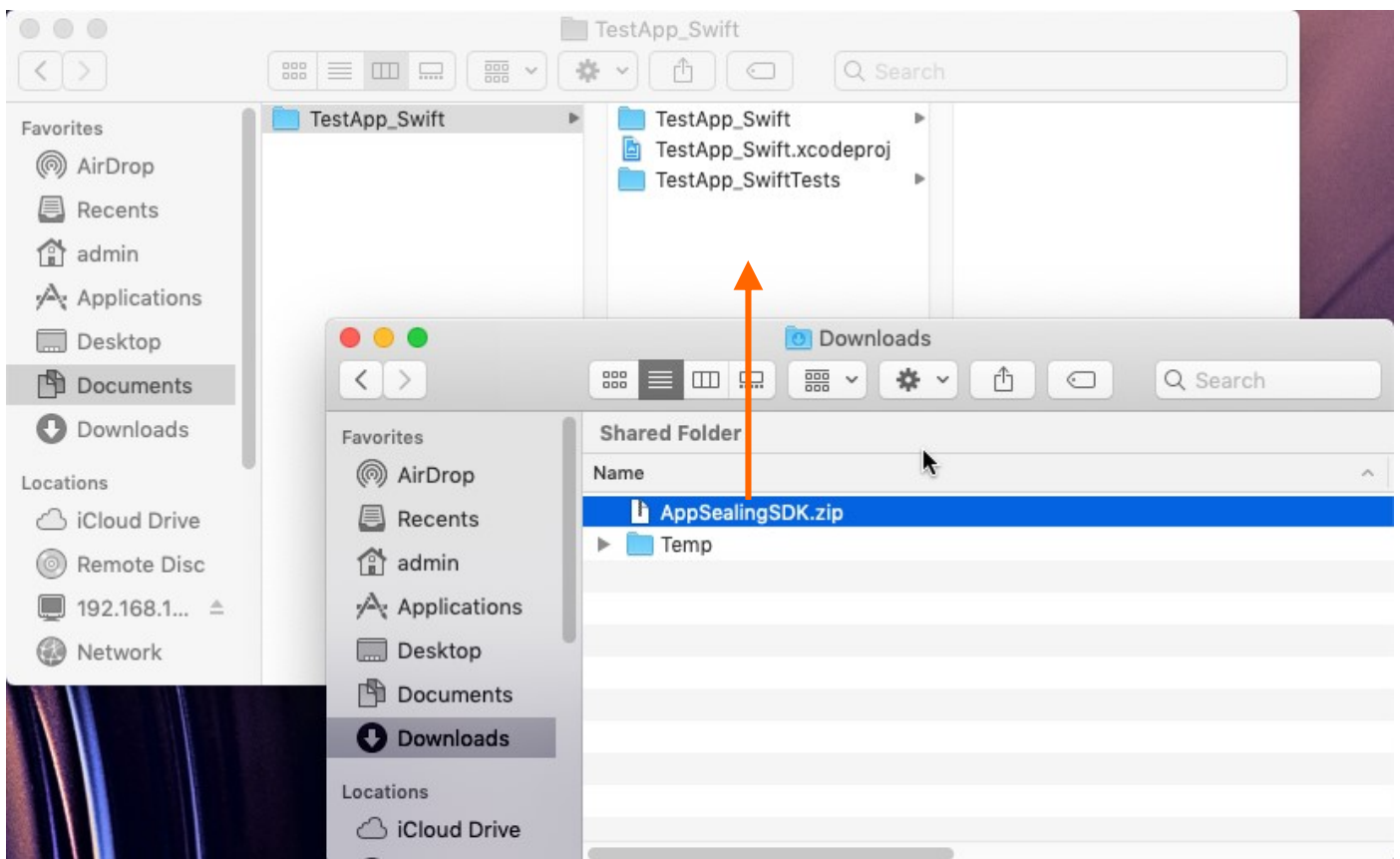
Supported CI/CD Tool

- Xcode Cloud
- Fastlane

Part 1. AppSealing SDK 폴더 위치

AppSealing SDK zip 파일을 다운로드한 후 Xcode 프로젝트 폴더에 압축을 해제합니다. 이 문서에서 안내되는 샘플 프로젝트 'TestApp_Swift'는 "~/Documents/TestApp_Swift"경로에 위치합니다.

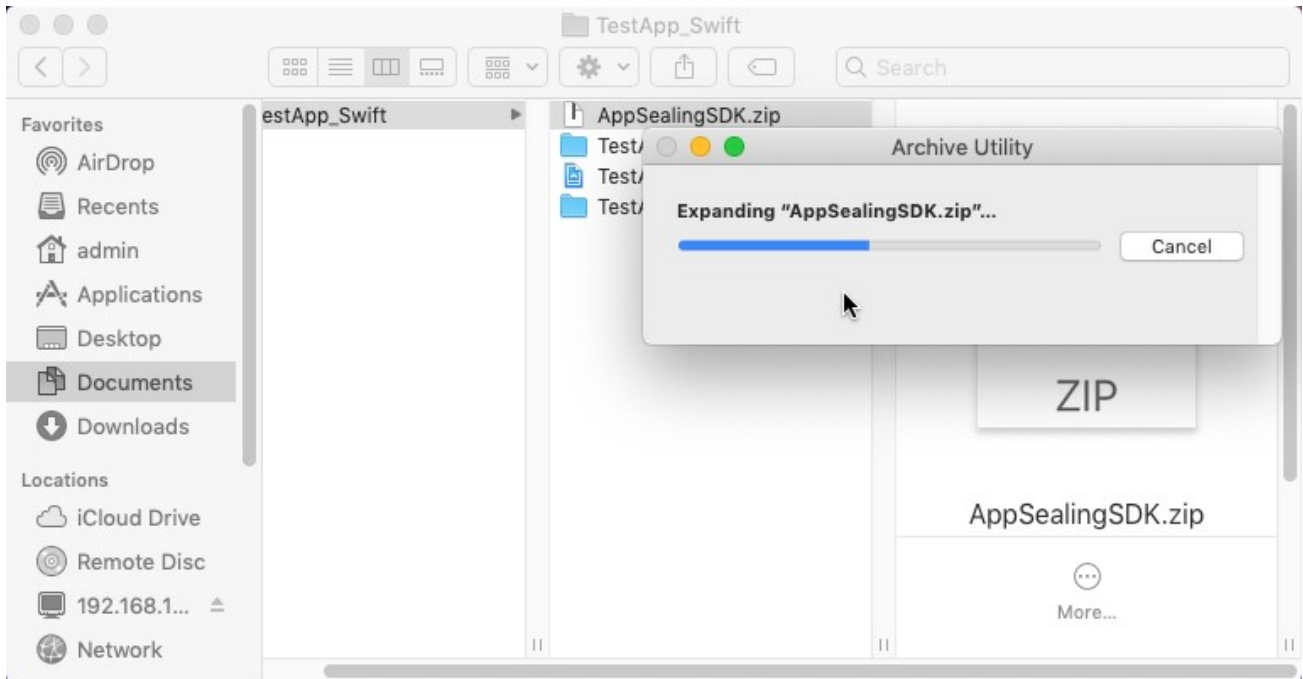
1-1 AppSealing SDK 프로젝트 폴더에 복사



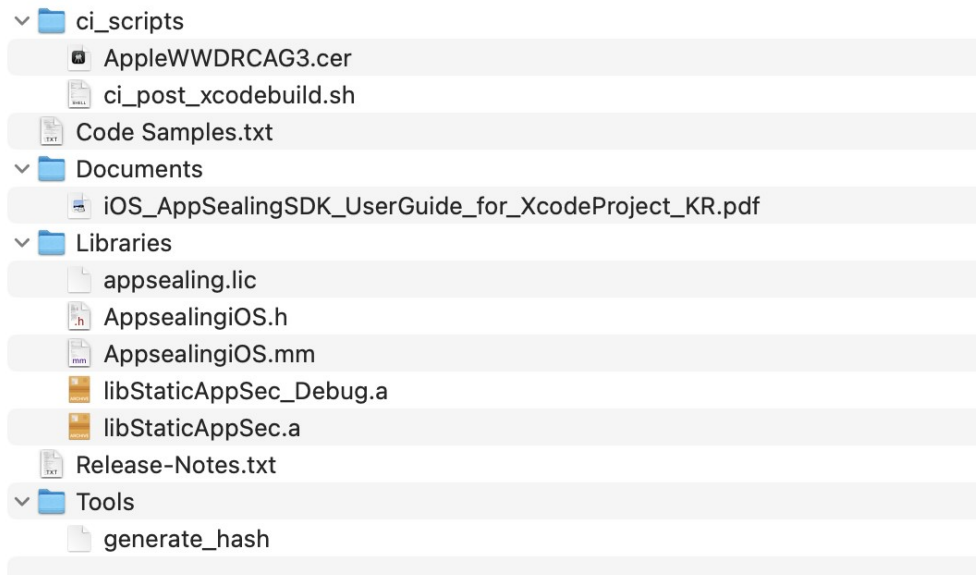
**** 주의:** SDK 압축 파일의 이름이 "AdhocEnabled"를 포함하는 버전은 공식 App Store 혹은 TestFlight 이외의 방식(Ad Hoc, enterprise, MDM 및 기타 third party store)으로 앱을 배포할 수 있도록 일부 보안 기능이 약화된 SDK입니다. App Store을 통하지 않은 배포는 공격에 취약해질 수 있으므로, 반드시 필요한 경우가 아니라면 AdhocEnabled가 아닌 버전으로 다운로드 받으시기 바랍니다.

1-2 AppSealing SDK 압축 해제

복사한 AppSealingSDK.zip 파일을 더블클릭 하여 압축을 해제합니다.



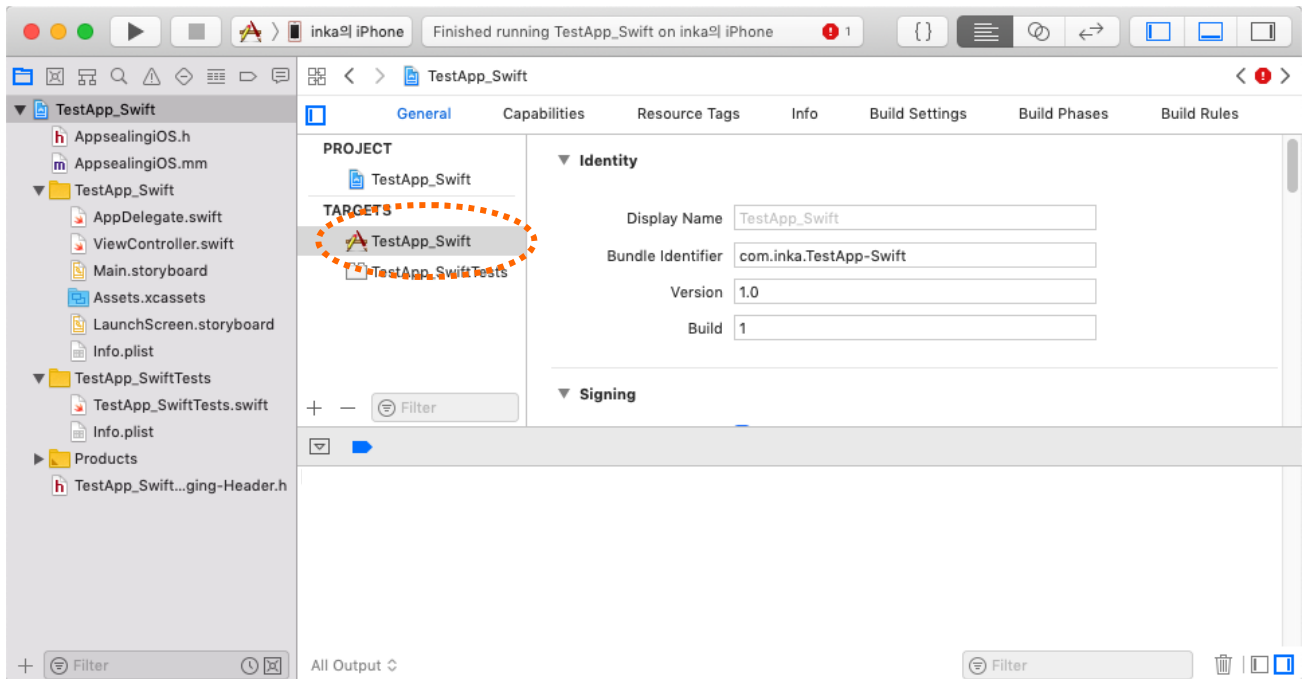
Zip 파일 압축 해제 후 다음과 같은 구조처럼 모든 파일이 생성되는지 확인하십시오.



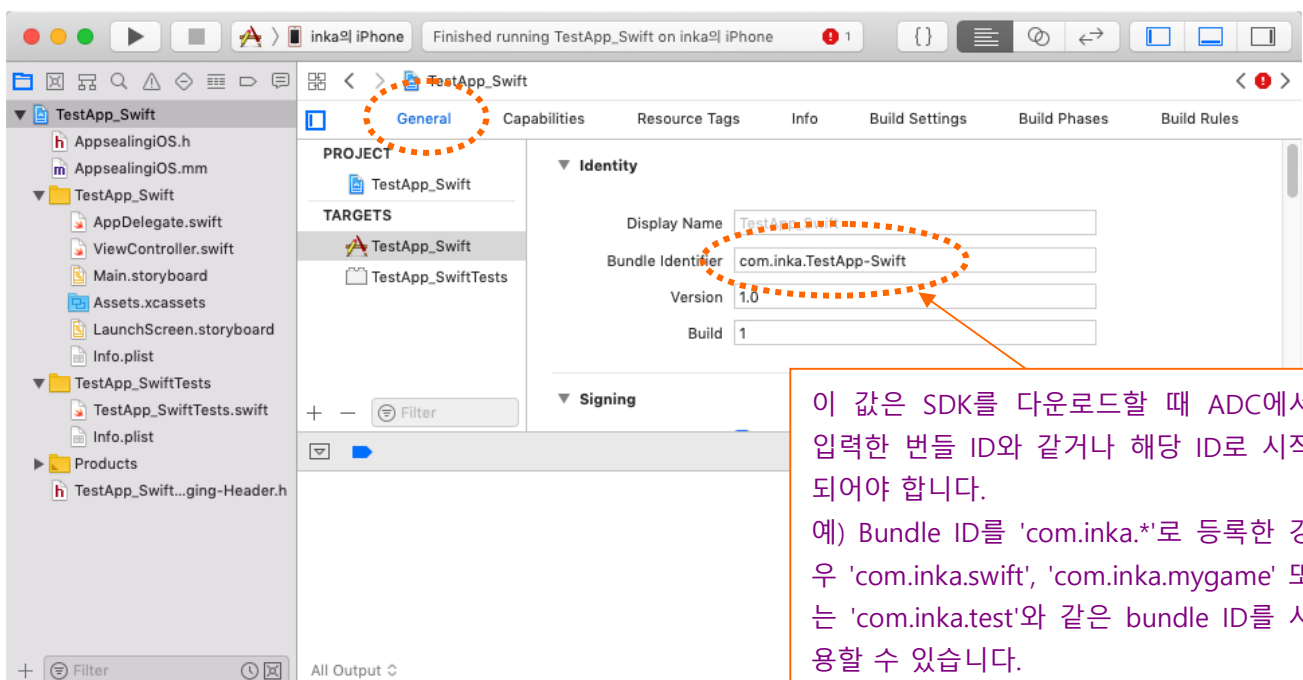
파일이 없는 경우 SDK를 다시 내려 받거나 zip 파일 압축을 다시 해제해 보십시오.

1-3 프로젝트 Bundle ID 와 SDK에 등록된 **Bundle ID** 비교

SDK zip 파일을 압축 해제하고 프로젝트의 bundle ID가 유효한지 확인하십시오. Xcode를 열고 왼쪽 패널에서 프로젝트를 선택하고 Targets 목록에서 프로젝트 이름 (TestApp_Swift)를 선택하십시오.

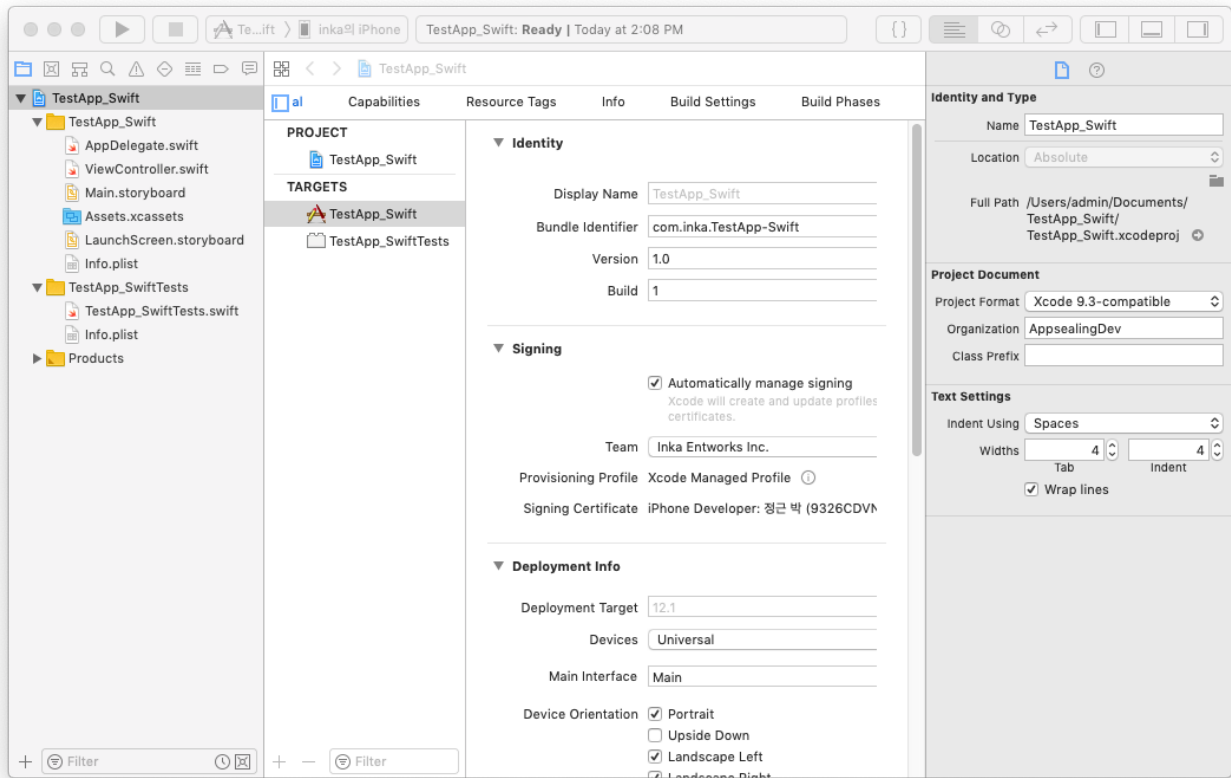


그 다음 "General" 탭을 선택하고 bundle ID가 AppSealing Developer Center (ADC)에서 SDK를 다운로드할 때 입력한 값과 같거나 해당 값으로 시작되는지 확인하십시오.

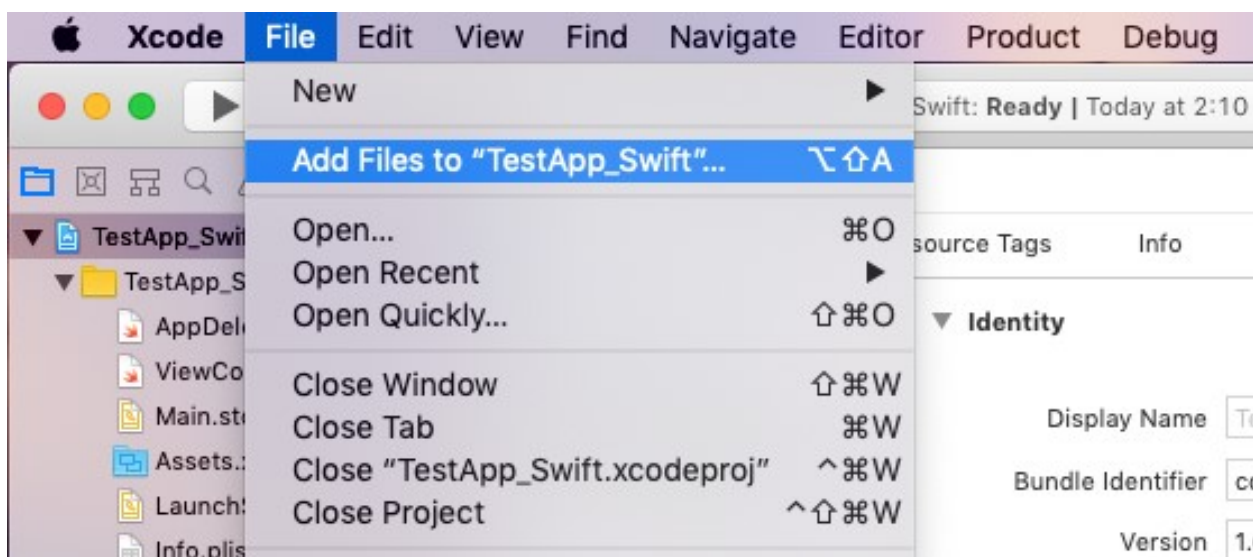


Part 2. 프로젝트에 AppSealing 파일 추가

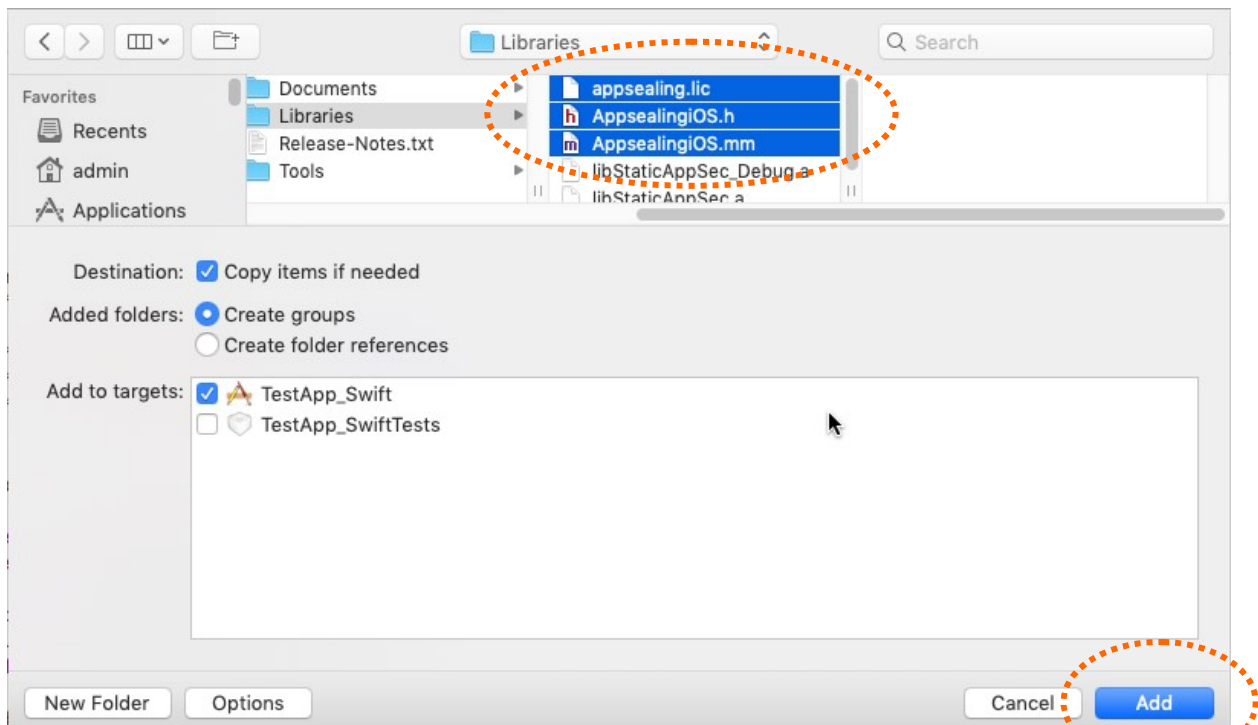
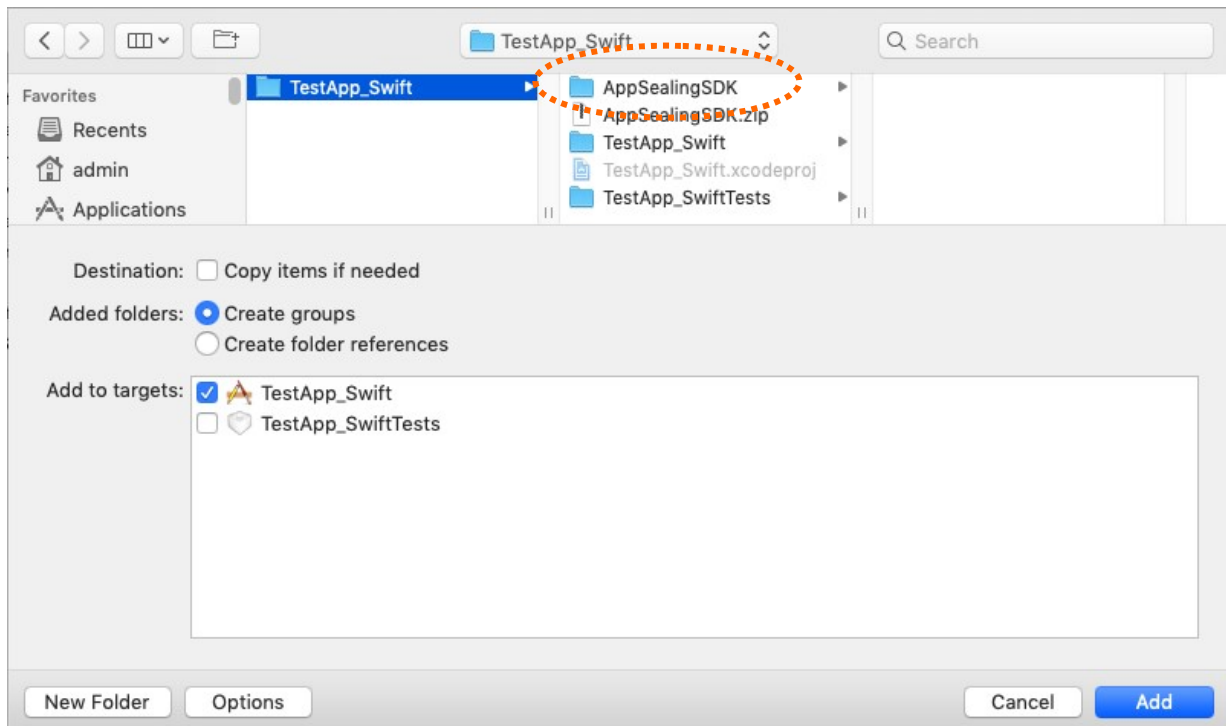
2-1 Xcode Project 열기



2-2 'File >> Add Files to "TestApp_Swift"...' 를 이용하여 파일 추가



대화 상자에서 "AppSealingSDK/Library/" 폴더의 "**appsealing.lic**", "**AppsealingiOS.h**" and "**AppsealingiOS.mm**"를 선택하고 "Add" 버튼을 눌러 프로젝트에 추가하십시오.



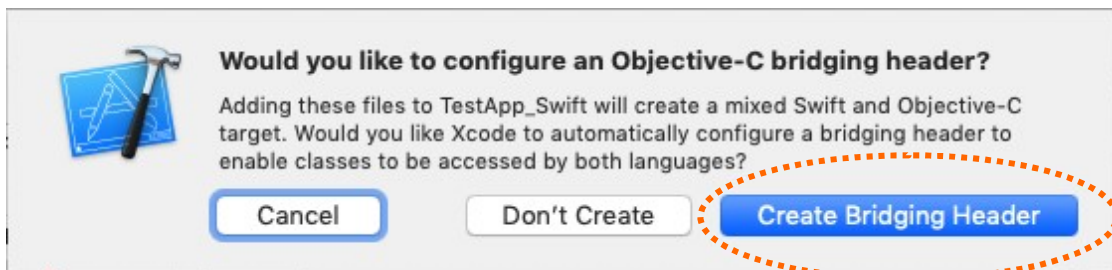
파일 추가를 위해 버튼을 누르십시오

2-3 Swift 프로젝트에 **Bridging Header** 추가

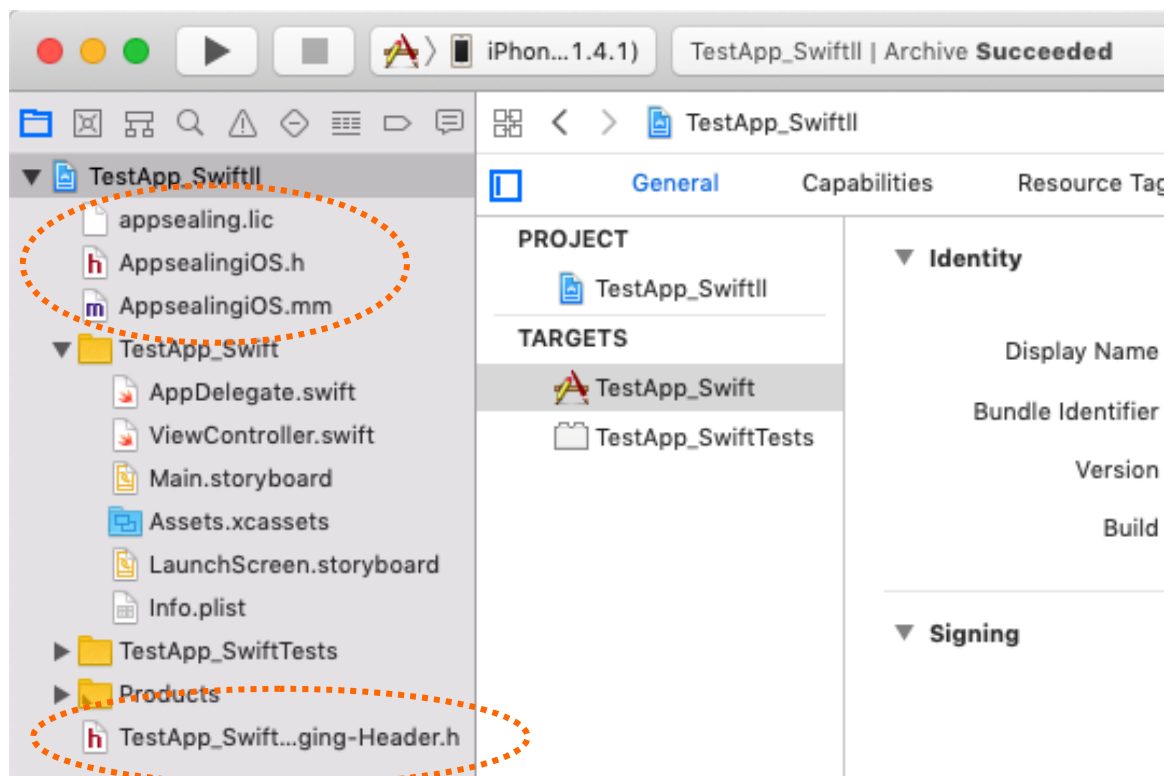
프로젝트가 Swift 기반인 경우에만 진행하십시오.

(Objective-C 기반 프로젝트인 경우 이 페이지와 다음 페이지를 참조하지 마십시오.)

프로젝트가 swift 기반인 프로젝트에 bridging header가 없는 경우 bridging header를 생성하라는 대화 상자가 나타날 수 있습니다. "Create Bridging Header"을 클릭하여 bridging header를 만드십시오.



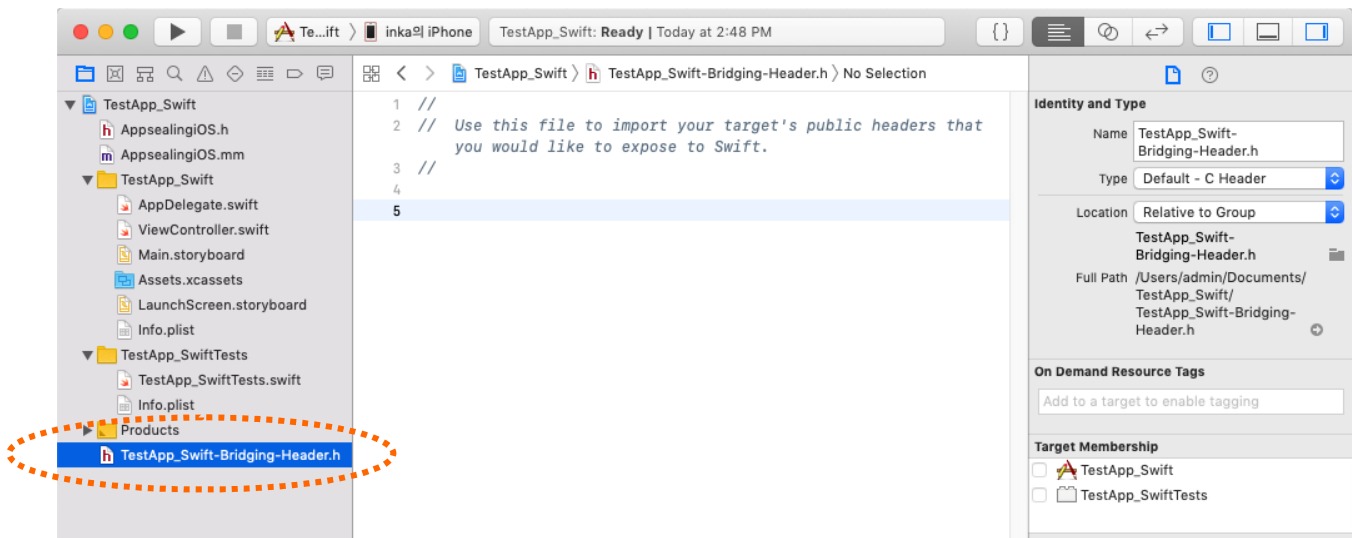
Bridging Header를 만들기 위해 이 버튼을 클릭하십시오.



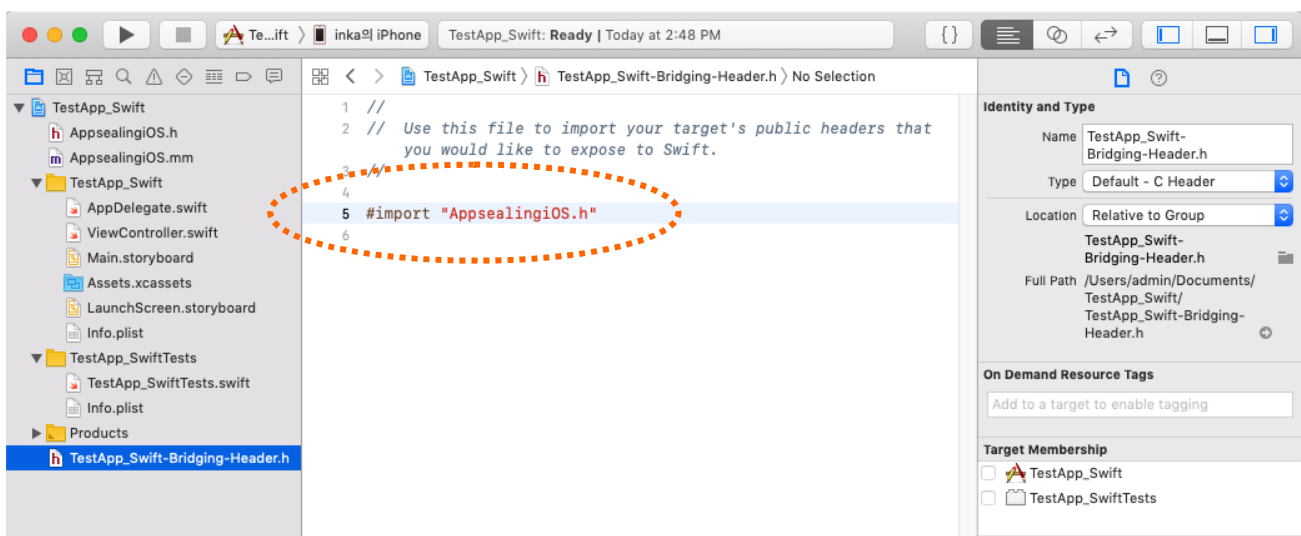
* 프로젝트에 이미 bridging header가 있는 경우, bridging header 생성 대화상자가 나타나지 않고 기존 bridging header 파일을 사용할 수 있습니다.

** 프로젝트가 Objective-C 기반인 경우 bridging header 파일은 사용되지 않습니다.

2-4 Bridging header 파일에 AppSealing header 추가 (Swift project)



새로 생성된 "TestApp_Swift-Bridging-Header.h" 파일을 선택하고 `#import "AppsealingiOS.h"`를 문서 하단에 추가하십시오.

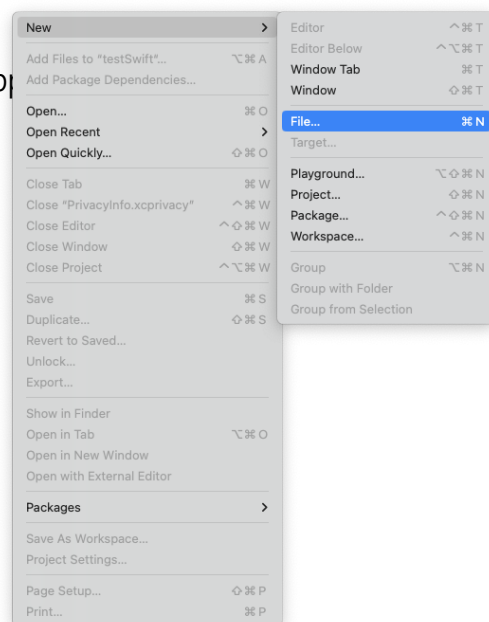


2-5 PrivacyInfo.xcprivacy 파일에 항목 추가 (App Store upload)

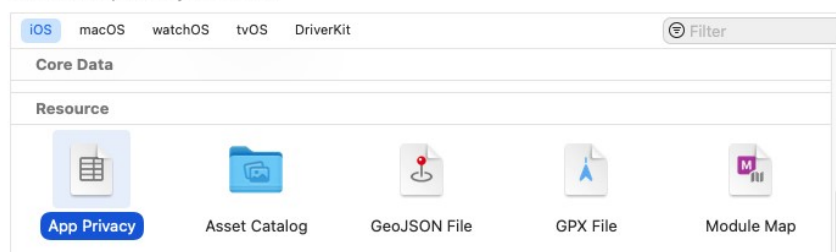
2024년 5월부터 App Store 업로드에 요구되는 privacy manifest 파일에도 항목 추가가 필요합니다.

현재 프로젝트에 "PrivacyInfo.xcprivacy" 파일이 없다면 프로젝트의 .xcodeproj 혹은 .xcworkspace 파일을 Xcode로 열고 메뉴의 "File > New > File..."을 선택하십시오:

"Resource" 섹션의 "App Privacy Manifest" 파일을 추가할 수 있습니다:



Choose a template for your new file:



Privacy manifest 파일에 대한 더 자세한 설명은 아래의 링크를 참고해주시오:

https://developer.apple.com/documentation/bundleresources/privacy_manifest_files#4284009

"PrivacyInfo.xcprivacy" 파일을 열고 다음과 같은 하위 구조를 포함하도록 항목을 추가해주시오.

- key "NSPrivacyAccessedAPITypes"
 - array
 - ◆ key "NSPrivacyAccessedAPIType": string "NSCategoryFileTimestamp"
 - ◆ key "NSPrivacyAccessedAPITypeReasons"
 - array
 - string "C617.1"

결과적으로 Xcode에서 "PrivacyInfo.xcprivacy"를 확인했을 때 다음과 같은 구조를 포함해야 합니다:

▼ Privacy Accessed API Types	↕ Array	(1 item)	
▼ Item 0	Dictionary	(2 items)	
Privacy Accessed API Type	↕ String	File Timestamp	↕
▼ Privacy Accessed API Reasons	↕ Array	(1 item)	
Item 0	String	C617.1: Inside app or group container, per documentation	↕

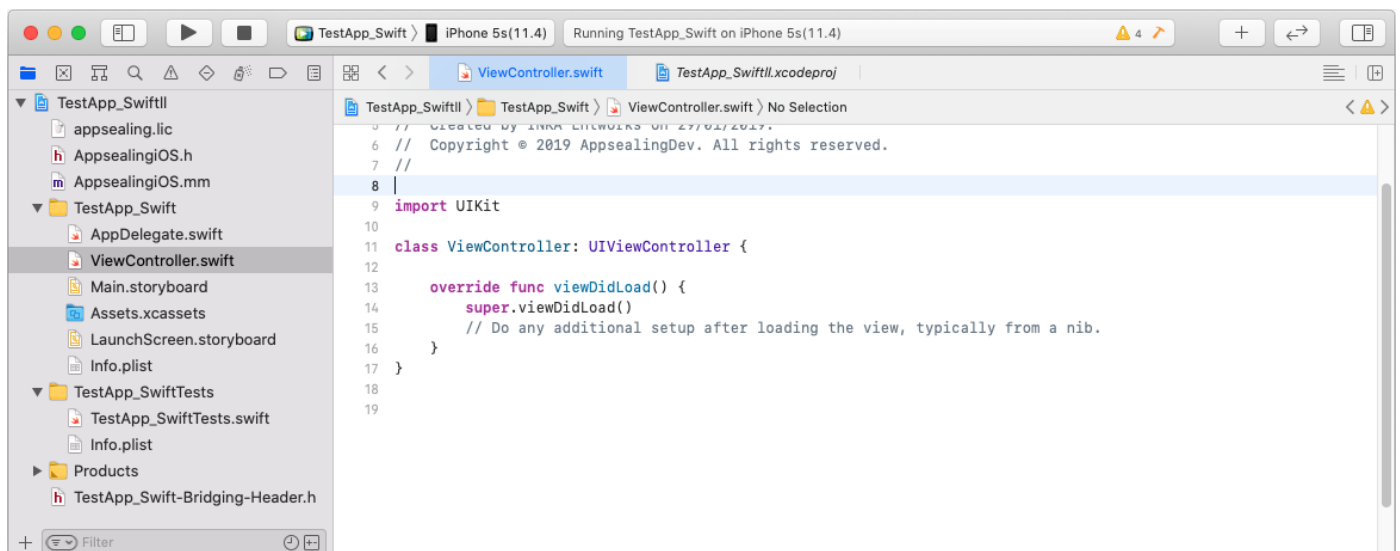
Part 3. 보안 위협 안내를 위한 GUI

AppSealing 라이브러리는 앱이 실행될 때 자동으로 활성화됩니다. AppSealing 라이브러리가 비정상적인 환경 (탈옥된 장치, 실행 파일의 복호화 된 상태, 디버거 연결)을 감지한 경우 사용자 조치와 관계없이 20초 후에 앱이 닫히도록 설계되어 있습니다. 앱 탐지 결과를 사용자에게 통지하고 사용자가 일부 잘못된 환경에 있음을 인식할 수 있도록 적절한 메시지 상자를 표시하십시오.

앱에서 해당 대화 상자를 표시하려면 아래 과정을 참고하여 "ViewController.swift" 파일에 코드를 추가하십시오. (프로젝트가 Objective-C 기반인 경우 ViewController.m 파일의 확장자를 mm 으로 바꾼 후에 수정)

3-1 앱에서 UIAlertController 출력

먼저 Xcode project를 열고 왼쪽 프로젝트 패널에서 "ViewController.swift" 파일을 선택합니다.



ViewController.swift 파일을 연 후 해당 파일에 다음 코드를 삽입하십시오. 만약 ViewController.swift 파일에 'viewDidAppear' 메서드가 이미 포함되어 있다면 'super.viewDidAppear(animated);' 줄 바로 아래 부분의 코드를 삽입하십시오.

Simple UI code into 'ViewController.swift' for Swift project

```

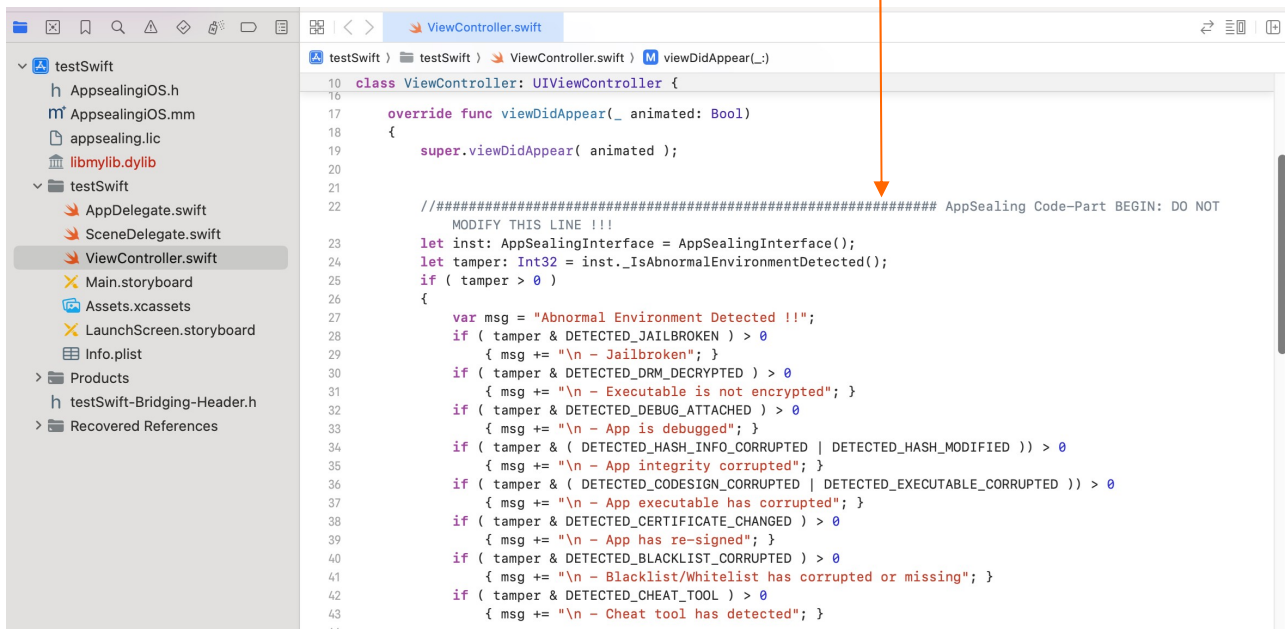
override func viewDidLoad(_ animated: Bool)
{
    super.viewDidLoad( animated );

    //##### AppSealing Code-Part BEGIN: DO NOT MODIFY THIS LINE !!!
    let inst: AppSealingInterface = AppSealingInterface();
    let tamper: Int32 = inst._IsAbnormalEnvironmentDetected();
    if ( tamper > 0 )
    {
        var msg = "Abnormal Environment Detected !!";
        if ( tamper & DETECTED_JAILBROKEN ) > 0
        { msg += "\n - Jailbroken"; }
        if ( tamper & DETECTED_DRM_DECRYPTED ) > 0
        { msg += "\n - Executable is not encrypted"; }
        if ( tamper & DETECTED_DEBUG_ATTACHED ) > 0
        { msg += "\n - App is debugged"; }
        if ( tamper & ( DETECTED_HASH_INFO_CORRUPTED | DETECTED_HASH_MODIFIED ) > 0
        { msg += "\n - App integrity corrupted"; }
        if ( tamper & ( DETECTED_CODESIGN_CORRUPTED | DETECTED_EXECUTABLE_CORRUPTED ) > 0
        { msg += "\n - App executable has corrupted"; }
        if ( tamper & DETECTED_CERTIFICATE_CHANGED ) > 0
        { msg += "\n - App has re-signed"; }
        if ( tamper & DETECTED_BLACKLIST_CORRUPTED ) > 0
        { msg += "\n - Blacklist/Whitelist has corrupted or missing"; }
        if ( tamper & DETECTED_CHEAT_TOOL ) > 0
        { msg += "\n - Cheat tool has detected"; }

        let alertController = UIAlertController(title: "AppSealing", message: msg, preferredStyle: .alert );
        alertController.addAction(UIAlertAction(title: "Confirm", style: .default,
            handler: { (action:UIAlertAction!) -> Void in
                #if !DEBUG // Debug mode does not kill app even if security threat has found
                    exit(0);
                #endif
            } ));
        self.present(alertController, animated: true, completion: nil);
    }

    AppSealingInterface._NotifySwizzlingDetected( { (msg: String?) -> () in
        let alertController = UIAlertController( title: "AppSealing Security", message: msg, preferredStyle: .alert )
        alertController.addAction( UIAlertAction( title: "Confirm", style: .default,
            handler: { ( action:UIAlertAction! ) -> Void in
                #if !DEBUG
                    exit(0);
                #endif
            } ));
        self.present( alertController, animated: true, completion: nil );
    });
    //##### AppSealing Code-Part END: DO NOT MODIFY THIS LINE !!!
}

```



위의 샘플 UI 코드는 "Code Samples.txt" 파일에도 포함되어 있습니다. 코드를 복사하여 붙여 넣을 수 있습니다.

프로젝트가 Objective-C 기반인 경우 다음 코드를 사용하여 UI를 표시할 수 있습니다.

Simple UI code into 'ViewController.mm' for **Objective-C** project

```
#include "AppsealingiOS.h"
-(void)viewDidAppear:(BOOL)animated
{
    [super viewDidAppear:animated];

    //##### AppSealing Code-Part BEGIN: DO NOT MODIFY THIS LINE !!!
    int tamper = ObjC_IsAbnormalEnvironmentDetected();
    if ( tamper > 0 )
    {
        NSString* msg = @"Abnormal Environment Detected !!";
        if ( ( tamper & DETECTED_JAILBROKEN ) > 0 )
            msg = [msg stringByAppendingString:@"\n - Jailbroken"];
        if ( ( tamper & DETECTED_DRM_DECRYPTED ) > 0 )
            msg = [msg stringByAppendingString:@"\n - Executable is not encrypted"];
        if ( ( tamper & DETECTED_DEBUG_ATTACHED ) > 0 )
            msg = [msg stringByAppendingString:@"\n - App is debugged"];
        if ( ( tamper & ( DETECTED_HASH_INFO_CORRUPTED | DETECTED_HASH_MODIFIED ) ) > 0 )
            msg = [msg stringByAppendingString:@"\n - App integrity corrupted"];
        if ( ( tamper & ( DETECTED_CODESIGN_CORRUPTED | DETECTED_EXECUTABLE_CORRUPTED ) ) > 0 )
            msg = [msg stringByAppendingString:@"\n - App executable has corrupted"];
        if ( ( tamper & DETECTED_CERTIFICATE_CHANGED ) > 0 )
            msg = [msg stringByAppendingString:@"\n - App has re-signed"];
        if ( ( tamper & DETECTED_BLACKLIST_CORRUPTED ) > 0 )
            msg = [msg stringByAppendingString:@"\n - Blacklist/Whitelist has corrupted or missing"];
        if ( ( tamper & DETECTED_CHEAT_TOOL ) > 0 )
            msg = [msg stringByAppendingString:@"\n - Cheat tool has detected"];

        UIAlertController *alert = [UIAlertController alertControllerWithTitle:@"AppSealing"
                                                                    message:msg
                                                                    preferredStyle:UIAlertControllerStyleAlert];
        UIAlertAction *confirm = [UIAlertAction actionWithTitle:@"Confirm"
                                                            style:UIAlertActionStyleDefault
                                                            handler:^(UIAlertAction * _Nonnull action) {
                                                                #if !DEBUG && !defined(DEBUG) // Debug mode does not kill app even if security threat has found
                                                                    exit(0);
                                                                #endif
                                                            }];
        [alert addAction:confirm];
        [self presentViewController:alert animated:YES completion:nil];
    }

    [AppSealingInterface _NotifySwizzlingDetected:^(NSString* msg){
        UIAlertController *alert = [UIAlertController alertControllerWithTitle:@"AppSealing"
                                                                    message:msg
                                                                    preferredStyle:UIAlertControllerStyleAlert];
        UIAlertAction *confirm = [UIAlertAction actionWithTitle:@"Confirm"
                                                            style:UIAlertActionStyleDefault
                                                            handler:^(UIAlertAction * _Nonnull action) {
                                                                #if !DEBUG && !defined(DEBUG) // Debug mode does not kill app even if security threat has found
                                                                    exit(0);
                                                                #endif
                                                            }];
        [alert addAction:confirm];
        [self presentViewController:alert animated:YES completion:nil];
    }];
    //##### AppSealing Code-Part END: DO NOT MODIFY THIS LINE !!!
}
```

Xcode나 gdb를 사용하여 앱을 디버깅하거나 비정상적인 장치에서 앱을 실행하면 앱에서 아래와 같은 간단한 경고 상자가 표시됩니다. 이런 상황에서 사용자 조치와 상관없이 앱은 20초 후에 자동으로 종료됩니다.

iOS 기기에서 보안 침입이 감지되면 경고 창이 출력되고 20초 후 또는 사용자가 "Confirm" 버튼을 누르면 앱이 닫힙니다.





프로젝트가 flutter 기반인 경우 ViewController.swift 파일이 생성되지 않으므로 AppDelegate.swift 파일에 다음과 같이 수정된 코드를 추가하여 GUI를 생성할 수 있습니다.

Simple UI code into 'AppDelegate.swift' for flutter project

```
func application(_ application: UIApplication, didFinishLaunchingWithOptions
    launchOptions: [UIApplication.LaunchOptionsKey: Any]?) -> Bool
{
    // Override point for customization after application launch.

    ##### AppSealing Code-Part BEGIN: DO NOT MODIFY THIS LINE !!!
    let inst: AppSealingInterface = AppSealingInterface();
    let tamper: Int32 = inst._IsAbnormalEnvironmentDetected();
    if ( tamper > 0 )
    {
        var msg = "Abnormal Environment Detected !!";
        if ( tamper & DETECTED_JAILBROKEN ) > 0
        { msg += "\n - Jailbroken"; }
        if ( tamper & DETECTED_DRM_DECRYPTED ) > 0
        { msg += "\n - Executable is not encrypted"; }
        if ( tamper & DETECTED_DEBUG_ATTACHED ) > 0
        { msg += "\n - App is debugged"; }
        if ( tamper & ( DETECTED_HASH_INFO_CORRUPTED | DETECTED_HASH_MODIFIED ) ) > 0
        { msg += "\n - App integrity corrupted"; }
        if ( tamper & ( DETECTED_CODESIGN_CORRUPTED | DETECTED_EXECUTABLE_CORRUPTED ) ) > 0
        { msg += "\n - App executable has corrupted"; }
        if ( tamper & DETECTED_CERTIFICATE_CHANGED ) > 0
        { msg += "\n - App has re-signed"; }
        if ( tamper & DETECTED_BLACKLIST_CORRUPTED ) > 0
        { msg += "\n - Blacklist/Whitelist has corrupted or missing"; }
        if ( tamper & DETECTED_CHEAT_TOOL ) > 0
        { msg += "\n - Cheat tool has detected"; }

        let alertController = UIAlertController(title: "AppSealing", message: msg, preferredStyle: .alert );
        alertController.addAction(UIAlertAction(title: "Confirm", style: .default,
            handler: { (action: UIAlertAction!) -> Void in
                #if !DEBUG // Debug mode does not kill app even if security threat has found
                    exit(0);
                #endif
            } ));
        DispatchQueue.main.async {
            self.window?.rootViewController?.present( alertController, animated: true, completion: nil )
        };
    }

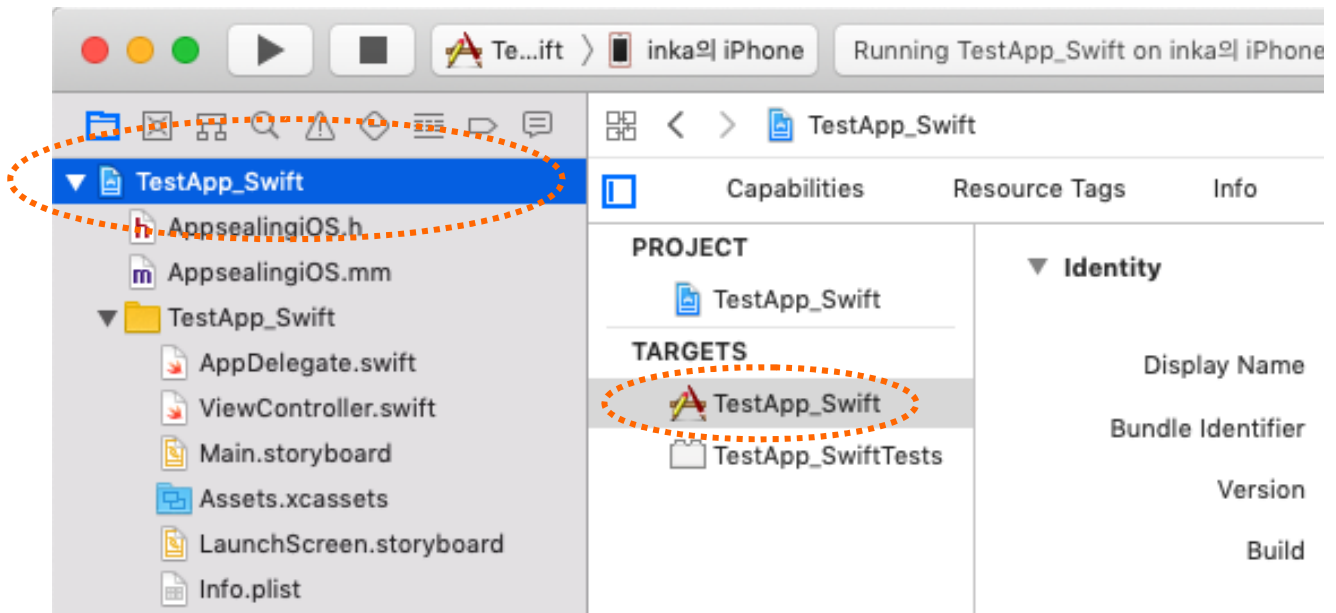
    AppSealingInterface._NotifySwizzlingDetected( { (msg: String?) -> () in
        let alertController = UIAlertController( title: "AppSealing Security", message: msg, preferredStyle: .alert );
        alertController.addAction( UIAlertAction( title: "Confirm", style: .default,
            handler: { ( action: UIAlertAction! ) -> Void in
                #if DEBUG
                #else
                    exit(0);
                #endif
            } ));
        DispatchQueue.main.async {
            self.window?.rootViewController?.present( alertController, animated: true, completion: nil )
        };
    });

    ##### AppSealing Code-Part END: DO NOT MODIFY THIS LINE !!!
    return true;
}
```

Part 4. 프로젝트 "Build Settings" 수정

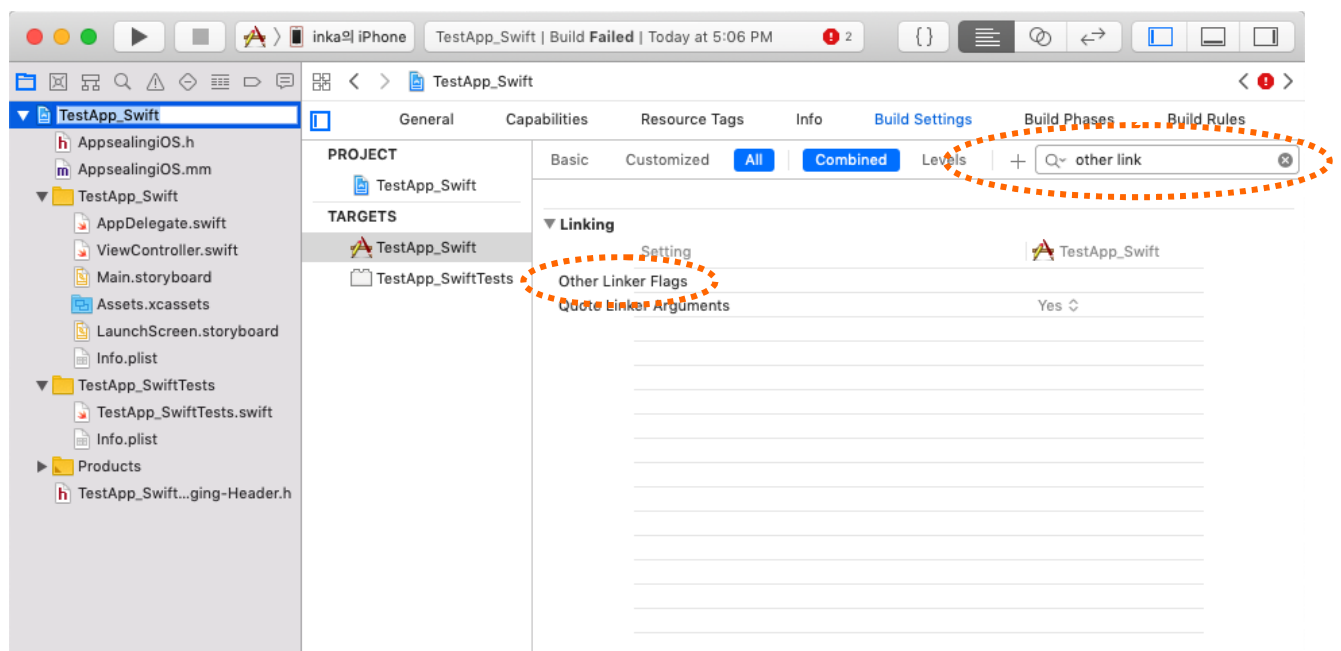
4-1 프로젝트 메인 target 선택

프로젝트 네비게이터에서 프로젝트를 선택하고 TARGETS에서 메인 타겟인 "TestApp_Swift"를 선택합니다.



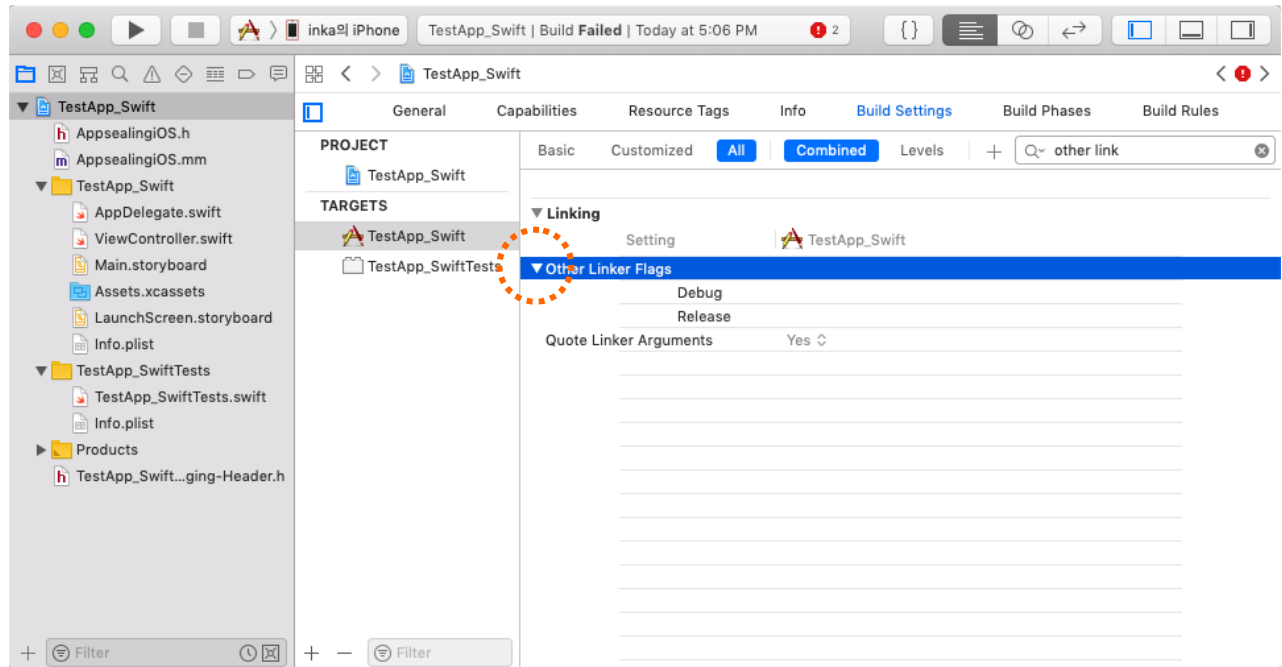
4-2 Build Settings "Other Linker Flags" 설정

프로젝트 선택 후 Build Settings에서 other link를 검색하십시오.



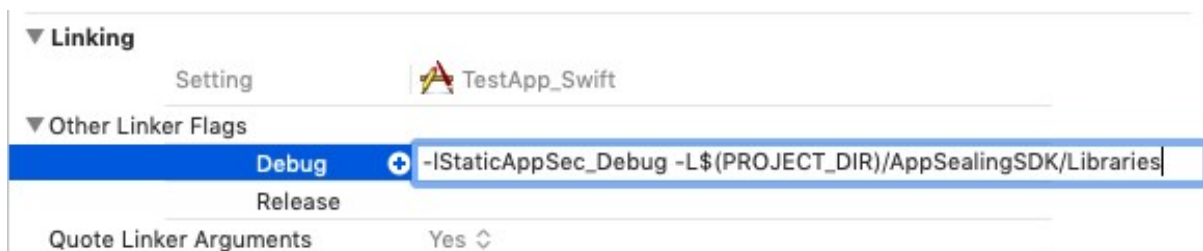
"Other Linker Flags" 필드가 나타나면 다음 단계를 수행하여 값을 설정하십시오.

1) "Other Linker Flags" 왼쪽 삼각형 아이콘을 눌러 "Other Linker Flags"를 확장하십시오.



2) "Debug" 아이템을 선택하고 클릭하여 아래 텍스트를 입력하십시오.

`-lStaticAppSec_Debug -L$(PROJECT_DIR)/AppSealingSDK/Libraries`



`$(PROJECT_DIR)` is an environment variable specifying the current project path. To replace this value with an absolute path, you must change the entire `$(PROJECT_DIR)`, not `PROJECT_DIR` in parentheses. For example, if the target project path is `/Users/myname/MyProject/SampleProject1`, the following string should be added to the value of Other Linker Flags.

`-L/Users/myname/MyProject/SampleProject1/AppSealingSDK/Libraries`

3) "Release" 아이템을 선택하고 클릭하여 아래 텍스트를 입력하십시오.

-IStaticAppSec -L\$(PROJECT_DIR)/AppSealingSDK/Libraries

▼ Linking	
Setting	 TestApp_Swift
▼ Other Linker Flags	<Multiple values>
Debug	-IStaticAppSec_Debug -L/Users/admin/Documents/TestApp_Swi...
Release	-IStaticAppSec -L/Users/admin/Documents/TestApp_Swift/AppS...
Quote Linker Arguments	Yes ⇅

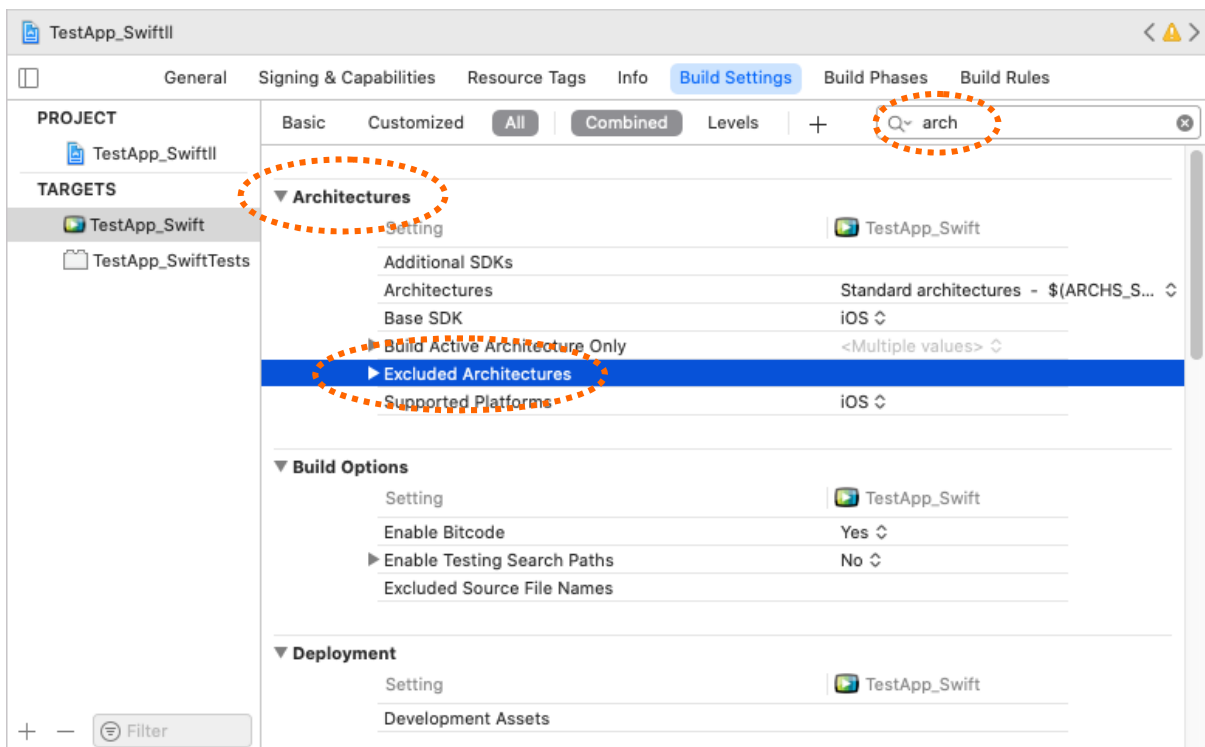
**** 주의: 옵션을 붙여 넣는 방법에 따라 아래와 같이 따옴표가 붙을 수 있습니다. 이 경우, 옵션을 띄어쓰기 단위로 나누어 입력하여 최종적으로 따옴표가 없어야 합니다.**

▼ Linking - General	
Setting	testSwift
▼ Other Linker Flags	<Multiple values>
Debug	"-IStaticAppSec_Debug -L/Users/kim-wontae/Desktop/project...
Release	"-IStaticAppSec -L/Users/kim-wontae/Desktop/projects/testS...
Quote Linker Arguments	Yes ⇅

다음은 Simulator에서 Release 빌드 실행이 필요한 경우를 위한 설정 내용입니다. Simulator에서 Debug 빌드만 실행하고 Release 빌드를 실행할 상황이 없다면 아래 단계는 진행하지 않아도 됩니다.

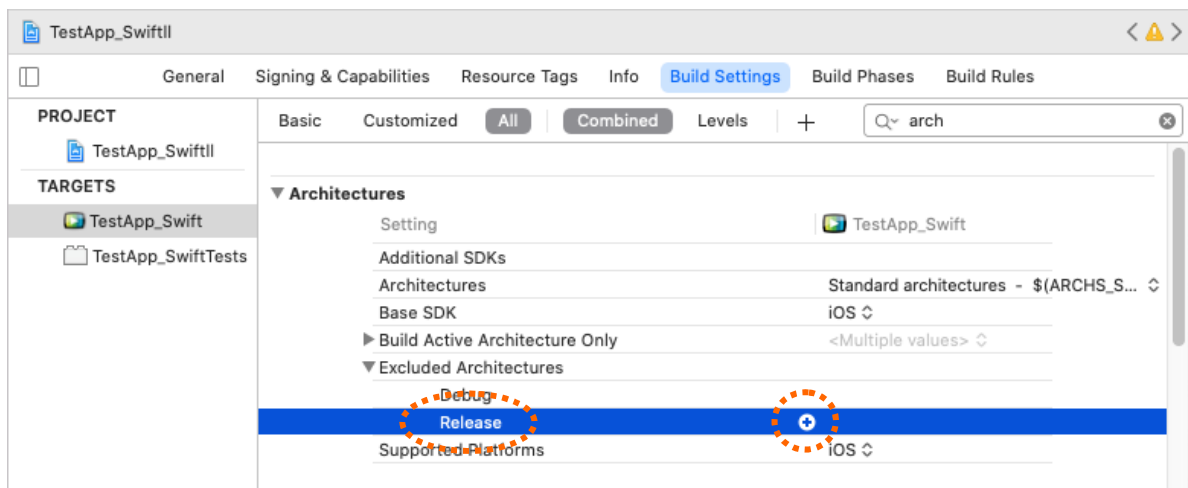
4-3 Build Settings "Architectures – Excluded Architectures" 설정

먼저 Build Settings에서 arch를 검색하십시오.

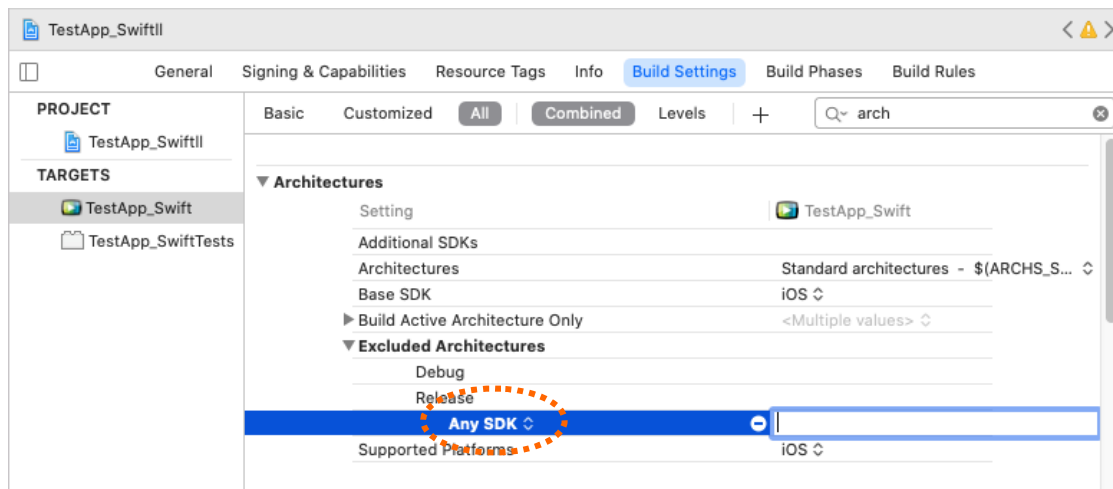


"Architectures" 필드가 나타나면 다음 단계를 수행하여 값을 설정하십시오.

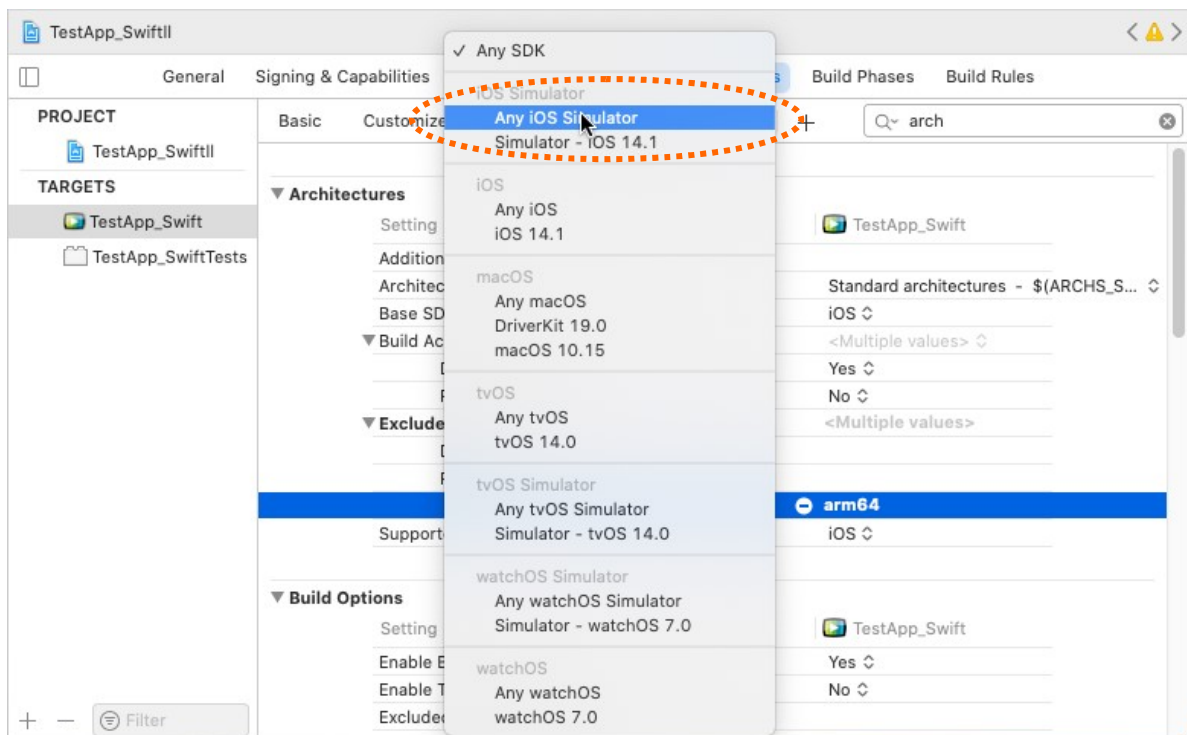
1) "Excluded Architectures" 왼쪽 삼각형 아이콘을 눌러 확장하십시오.



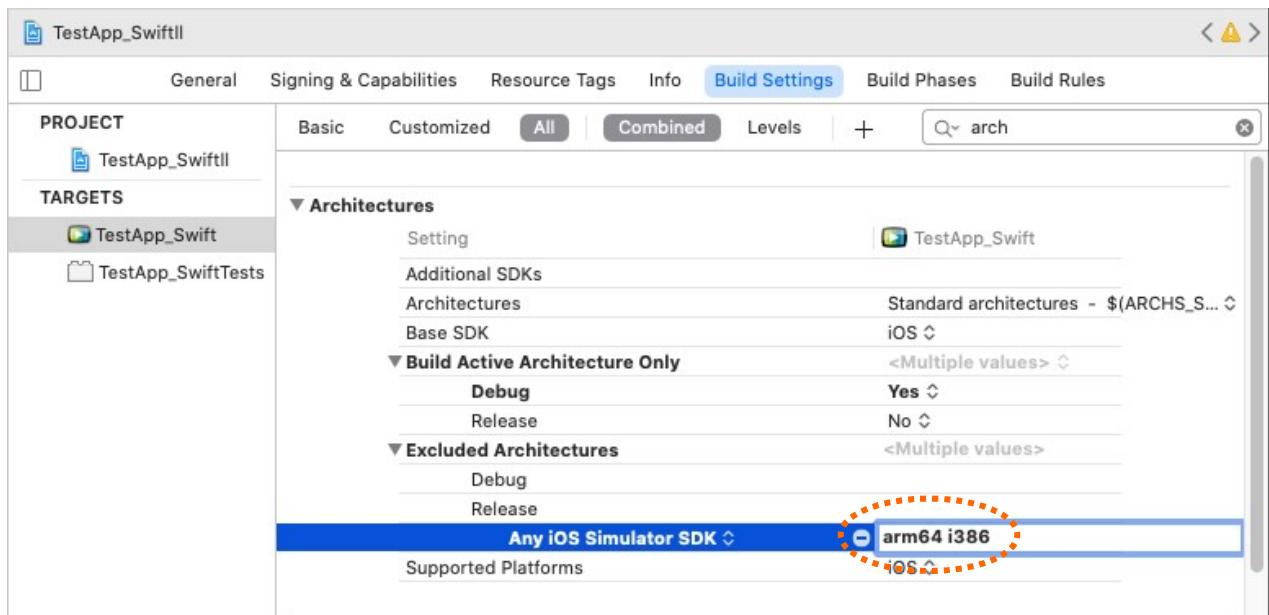
2) "Release" 항목을 선택하고 오른쪽의 "+" 아이콘을 클릭하십시오.



3) "Any SDK" 항목이 추가되면 클릭하여 팝업 메뉴를 띄우십시오.



4) 팝업 메뉴의 항목 중에서 **"Any iOS Simulator"**를 선택합니다.



5) "Any iOS Simulator SDK" 우측의 입력란에 **"arm64 i386"** 값을 입력합니다. (arm64와 i386 사이는 공백이 들어갑니다)

이제 AppSealing 보안 기능이 프로젝트에 추가되었습니다. 평소와 같이 'Build', 'Run' 또는 'Archive'를 수행하십시오.

4-4 앱 빌드 모드에 따른 주의 사항

* AppSealing은 디버그 모드와 릴리스 모드에서 다르게 동작합니다.

디버그 모드로 빌드 하는 경우 다음의 AppSealing 보안 기능은 원활한 개발을 위해서 비 활성화됩니다:

- 탈옥 감지 : Jailbreak detection
- 안티 디버깅 : Anti-debugging
- 안티 후킹 : Anti-hooking
- 실행 파일 암호화 감지 : Not encrypted executable file detection
- 앱 무결성 감지 : App-Integrity corruption detection
- 앱 재서명 감지 : Re-signing detection

이 기능들은 릴리스 모드로 빌드 하면 활성화됩니다.

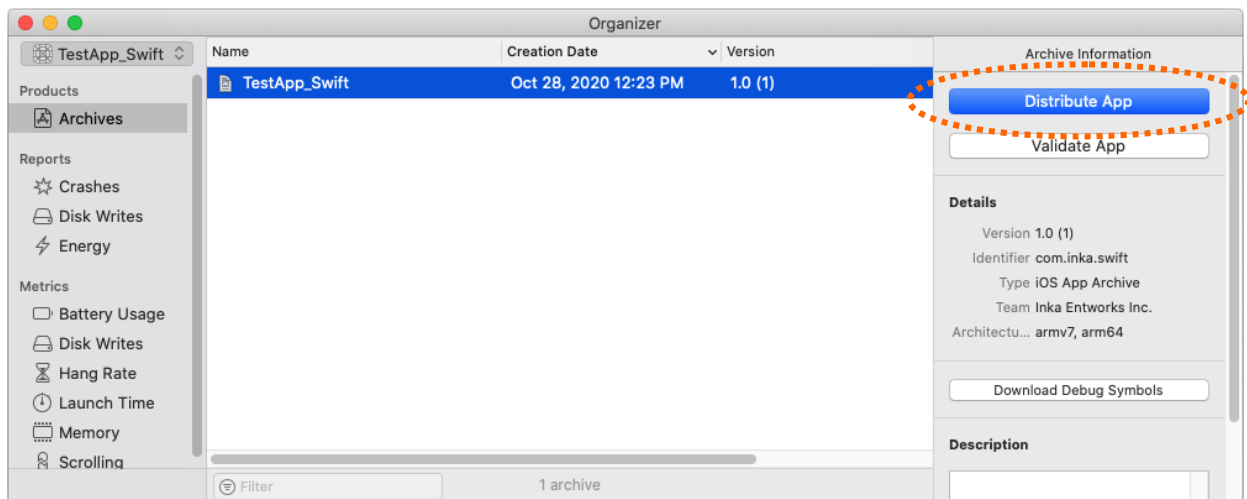
앱 스토어에 배포할 때 앱을 릴리스 모드로 빌드 하십시오. 릴리스 모드로 AppSealing을 테스트 할 경우 앱스토어 또는 TestFlight로 앱을 배포해야 합니다. 그렇지 않으면 실행 파일이 암호화되지 않은 것이 탐지되거나 앱이 변조된 것으로 탐지되어 일정 시간 후 앱이 종료됩니다.

4-5 앱 무결성 및 인증서 검증 정보 생성

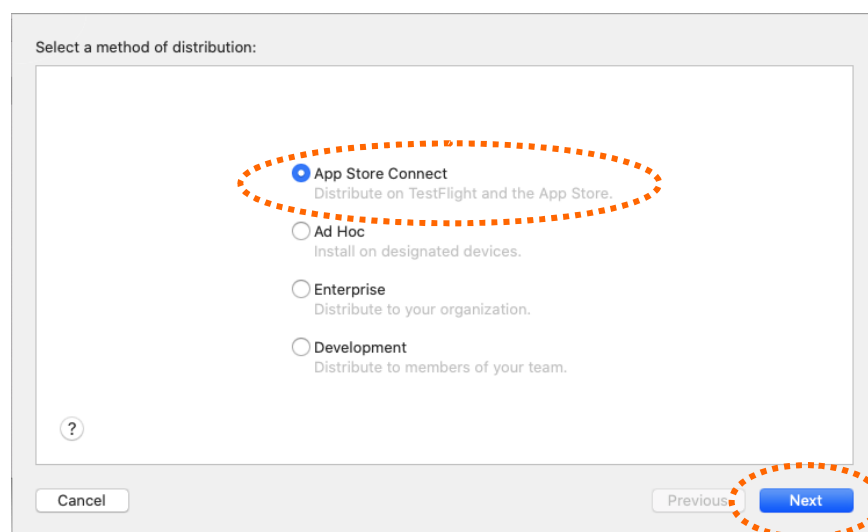
앱을 테스트 하거나 스토어에 배포할 경우 앱의 무결성 검증을 위한 추가 작업을 진행해야 합니다. 이 단계를 건너뛰는 경우 단말에서 앱을 실행하면 무결성 검증에 실패하여 일정 시간 후 앱이 종료됩니다.

TestFlight나 App Store 배포 시, 혹은 AdhocEnabled SDK를 사용하여 Ad Hoc 배포 시에는 반드시 이 단계를 수행해야 합니다.

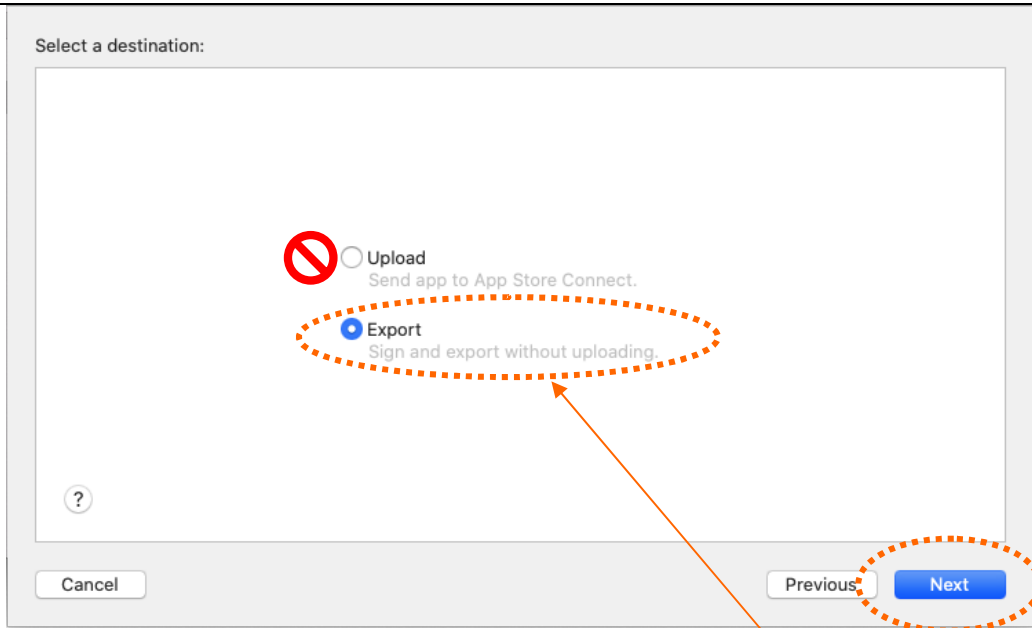
앱스토어나 TestFlight에 앱을 업로드 하기 위한 과정을 단계 별로 살펴 보겠습니다. 아래는 Archive가 완료된 상태의 Organizer 화면입니다.



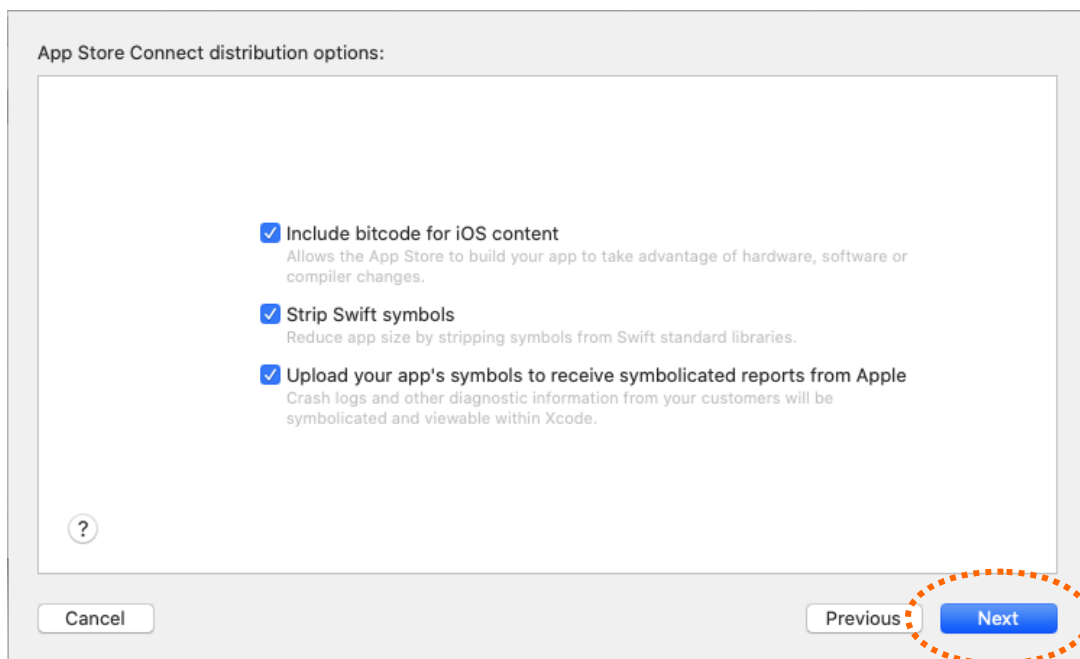
이 상태에서 업로드 할 IPA를 생성하기 위해 "Distribute App" 버튼을 클릭합니다.



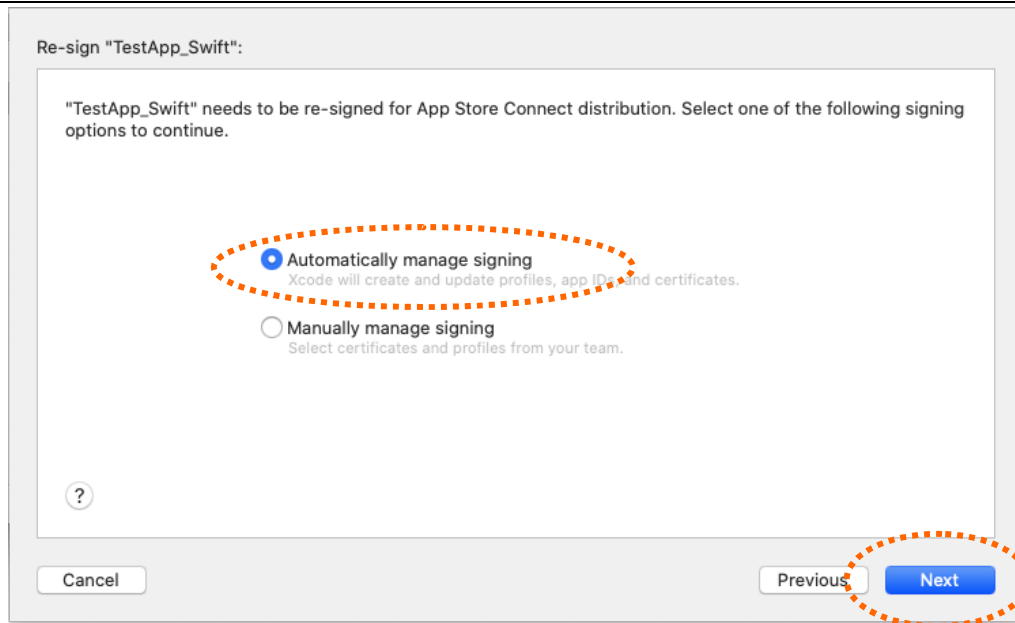
"App Store Connect"가 선택된 상태에서 "Next" 버튼을 클릭합니다. (AdhocEnabled SDK를 사용하는 경우 Ad Hoc도 선택이 가능합니다.)



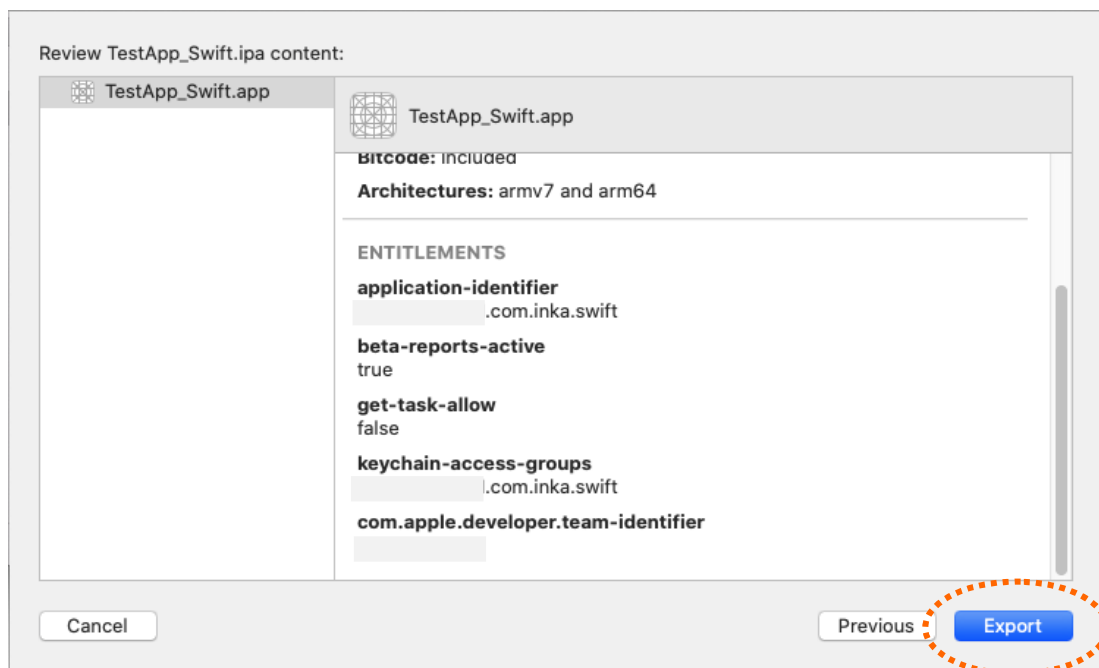
기존에는 "Upload"를 선택하는 경우가 대부분이지만 **AppSealing 적용을 위해서는 반드시 "Export"를 선택**해야 합니다. 이는 업로드 전 무결성 및 인증서 스냅샷 처리 작업이 필요하기 때문에 **이 과정이 잘못되거나 생략되면 앱이 정상적으로 실행되지 않으므로 유의해야 합니다.** "Export"가 선택된 상태에서 "Next" 버튼을 클릭합니다.



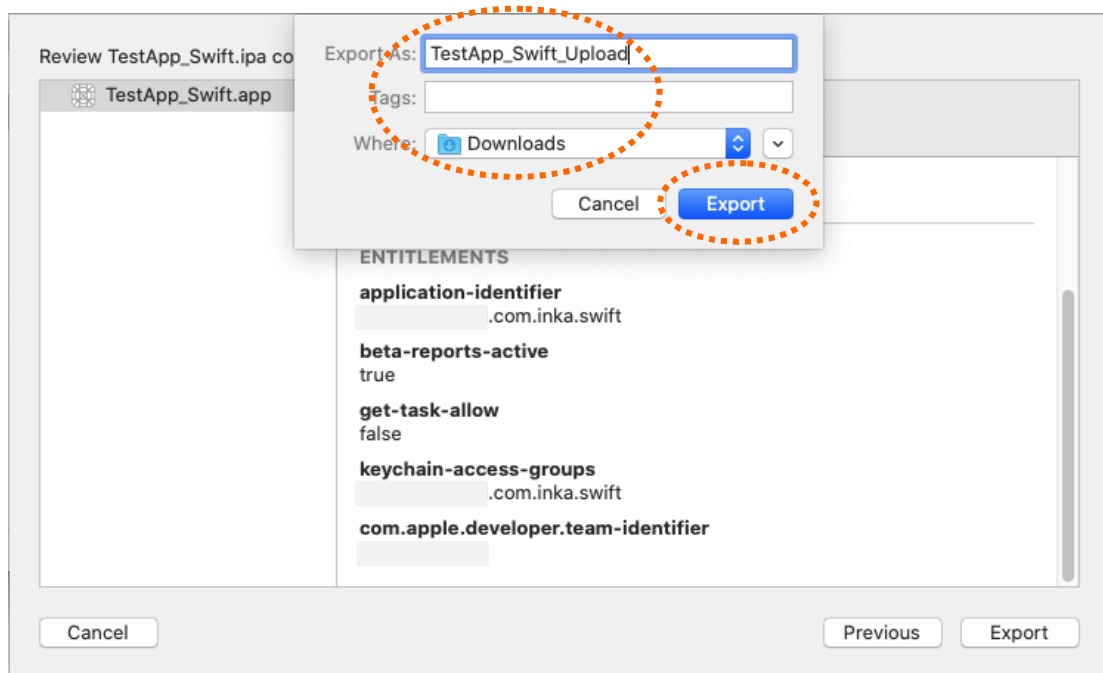
옵션 선택은 그대로 둔 상태로 "Next" 버튼을 클릭합니다.



옵션 선택은 그대로 둔 상태로 "Next" 버튼을 클릭합니다. 이제 아래와 같이 IPA 파일로 export 할 수 있는 창이 나타납니다.

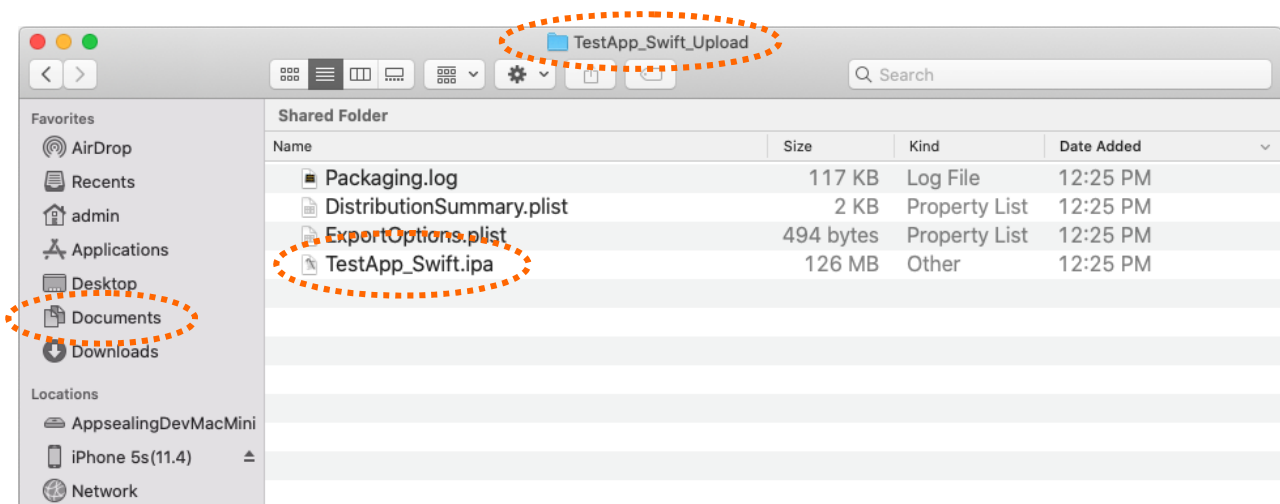


내용을 확인한 후 "Export" 버튼을 클릭합니다.

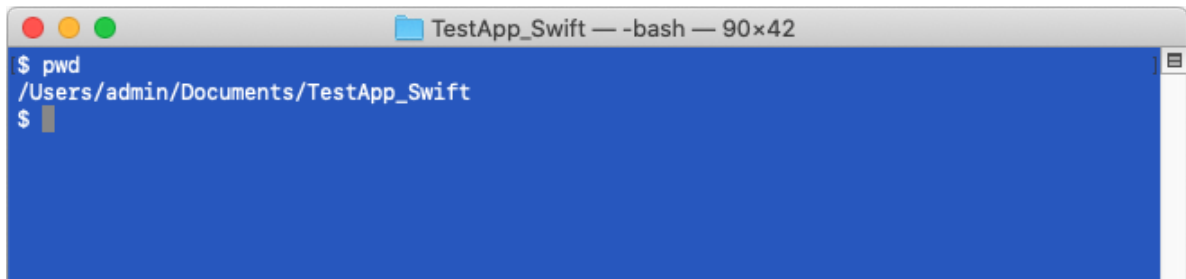


Export 위치를 지정하는 창이 표시되면 저장 위치를 선택하고 "Export"를 클릭합니다. 여기서는 ~/Downloads/TestApp_Swift_Upload 라는 이름의 폴더를 사용했습니다.

"Export"를 클릭하면 해당 폴더에 IPA 파일이 생성됩니다. 아래와 같이 Finder에서 확인할 수 있습니다. 이제 이 폴더의 위치를 잘 기억하거나 또는 Finder에서 해당 폴더로 이동한 후 창을 열어 놓습니다.



이제 Export된 IPA 파일을 가지고 작업을 진행해야 합니다. 터미널 앱을 실행하고 Xcode 프로젝트 폴더로 이동합니다.



```
TestApp_Swift — -bash — 90x42
$ pwd
/Users/admin/Documents/TestApp_Swift
$
```

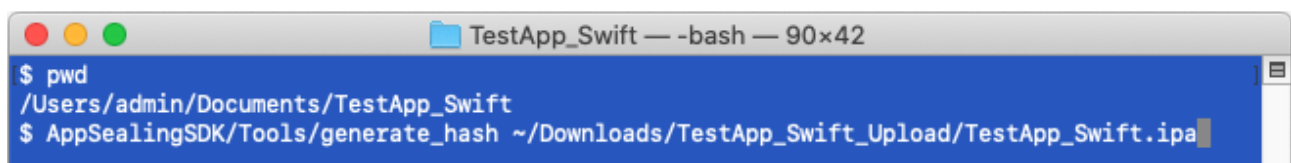
이 문서에서는 Xcode 프로젝트 경로로 ~/Documents/TestApp_Swift 를 사용했습니다. 위 그림처럼 해당 위치로 이동해서 pwd를 실행하면 경로가 출력되는 것을 확인할 수 있습니다.

다음 명령으로 Tools 폴더 안의 파일들에 실행 권한을 부여합니다.

```
$ chmod +x AppSealingSDK/Tools/*
```

이제 아래와 같이 generate_hash 스크립트를 실행합니다. 파라미터는 이전 단계에서 Export 했던 IPA 파일의 경로입니다. 경로를 직접 타이핑 하지 않고 열어 두었던 Finder 창에서 TestApp_Swift.ipa 파일을 터미널 창으로 드래그&드롭 해도 됩니다.

```
$ AppSealingSDK/Tools/generate_hash ~/Downloads/TestApp_Swift_Upload/TestApp_Swift.ipa
```



```
TestApp_Swift — -bash — 90x42
$ pwd
/Users/admin/Documents/TestApp_Swift
$ AppSealingSDK/Tools/generate_hash ~/Downloads/TestApp_Swift_Upload/TestApp_Swift.ipa
```

스크립트를 실행하면 아래 화면과 같이 진행 상황이 표시되면서 지정한 IPA에 무결성 검증 정보와 인증서 검증 정보를 추가하게 됩니다.

```
TestApp_Swift — -bash — 90x42
$ AppSealingSDK/Tools/generate_hash ~/Downloads/TestApp_Swift_Upload/TestApp_Swift.ipa

+-----+
| AppSealing IPA Hash Generator V1.0 : provided by INKA Entworks |
+-----+

[Target IPA]      = /Users/admin/Downloads/TestApp_Swift_Upload/TestApp_Swift.ipa

[Working Directory] = /var/folders/vd/qcxck8j5171d3ztm4m0wzjb80000gn/T/AppSealing/384e5429bee14b96a8d43f46f2824797/

1. Payload has extracted from the IPA ...
2. Trying to receive encryption key from AppSealing server ...
3. Successfully received encryption key ...
4. Generating app integrity/certificate snapshot ...
Executable=/private/var/folders/vd/qcxck8j5171d3ztm4m0wzjb80000gn/T/AppSealing/384e5429bee14b96a8d43f46f2824797/Package/Payload/TestApp_Swift.app/TestApp_Swift
5. Encrypting app integrity/certificate snapshot ...
6. Inserting app integrity/certificate snapshot into IPA ...
7. Codesigning your app using certificate used to sign your IPA ...
Executable=/private/var/folders/vd/qcxck8j5171d3ztm4m0wzjb80000gn/T/AppSealing/384e5429bee14b96a8d43f46f2824797/Package/Payload/TestApp_Swift.app/TestApp_Swift
Certificate="Apple Distribution: Inka Entworks Inc. ( )"
/var/folders/vd/qcxck8j5171d3ztm4m0wzjb80000gn/T/AppSealing/384e5429bee14b96a8d43f46f2824797/Package/Payload/TestApp_Swift.app/: replacing existing signature
8. Rebuilding & re-signing IPA ...

>>> All processes have done successfully .....

$
```

이 과정은 "Ad Hoc", "Enterprise", "Development"로 배포할 경우에도 동일하게 적용됩니다.

4-6 메소드 스위즐링 / 메소드 후킹 감지 기능 제어

메소드 스위즐링은 특정 클래스의 기존 메소드의 기능을 새로운 메소드의 기능으로 교환하여 원래의 메소드를 호출할 때 실제로는 새로운 메소드의 코드가 실행되도록 하는 기술로 주로 Objective-C에서 사용되며 이를 통해 기존 코드의 동작을 유연하게 수정하거나 확장할 수 있게 해줍니다.

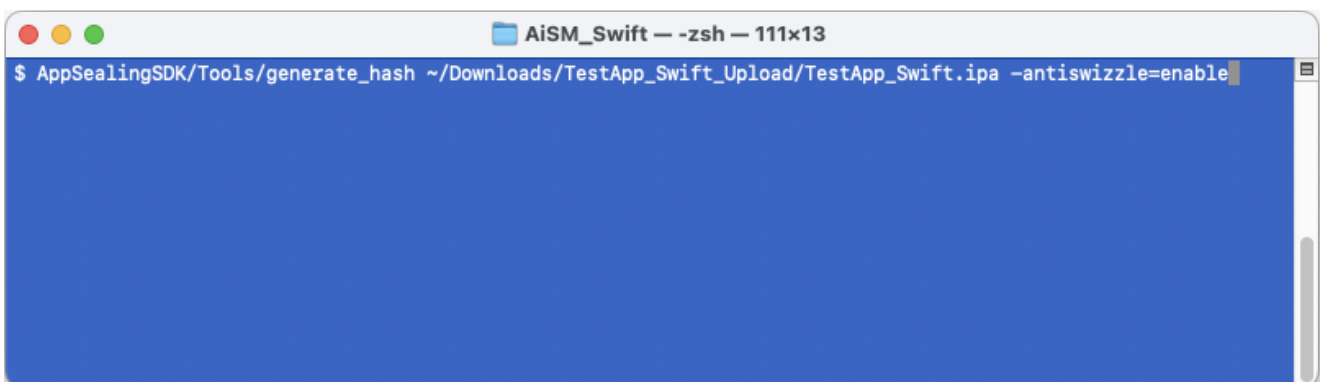
메소드 스위즐링은 유용한 기술이지만 공격자는 메소드 스위즐링을 통해 기존의 메소드를 자신의 악성 코드로 교환할 수도 있습니다. 예를 들어 사용자의 입력을 처리하는 메소드를 수정하여 데이터를 탈취하거나, 원하지 않는 행동을 실행하도록 만들 수 있습니다.

또한 시스템의 로깅 메소드를 스위즐링하여, 중요한 이벤트가 기록되지 않도록 할 수 있고 이를 통해 악의적인 행동이 감지되지 않게 할 수 있습니다.

AppSealing SDK에는 이러한 메소드 스위즐링 동작을 감지할 수 있는 기능이 추가되어 있지만 메소드 스위즐링이 항상 공격의 수단으로만 사용되는 것이 아니기에 이를 감지할 것인지 하지 않을 것인지를 직접 지정할 수 있습니다.

기본적으로 오동작을 막기 위해 메소드 스위즐링 감지 기능이 비활성화 된 상태로 SDK가 배포되지만 generate_hash 스크립트를 실행하는 과정에서 이를 활성화 시킬 수 있습니다. 만약 앱 테스트 단계에서 메소드 스위즐링이 감지되었다는 보안 경고가 발생할 경우 앱에 포함된 다른 라이브러리에서 의도적으로 메소드 스위즐링을 사용하는 경우가 있을 수 있기 때문에 이 경우는 메소드 스위즐링 감지를 활성화 시키면 안 됩니다.

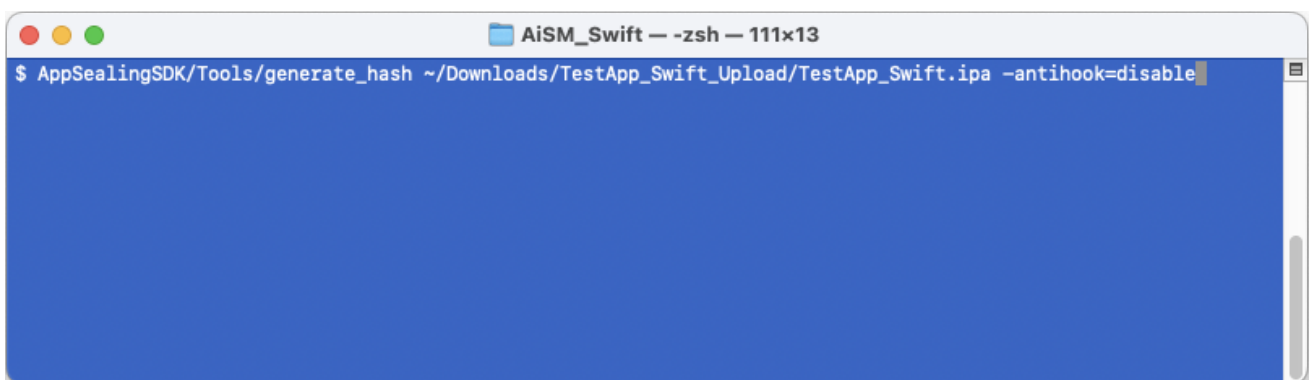
generate_hash 스크립트 실행 시 메소드 스위즐링 감지 기능을 활성화 하려면 다음과 같이 **antiswizzle=enable** 파라미터를 추가한 상태로 generate_hash를 실행하면 됩니다.

A screenshot of a macOS terminal window titled "AiSM_Swift - zsh - 111x13". The terminal shows a command being executed: "\$ AppSealingSDK/Tools/generate_hash ~/Downloads/TestApp_Swift_Upload/TestApp_Swift.ipa -antiswizzle=enable". The terminal background is blue, and the text is white. The command is entered at the prompt, and the cursor is at the end of the line.

```
$ AppSealingSDK/Tools/generate_hash ~/Downloads/TestApp_Swift_Upload/TestApp_Swift.ipa -antiswizzle=enable
```

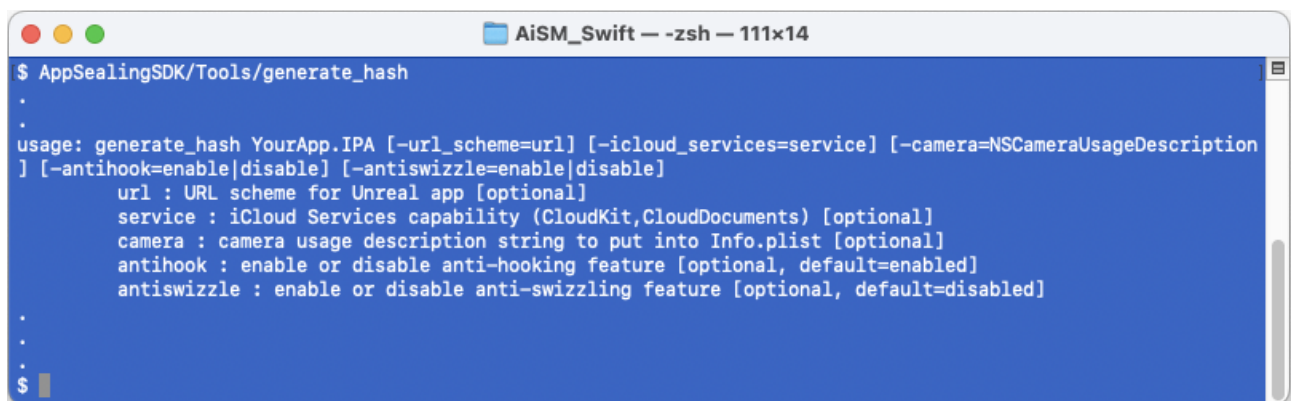
메소드 스위즐링과 함께 사용되는 공격 방법으로 메소드 후킹이 있습니다. 메소드 후킹은 메소드 스위즐링과 유사하지만 메소드를 서로 바꾸는 것이 아니라 메소드의 실행 흐름을 직접 조작하고 제어한다는 차이점이 있습니다. 메소드 스위즐링은 필요에 따라 앱이나 특정 라이브러리에서 사용될 수 있지만 정상적인 앱과 라이브러리에서 메소드 후킹이 사용되는 경우는 없기 때문에 AppSealing SDK에는 메소드 후킹 감지 기능이 기본으로 활성화되어 있지만 필요한 경우 이 기능을 비활성화 시킬 수 있습니다.

메소드 후킹 감지 기능을 비활성화 하려면 다음과 같이 `antihook=disable` 파라미터를 추가하여 `generate_hash` 스크립트를 실행하면 됩니다.



```
AiSM_Swift — -zsh — 111x13
$ AppSealingSDK/Tools/generate_hash ~/Downloads/TestApp_Swift_Upload/TestApp_Swift.ipa -antihook=disable
```

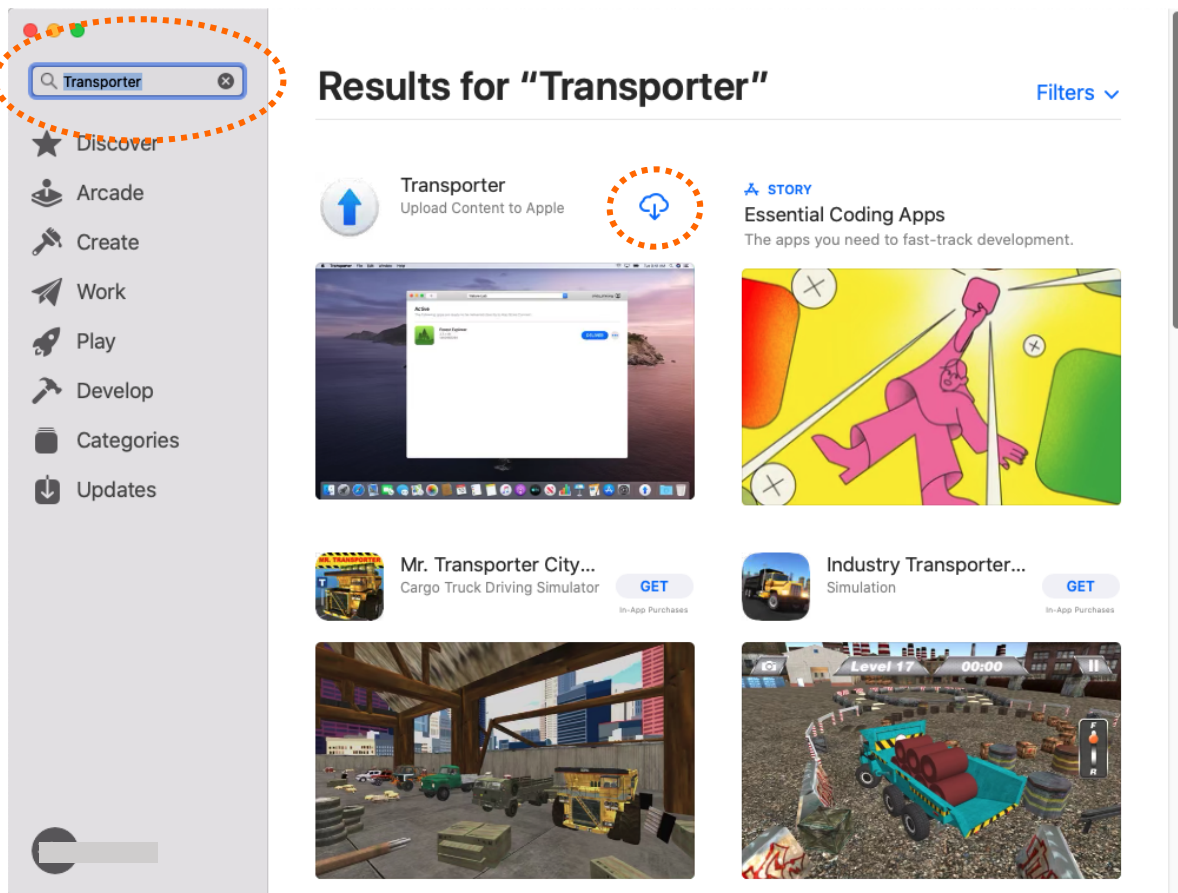
`generate_hash` 스크립트의 모든 파라미터를 확인하고 싶을 경우는 IPA 경로명 없이 스크립트만 실행하면 아래 화면과 같이 사용 가능한 모든 파라미터를 확인할 수 있습니다.



```
AiSM_Swift — -zsh — 111x14
$ AppSealingSDK/Tools/generate_hash
.
usage: generate_hash YourApp.IPA [-url_scheme=url] [-icloud_services=service] [-camera=NSCameraUsageDescription]
[-antihook=enable|disable] [-antiswizzle=enable|disable]
    url : URL scheme for Unreal app [optional]
    service : iCloud Services capability (CloudKit,CloudDocuments) [optional]
    camera : camera usage description string to put into Info.plist [optional]
    antihook : enable or disable anti-hooking feature [optional, default=enabled]
    antiswizzle : enable or disable anti-swizzling feature [optional, default=disabled]
.
.
.
$
```

4-7 재서명 앱 App Store Connect 업로드

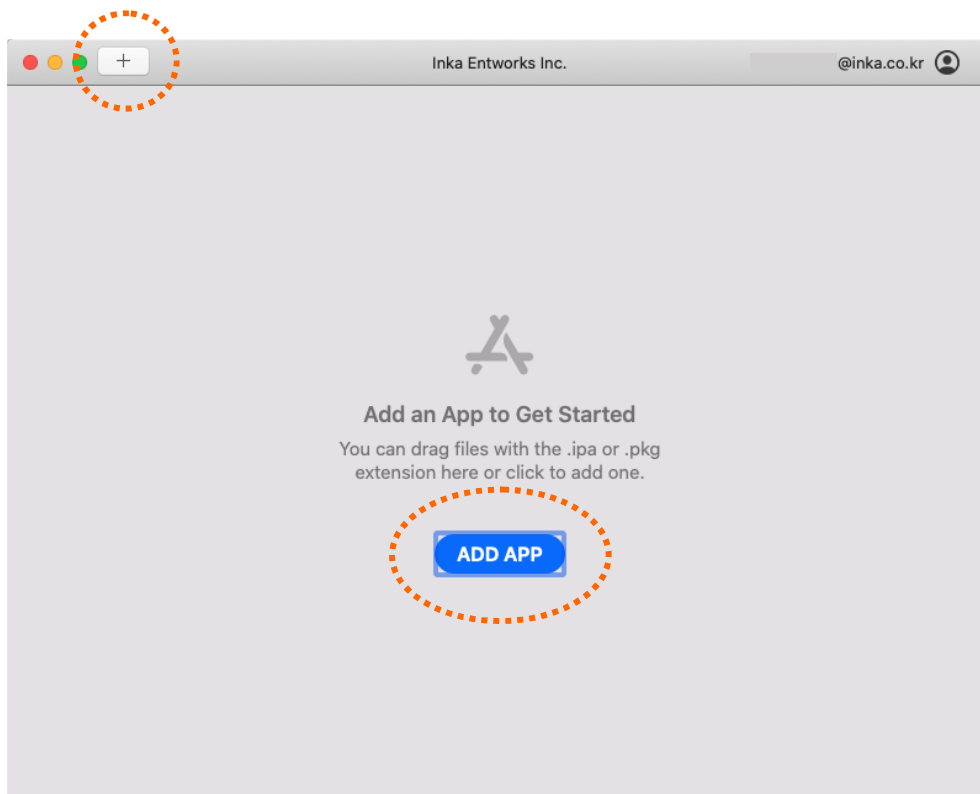
이제 재서명이 완료된 IPA를 App Store Connect에 업로드 합니다. (물론 AdhocEnabled SDK를 사용한 Ad Hoc 배포라면 이 과정이 필요 없습니다.) 간편한 업로드를 위해 Transporter 앱을 사용하겠습니다. 맥 OS에 Transporter가 설치되어 있지 않은 경우 맥 AppStore를 실행하고 "Transporter"를 검색하여 설치합니다.

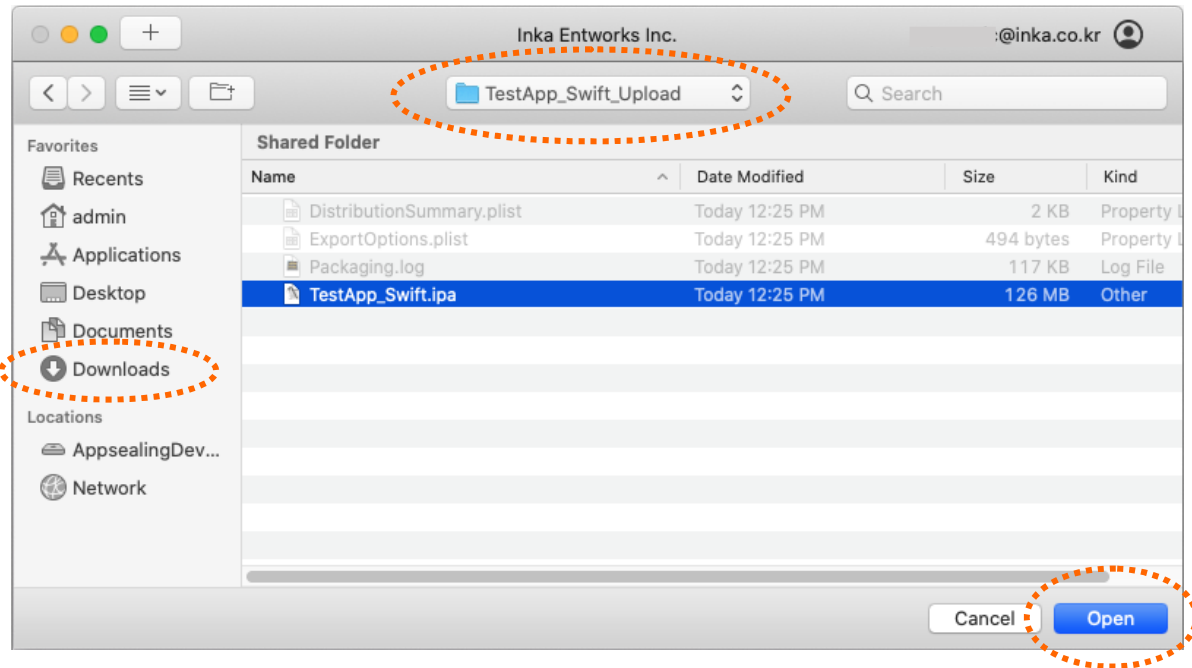


설치를 완료하고 실행하면 아래와 같이 Apple ID를 입력하는 창이 나타납니다. Apple 계정 ID와 비밀번호를 입력합니다. (이 과정은 최초 1회만 수행하면 됩니다)

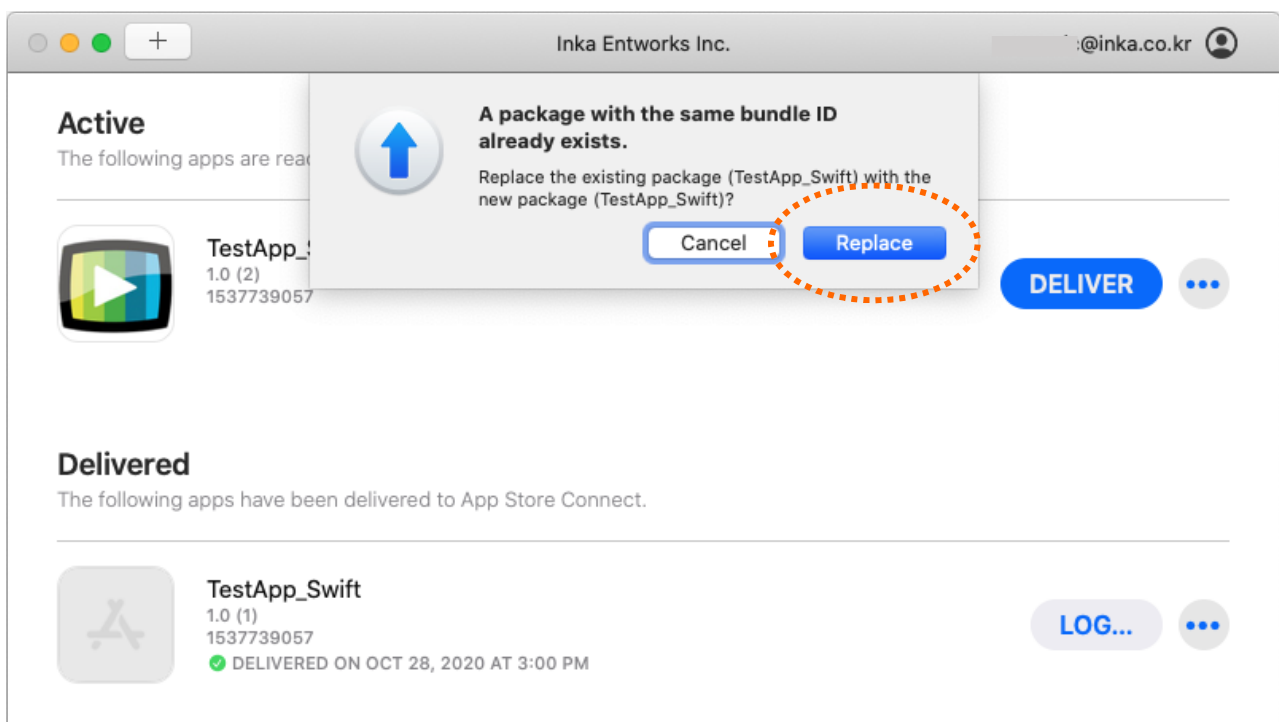


ID와 비밀번호를 입력하고 로그인 하면 아래와 같은 창이 나타나며 여기서 업로드 할 IPA를 선택하기 위해 좌측 상단의 + 버튼 또는 창 중앙의 "ADD APP" 버튼을 클릭하고 이전 단계에서 재서명이 완료된 IPA 파일을 선택합니다.

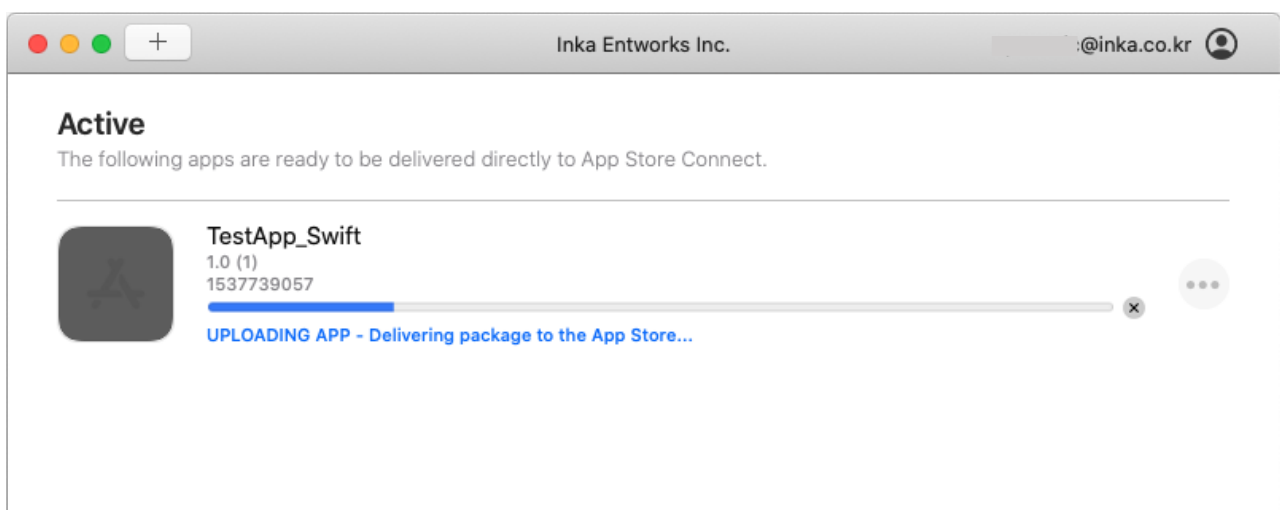
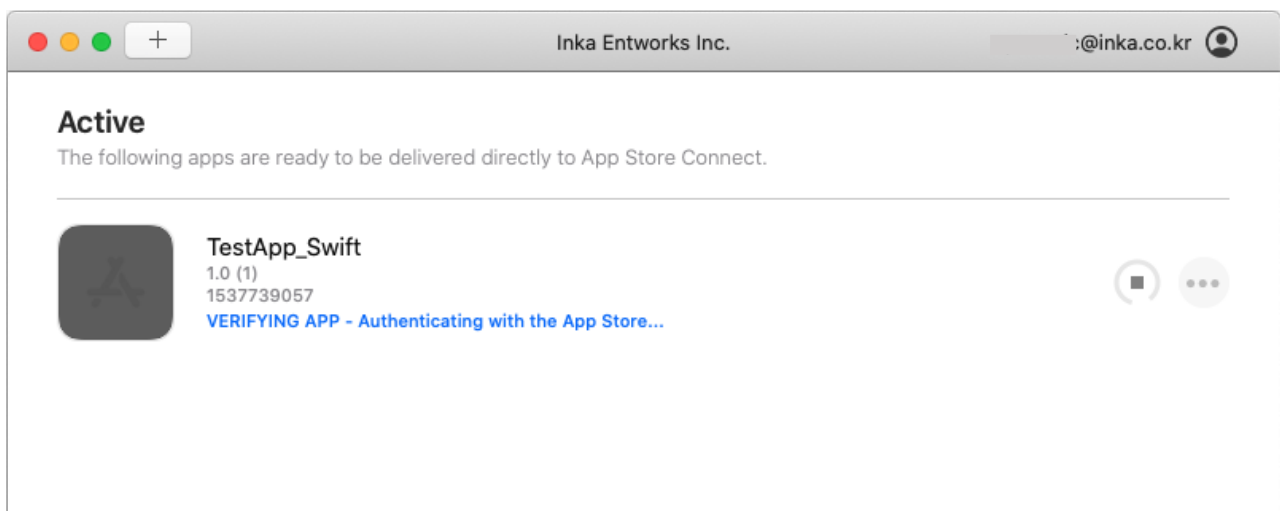
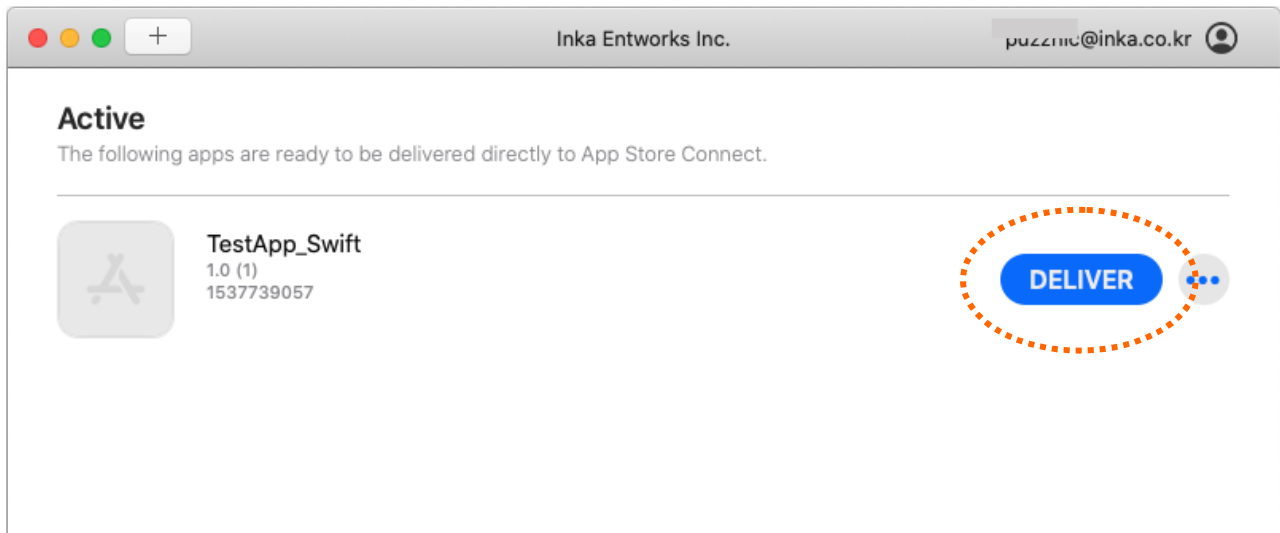


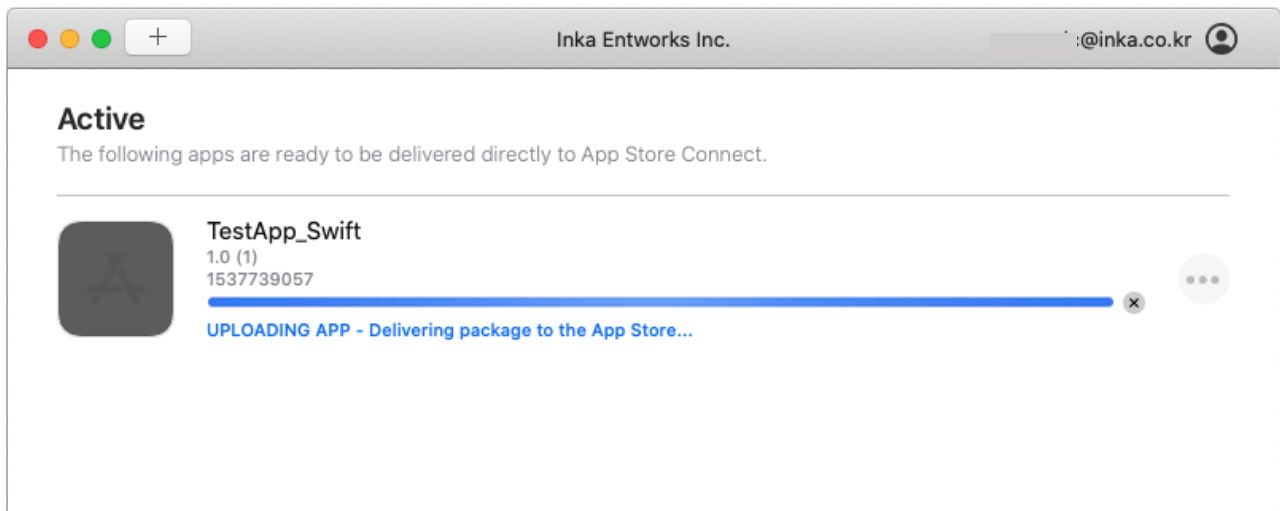


앱 최초 전송 이후 앱의 버전이나 빌드 번호를 증가시키고 새로 업로드 하려고 할 경우 IPA 파일을 추가하면 번들 ID가 동일하기 때문에 아래와 같은 확인 창이 나타날 수 있습니다. 이 경우 "Replace" 버튼을 클릭합니다.

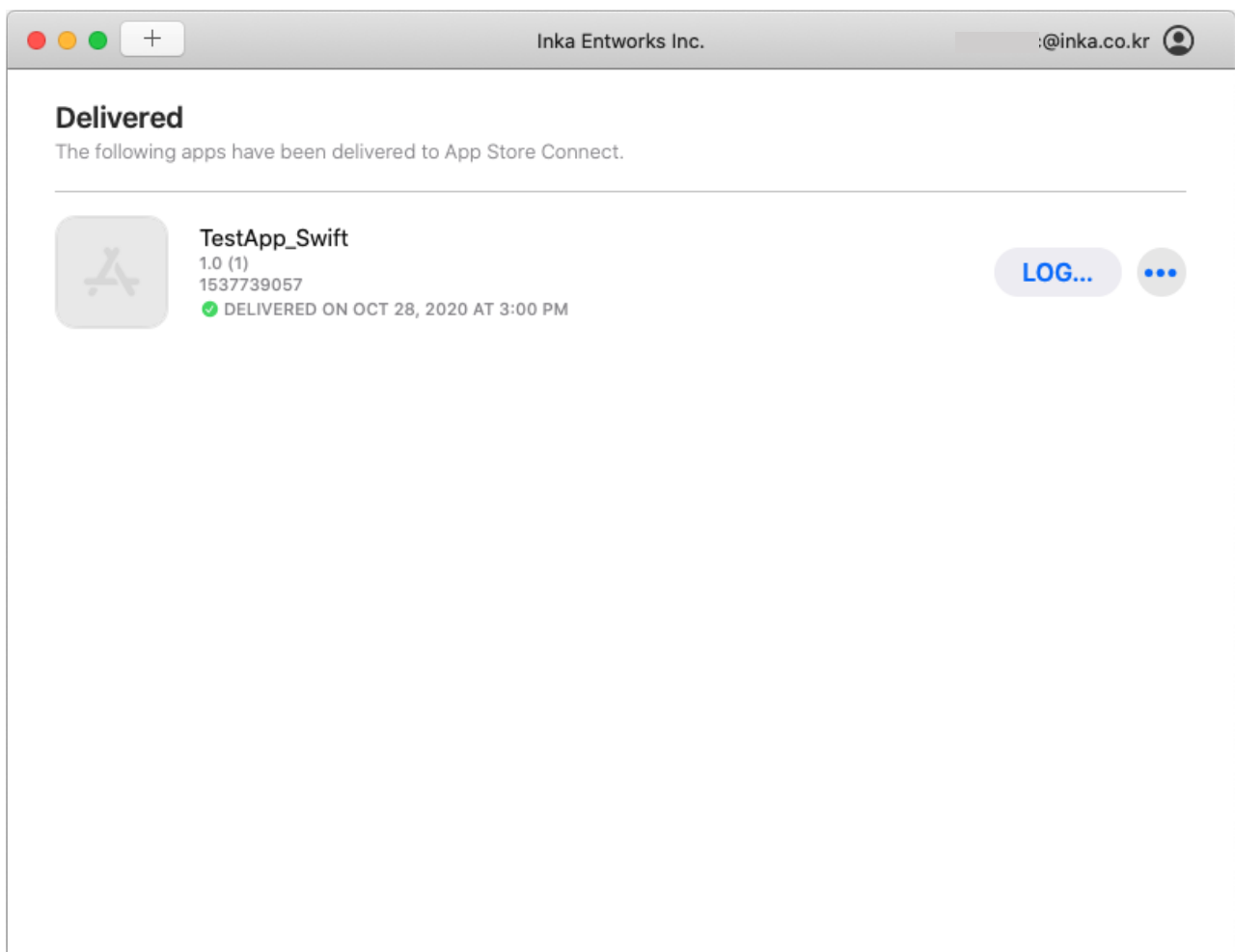


IPA가 추가된 상태에서 "DELIVER" 버튼을 클릭하면 앱 검증과 App Store Connect 업로드 과정이 진행됩니다.





아래 화면과 같이 전송이 완료되면 App Store Connect에서 심사를 위한 빌드 제출이나 TestFlight용 빌드 배포를 진행할 수 있습니다.



Part 5. AppSealing 기기 고유 ID 획득

AppSealing SDK는 기기 별로 고유 ID를 생성하고 관리합니다. AppSealing SDK를 사용하는 고객은 필요한 경우 AppSealing에서 제공하는 인터페이스를 이용하여 기기 고유 ID를 확인할 수 있습니다. 확인된 고유 ID는 AppSealing에서 제공하는 해킹 데이터 서비스와 연계하여 비즈니스에 활용할 수 있습니다.

5-1 기기 고유 ID 확인

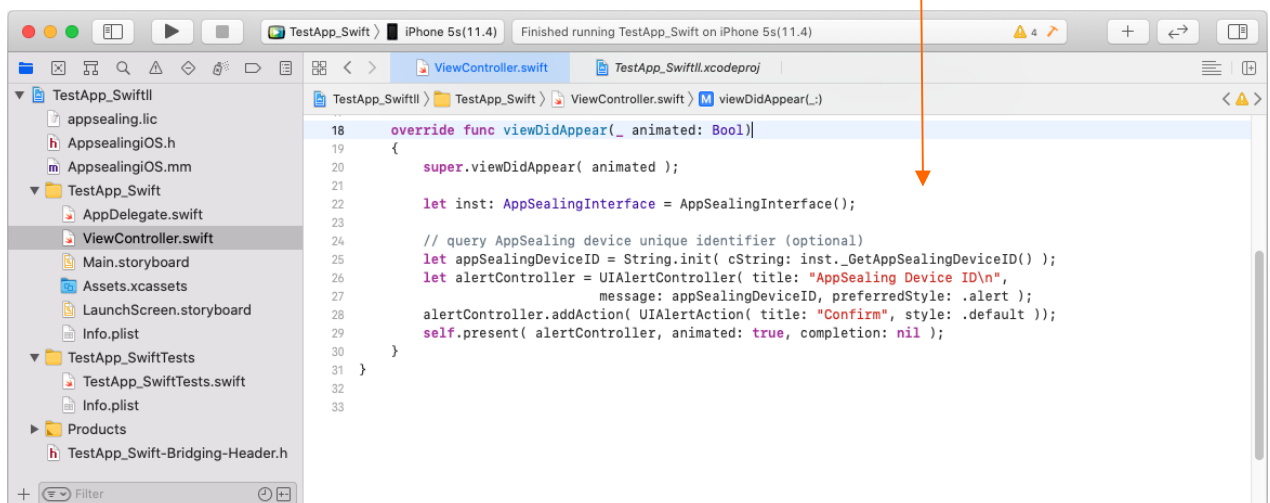
프로젝트의 "ViewController.swift" 파일을 열고 다음 코드를 삽입하십시오. 만약 ViewController.swift 파일에 'viewDidAppear' 메서드가 이미 있다면 'super.viewDidAppear(animated);' 줄 아래에 다음 코드를 삽입하십시오.)

Simple UI code into 'ViewController.swift' for **Swift** project

```
override func viewDidAppear(_ animated: Bool)
{
    super.viewDidAppear( animated );

    let inst: AppSealingInterface = AppSealingInterface();
    let appSealingDeviceID = String.init( cString: inst._GetAppSealingDeviceID() );
    let alertController = UIAlertController( title: "AppSealing DeviceID",
                                           message: appSealingDeviceID, preferredStyle: .alert );

    alertController.addAction( UIAlertAction( title: "Confirm", style: .default ));
    self.present( alertController, animated: true, completion: nil );
}
```



프로젝트가 Objective-C 기반인 경우 다음 코드를 사용하여 UI를 표시할 수 있습니다.

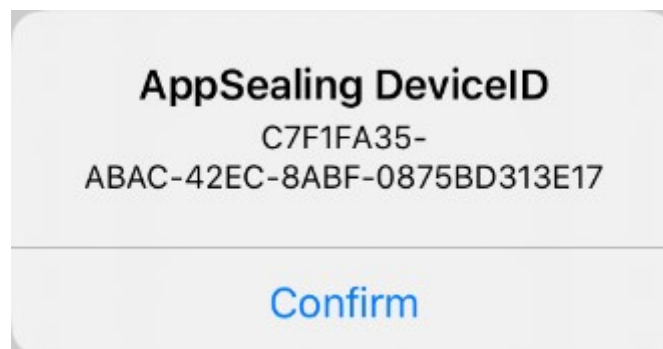
Simple UI code into 'ViewController.mm' for **Objective-C** project

```
#include "AppsealingiOS.h"

char _appSealingDeviceID[64];
- (void)viewDidAppear:(BOOL)animated
{
    [super viewDidAppear:animated];

    if ( ObjC_GetAppSealingDeviceID( _appSealingDeviceID ) == 0 )
    {
        UIAlertController *alert = [UIAlertController alertControllerWithTitle:@"AppSealing DeviceID"
                                     message:[NSString alloc] initWithUTF8String:_appSealingDeviceID
                                     preferredStyle:UIAlertControllerStyleAlert];
        UIAlertAction *confirm = [UIAlertAction actionWithTitle:@"Confirm"
                                                                style:UIAlertActionStyleDefault
                                                                handler:^(UIAlertAction * _Nonnull action) { }];
        [alert addAction:confirm];
        [self presentViewController:alert animated:YES completion:nil];
    }
}
```

앱을 실행하면 아래와 같이 메시지 박스에서 기기 고유 ID를 확인할 수 있습니다.



Part 6. 앱 서버를 이용한 강화된 탈옥 감지 적용 방법

6-1 강화된 탈옥 감지 방안의 개요 및 필요성

AppSealing SDK의 주요 기능 중 탈옥된 단말의 환경을 감지하여 강제로 앱을 종료시키는 기능이 포함되어 있습니다. 그러나 더욱 고도화된 공격 방법에 의해 이러한 탐지 기능이 우회될 가능성이 존재합니다. 이는 iOS의 운영체제 특성 상 앱 실행 시 로드된 동적 라이브러리(dylib)의 코드가 먼저 실행되기 때문인데 공격자는 이러한 동적 라이브러리에 실행 파일의 특정 영역을 패치하는 코드를 담아 배포하는 경우가 있기 때문입니다.

AppSealing의 탐지 로직이 실행되기 이전 시점에 이러한 코드 패치가 일어날 경우 앱을 종료시키는 코드가 제거된 상태가 때문에 탈옥이 감지되더라도 앱은 계속 실행 중인 상태가 됩니다.

물론 이런 유형의 공격을 누구나 쉽게 할 수 있는 것은 아니지만 전문적인 해킹 지식을 가진 해커 그룹에 의해 이런 방식의 공격이 실제 이루어진 상황이 확인되었으므로 AppSealing은 이런 공격 상황에도 대비할 수 있는 추가적인 탈옥 감지 방법을 제공합니다.

이 공격 방식의 특징은 실행 중인 앱의 코드를 사전에 변경시켜 버리는 것이기 때문에 AppSealing 라이브러리 자체에 아무리 강력한 탐지 로직을 추가하더라도 동적 라이브러리에 의해 코드가 패치 되는 상황은 피할 수 없게 됩니다. 따라서 신규로 제공하는 탈옥 감지 기능은 앱에서 감지를 하는 것이 아니라 앱과 연동되는 서버에서 탈옥이 의심되는 단말일 경우 로그인이나 API 호출 수행 등의 모든 서비스와 작업을 거부하는 방식으로 이루어집니다.

기본적인 방식은 앱에서 AppSealing 인터페이스를 통해 서버 자격 증명(credential)을 가져오고 이를 기존의 인증 정보에 추가하여 서버로 전송하면 서버가 이 값의 진위 여부를 검증하여 서비스를 지속하거나 거부하는 등의 동작을 하도록 하는 것입니다.

서버와 연동되지 않는 클라이언트 단독 앱일 경우 이 방법을 적용할 수 없습니다.

다음 절에서 서버 자격 증명을 얻고 이를 검증하기 위한 방법을 예제 코드와 함께 설명합니다.

6-2 iOS 앱 코드 작업

서버 자격 증명 검증을 위해 앱에서 추가해야 할 작업은 AppSealing SDK의 함수를 호출하여 서버 자격 증명 문자열을 가져온 뒤 이를 기존의 인증 정보와 함께 서버로 전송하는 것입니다.

서버와 연동되는 앱의 경우 대부분 사용자 인증이나 로그인 과정을 거칠 것이고 이 과정에서 사용자가 입력한 계정 정보를 서버로 전송할 것입니다. 이때 서버로 전송하는 파라미터에 서버 자격 증명 문자열을 추가하면 됩니다.

서버 자격 증명 문자열은 다음과 같은 방법으로 가져옵니다.

Simple UI code into 'ViewController.swift' for **Swift** project

```
func userLogin( userID: String, password: String ) -> Bool
{
    let inst: AppSealingInterface = AppSealingInterface();
    let appSealingCredential = String.init( cString: inst._GetEncryptedCredential() );
    // credential 값을 인증 정보와 함께 서버로 올린다
    let loginResult = processLogin( user: userID, pwd: password, credential: appSealingCredential );
    ...
    ...
}
```

Simple UI code into 'ViewController.mm' for **Objective-C** project

```
- (BOOL)userLogin:(NSString*)userID withPassword:(NSString*)password
{
    char _appSealingCredential[290] = { 0, };
    ObjC_GetEncryptedCredential( _appSealingCredential );
    // credential 값을 인증 정보와 함께 서버로 올린다
    BOOL loginResult = processLogin( userID, password, _appSealingCredential );
    ...
    ...
}
```

서버에서 자격 증명 검증에 실패할 경우 로그인도 실패 처리하고 이후 앱이 더 이상 진행되지 않도록 구성해야 합니다. 그러나 앱에서 결과를 확인하고 앱을 종료하는 등의 코드는 공격자에 의해 변조될 가능성이 있기 때문에 가장 좋은 방법은 서버에서 자격 증명 검증에 실패한 이후 해당 클라이언트의 어떤 요청에 대해서도 서비스나 응답을 거부하도록 서버를 구성하는 것입니다.

이에 대해서는 다음 절에서 다시 설명됩니다.

6-3 서버 측 검증 작업

AppSealing 모듈에서 인터페이스 호출을 통해 반환되는 자격증명 데이터(credential)는 AppSealing 내부의 보안 로직이 정상적으로 수행되고 단말에서 위험 상황이 감지되지 않았을 경우만 정상적인 값이 리턴 됩니다.

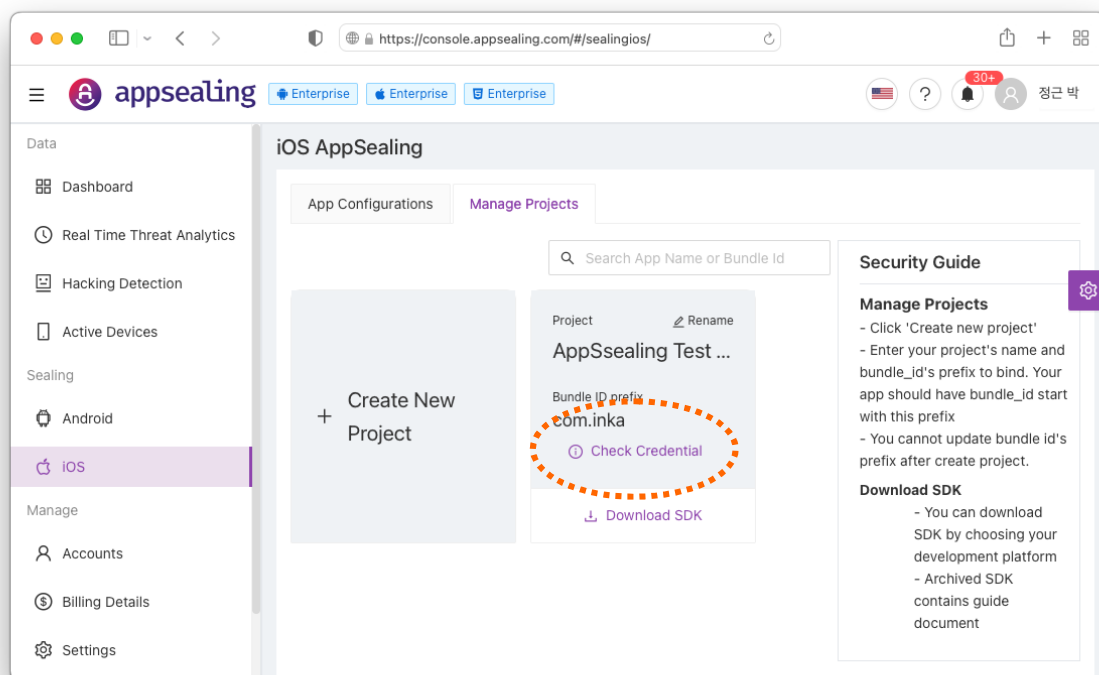
만약 동적 라이브러리를 통한 코드 패치 공격이 이루어졌거나 기타 다른 방법으로 보안 로직을 우회했다면 정상적인 자격증명 데이터가 생성되지 않을 것이기 때문에 서버에서 이를 검증하여 단말의 공격 상황을 차단할 수 있게 됩니다.

앱 서버는 클라이언트(앱)가 전송한 자격 증명 값이 올바른 것인지 확인하고 만약 올바르지 않다면 인증(로그인)을 거부한 후 이후에 해당 클라이언트가 요청하는 모든 서비스(API 호출)에 대해서도 작업을 거부해야 합니다.

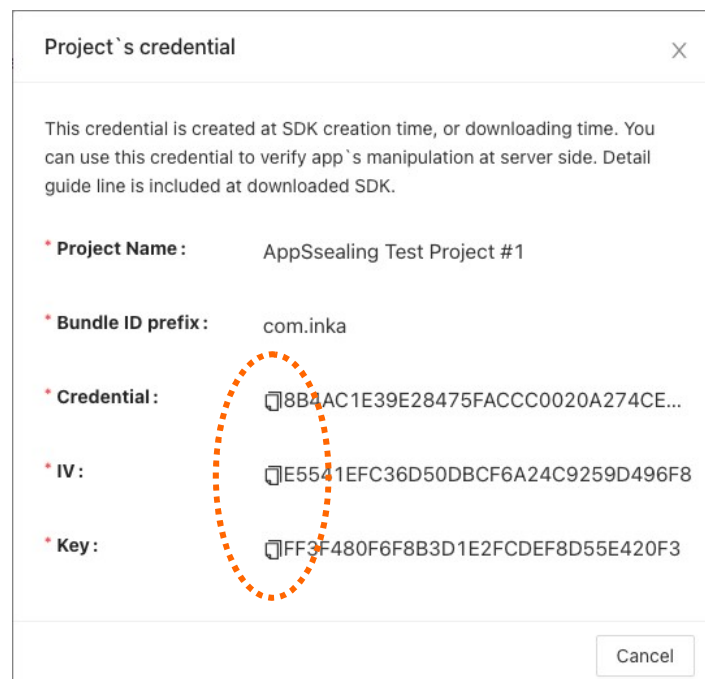
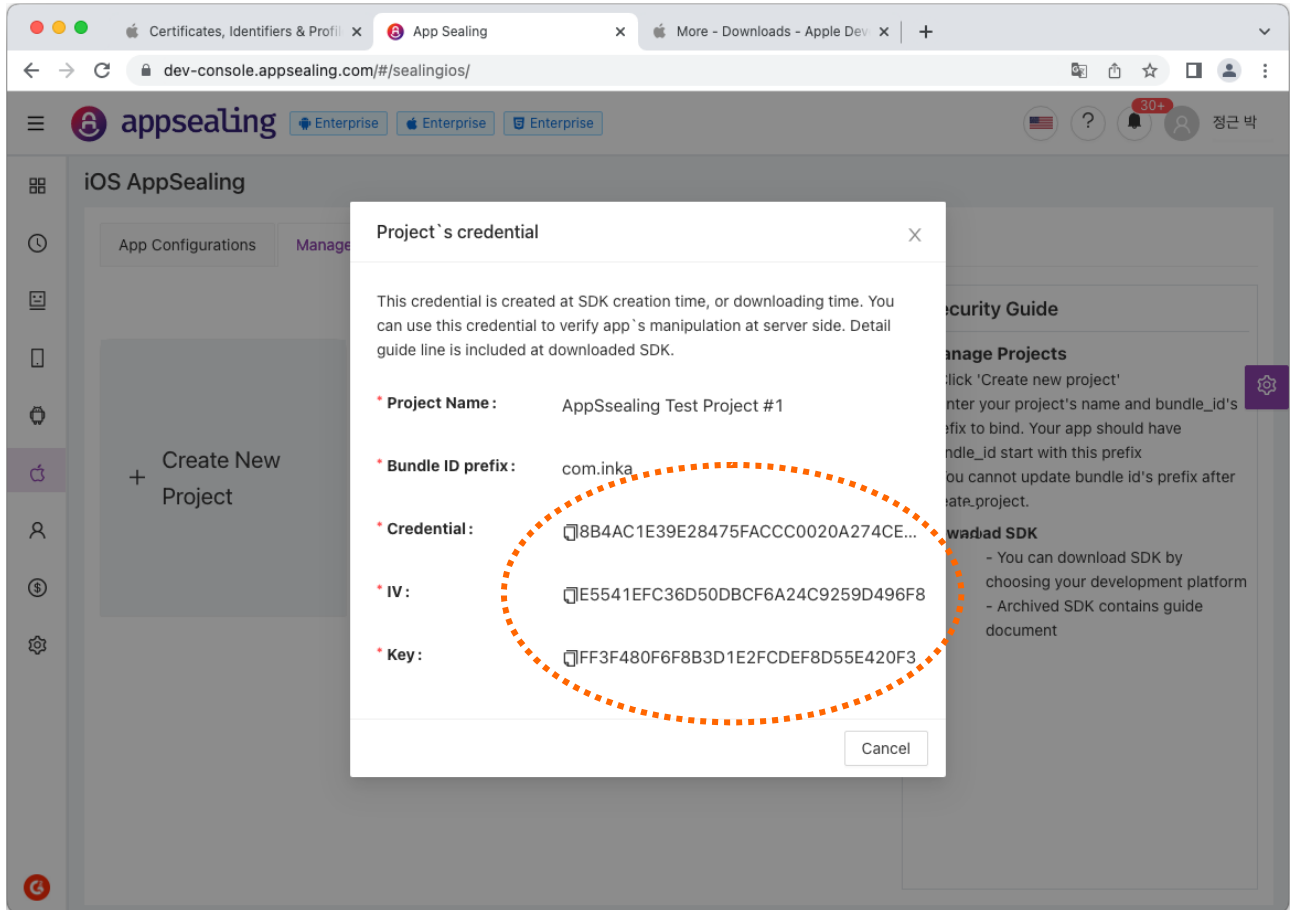
[준비 과정]

서버에서 자격증명 데이터를 검증하려면 클라이언트에서 전송된 데이터를 복호화 하기 위한 AES Key와 IV, 비교 검증할 원본 자격증명 데이터가 필요합니다.

이 값들은 모두 ADC에서 해당 프로젝트의 "Check Credential" 버튼을 통해 확인할 수 있습니다. 여기 표시되는 Hex 문자열을 그대로 복사하여 예제 코드에 붙여 놓고 사용하면 됩니다. 먼저 아래 화면처럼 ADC로 접속하여 프로젝트 박스에서 "Check Credential" 버튼을 클릭합니다.



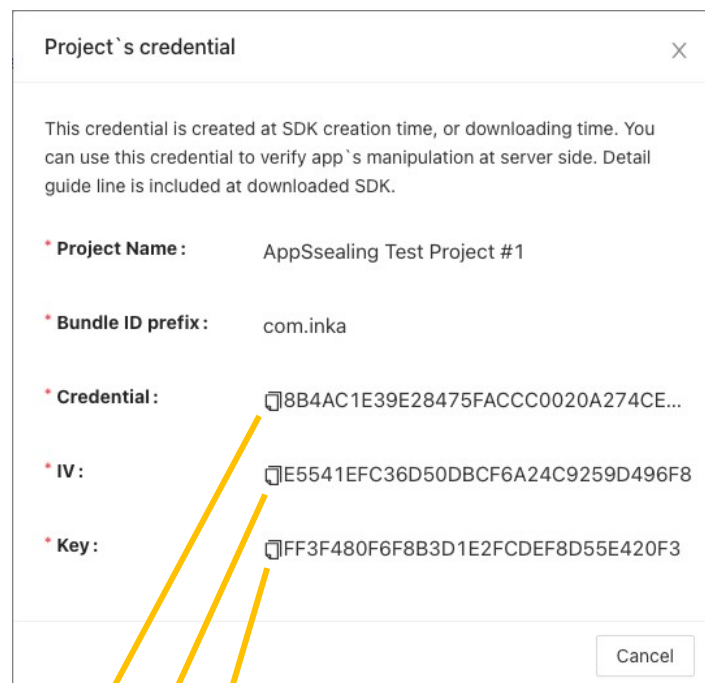
버튼을 클릭하면 아래와 같은 창이 표시되는데 여기에서 Credential 값과 복호화에 사용될 IV, key를 확인할 있습니다. 문자열 왼쪽의 복사 버튼을 이용해 이 값을 그대로 복사하여 서버 쪽 검증 코드에 붙여 넣고 사용하면 됩니다.



[서버 코드가 Node.js/Javascript로 구성되어 있는 경우]

아래의 코드를 기존 코드에 추가하고 자격증명 데이터 검증을 위해 사용하십시오. 기존 서버 코드의 로그인 또는 인증 함수에서 파라미터로 함께 올라온 credential 값을 아래 코드에서 제공되는 verifyAppSealingCredential 함수에 넘겨 진위 여부를 판단할 수 있습니다. (아래 코드는 SDK 내에 appsealing_credential.js 이름의 파일로도 포함되어 있습니다)

**** 주의: 아래 코드 중 ORG_CREDENTIAL과 AES_IV, AES_KEY는 반드시 ADC의 해당 프로젝트의 "Check Credential" 기능을 통해 가져온 값으로 대체해야 합니다. 예제 코드를 그냥 가져다 바로 사용할 경우 제대로 검증이 되지 않습니다.**



```
var crypto = require('crypto');

function verifyAppSealingCredential( credential )
{
    // Need to Change : Get From ADC (via 'Check Credential') -----
    const ORG_CREDENTIAL = "572E0E1459453F2078D6576FF71ECD0DBCA0484430C7FA7FE45B788A37DE3A04204F5A55FEA83AC9AFBA2C688594F75A3828B23972DB34858EC4F6CC3202533E44121E5F2614B227E18B6419A83810F7511D5E51FCACD5175A1CC550F83CB874A7378ACDAFE78EB2E329CD5D3C384061C4669674F1EE6B1B59FB7D91835DB7EE";

    const AES_IV = "055772B7434A4174749A209B1413472";
    const AES_KEY = "71CA94A64ADEBF5566495AB03F6798F";
    //-----

    // convert credential from hex string to byte array
    let decrypted.UTC = 0, decrypted_buffer, aes_key2;

    // decrypt UTC
```

```

try
{
    const decipher = crypto.createDecipheriv( 'aes-128-ctr', Buffer.from( AES_KEY, 'hex' ), Buffer.from( AES_IV, 'hex' ));
    decrypted_buffer = Buffer.concat( [decipher.update( credential.substr( 0, 32 ), 'hex' ), decipher.final()] );
    decrypted_UTC = decrypted_buffer.slice( 0, 8 ).readUInt32LE();
}
catch( error )
{
    throw error;
}

// verify UTC with current time (+/-) 10sec
const current_UTC = parseInt( Date.now() / 1000 ); // get current UTC in seconds
if ( Math.abs( current_UTC - decrypted_UTC ) > 10 )
{
    console.log( "Invalid UTC value has sent, deny login & all services for this client... " + Math.abs( current_UTC -
decrypted_UTC ) );
    return false;
}
console.log( "*** UTC verified : " + decrypted_UTC + " (current = " + current_UTC + ", diff = " + Math.abs( current_UTC -
decrypted_UTC ) + ")" );

// get AES KEY2
aes_key2 = Buffer.concat( [new Uint8Array( decrypted_buffer.slice( 0, 8 )), new
Uint8Array( Buffer.from( ORG_CREDENTIAL.substring( 52, 52 + 16 ), 'hex' ))] );
for( let i = 0; i < 16; i++ )
    aes_key2[i] ^= Buffer.from( AES_IV.substring( i * 2, i * 2 + 2 ), 'hex' ).readUInt8();

// decrypt credential
let decrypted_credential = [];
try
{
    const decipher = crypto.createDecipheriv( 'aes-128-ctr', aes_key2, Buffer.from( AES_IV, 'hex' ));
    decrypted_credential = Buffer.concat( [decipher.update( credential.substr( 32, 256 ), 'hex' ), decipher.final()] );
}
catch( error )
{
    throw error;
}

// verify credential with CREDENTIAL(ADC)
// return if fail
if ( ORG_CREDENTIAL.toLowerCase() != decrypted_credential.toString( 'hex' ).toLowerCase() )
{
    console.log( "Invalid credential value has sent, deny login & all services for this client..." );
    return false;
}

console.log( "*** Credential verified : PASS" );
return true;
}

```

[서버 코드가 Java로 구성되어 있는 경우]

아래의 코드를 기존 코드에 추가하고 자격증명 데이터 검증을 위해 사용하십시오. 기존 서버 코드의 로그인 또는 인증 함수에서 파라미터로 함께 올라온 credential 값을 아래 코드에서 제공되는 verifyAppSealingCredential 함수에 넘겨 진위 여부를 판단할 수 있습니다. (아래 코드는 SDK 내에 appsealing_credential.java 이름의 파일로도 포함되어 있습니다)

**** 주의: 아래 코드 중 ORG_CREDENTIAL과 AES_IV, AES_KEY는 반드시 ADC의 해당 프로젝트의 "Check Credential" 기능을 통해 가져온 값으로 대체해야 합니다. 예제 코드를 그냥 가져다 바로 사용할 경우 제대로 검증이 되지 않습니다.**

```
import java.nio.ByteBuffer;
import java.nio.ByteOrder;
import java.util.Arrays;

import javax.crypto.Cipher;
import javax.crypto.spec.IvParameterSpec;
import javax.crypto.spec.SecretKeySpec;
import javax.xml.bind.DatatypeConverter;

public class AppSealingCredential
{
    private static boolean verifyAppSealingCredential( final String credential )
    {
        // Need to Change : Get From ADC (via 'Check Credential') -----
        final String ORG_CREDENTIAL =
"572E0E1459453F2078D6576FF71ECD0DBCA0484430C7FA7FE45B788A37DE3A04204F5A55FEA83AC9AFBA2C688594F75A3828B23972DB34858EC4F6CC
3202533E44121E5F2614B227E18B6419A83810F7511D5E51FCACD5175A1CC550F83CB874A7378ACDAFE78EB2E329CD5D3C384061C4669674F1EE6B1B5
9FB7D91835DB7EE";

        final String AES_IV = "055772B7434A4174749AFE09B1413472";
        final String AES_KEY = "71CA94A64A4DEBF5566495AB03F6798F";
        //-----

        long decrypted.UTC = 0;
        byte[] decryptedUTC = new byte[8];

        // decrypt UTC
        try
        {
            SecretKeySpec key = new SecretKeySpec( DatatypeConverter.parseHexBinary( AES_KEY ), "AES" );
            IvParameterSpec ivSpec = new IvParameterSpec( DatatypeConverter.parseHexBinary( AES_IV ));

            Cipher cipher = Cipher.getInstance( "AES/CTR/NoPadding" );
            cipher.init( Cipher.DECRYPT_MODE, key, ivSpec );

            byte[] encryptedUTC = DatatypeConverter.parseHexBinary( credential.substring( 0, 32 ));
            System.arraycopy( cipher.doFinal( encryptedUTC ), 0, decryptedUTC, 0, 8 );
            System.out.println( DatatypeConverter.printHexBinary( decryptedUTC ));
        }
    }
}
```

```

        decrypted.UTC = ByteBuffer.wrap( decryptedUTC ).order( ByteOrder.LITTLE_ENDIAN ).getInt() & 0xFFFFFFFFL;
    }
    catch( Exception e )
    {
        System.out.println( "[Error] " + e.getLocalizedMessage() );
    }

    // verify UTC with current time (+/-) 10sec
    long current.UTC = System.currentTimeMillis() / 1000;           // get current UTC in seconds

    if ( Math.abs( current.UTC - decrypted.UTC ) > 10 )
    {
        System.out.println( "Invalid UTC value has sent, deny login & all services for this client..." );
        return false;
    }

    System.out.println( "*** UTC verified : " + decrypted.UTC + " (current = " + current.UTC + ", diff = " +
Math.abs( current.UTC - decrypted.UTC ) + ")" );

    // get AES KEY2
    byte[] aes_key2 = Arrays.copyOf( decryptedUTC, 16 );
    byte[] xor = DatatypeConverter.parseHexBinary( ORG_CREDENTIAL.substring( 52, 52 + 16 ));
    System.arraycopy( xor, 0, aes_key2, decryptedUTC.length, xor.length );

    for( int i = 0; i < 16; i++ )
        aes_key2[i] ^= ( byte )DatatypeConverter.parseHexBinary( AES_IV.substring( i * 2, i * 2 + 2 ))[0];
    System.out.println( DatatypeConverter.printHexBinary( aes_key2 ));

    // decrypt credential
    byte[] decrypted_credential = null;
    try
    {
        SecretKeySpec key = new SecretKeySpec( aes_key2, "AES" );
        IvParameterSpec ivSpec = new IvParameterSpec( DatatypeConverter.parseHexBinary( AES_IV ));

        Cipher cipher = Cipher.getInstance( "AES/CTR/NoPadding" );
        cipher.init( Cipher.DECRYPT_MODE, key, ivSpec );

        System.out.println( "##### " + credential.substring( 32, 32 + 256 ) );
        byte[] encrypted_credential = DatatypeConverter.parseHexBinary( credential.substring( 32, 32 + 256 ));
        decrypted_credential = cipher.doFinal( encrypted_credential );
    }
    catch( Exception e )
    {
        System.out.println( "[Error] " + e.getLocalizedMessage() );
    }

    // verify credential with CREDENTIAL(ADC)
    // return if fail
    if ( !ORG_CREDENTIAL.equalsIgnoreCase( DatatypeConverter.printHexBinary( decrypted_credential )))
    {
        System.out.println( "Invalid credential value has sent, deny login & all services for this client..." );
        return false;
    }

    System.out.println( "*** Credential verified : PASS" );
    return true;
}
}

```

[서버 코드가 ASP.NET C#으로 구성되어 있는 경우]

아래의 코드를 기존 코드에 추가하고 자격증명 데이터 검증을 위해 사용하십시오. 기존 서버 코드의 로그인 또는 인증 함수에서 파라미터로 함께 올라온 credential 값을 아래 코드에서 제공되는 verifyAppSealingCredential 함수에 넘겨 진위 여부를 판단할 수 있습니다. (아래 코드는 SDK 내에 appsealing_credential.cs 이름의 파일로도 포함되어 있습니다. C#의 경우 AES CTR 모드를 지원하지 않기 때문에 아래 코드와 같이 CTR 처리를 위해 AesCtrTransform 함수를 반드시 포함시켜야 합니다.)

**** 주의: 아래 코드 중 ORG_CREDENTIAL과 AES_IV, AES_KEY는 반드시 ADC의 해당 프로젝트의 "Check Credential" 기능을 통해 가져온 값으로 대체해야 합니다. 예제 코드를 그냥 가져다 바로 사용할 경우 제대로 검증이 되지 않습니다.**

```
using System;
using System.Collections.Generic;
using System.IO;
using System.Linq;
using System.Security.Cryptography;
using System.Text;
using System.Threading.Tasks;

namespace AppSealingSDK
{
    class AppSealingCredential
    {
        public static void AesCtrTransform( byte[] key, byte[] salt, Stream inputStream, Stream outputStream )
        {
            SymmetricAlgorithm aes = new AesManaged { Mode = CipherMode.ECB, Padding = PaddingMode.None };

            int blockSize = aes.BlockSize / 8;
            if ( salt.Length != blockSize )
                throw new ArgumentException( "Salt size must be same as block size " + $"(actual: {salt.Length}, expected: {blockSize})" );

            byte[] counter = ( byte[] )salt.Clone();

            Queue<byte> xorMask = new Queue<byte>();
            var zeroIv = new byte[blockSize];
            ICryptoTransform counterEncryptor = aes.CreateEncryptor( key, zeroIv );

            int b;
            while( ( b = inputStream.ReadByte() ) != -1 )
            {
                if ( xorMask.Count == 0 )
                {
                    var counterModeBlock = new byte[blockSize];
                    counterEncryptor.TransformBlock( counter, 0, counter.Length, counterModeBlock, 0 );

                    for( var i2 = counter.Length - 1; i2 >= 0; i2-- )
                    {
```

```

        if ( ++counter[i2] != 0 )
            break;
    }
    foreach( var b2 in counterModeBlock )
        xorMask.Enqueue( b2 );
    }
    var mask = xorMask.Dequeue();
    outputStream.WriteByte(( byte )((( byte )b ) ^ mask ));
}
}
public static byte[] StringToByteArray( String hex )
{
    return Enumerable.Range( 0, hex.Length / 2 ).Select( x => Convert.ToByte( hex.Substring( x * 2, 2 ),
16  )).ToArray();
}
public static bool VerifyAppSealingCredential( String credential )
{
    // Need to Change : Get From ADC (via 'Check Credential') -----
    const String ORG_CREDENTIAL =
"572E0E1459453F2078D6576FF71ECD0DBCA0484430C7FA7FE45B788A37DE3A04204F5A55FEA83AC9AFBA2C688594F75A3828B23972DB34858EC4F6CC
3202533E44121E5F2614B227E18B6419A83810F7511D5E51FCACD5175A1CC550F83CB874A7378ACDAFE78EB2E329CD5D3C384061C4669674F1EE6B1B5
9FB7D91835DB7EE";

    const String AES_IV = "055772B7434A4174749AFE09B1413472";
    const String AES_KEY = "71CA94A64A4DEBF5566495AB03F6798F";
    //-----

    long decrypted.UTC = 0;
    byte[] decrypted_buffer = new byte[8];

    // decrypt UTC
    try
    {
        byte[] encrypted.UTC = StringToByteArray( credential.Substring( 0, 32 ));
        byte[] result = new byte[16];
        AesCtrTransform( StringToByteArray( AES_KEY ), StringToByteArray( AES_IV ), new
MemoryStream( encrypted.UTC ), new MemoryStream( result ));
        Array.Copy( result, 0, decrypted_buffer, 0, 8 );
        decrypted.UTC = BitConverter.ToInt64( decrypted_buffer, 0 );
        System.Console.WriteLine( "*** UTC = " + decrypted.UTC );
    }
    catch( Exception e )
    {
        System.Console.WriteLine( "[Error] " + e.Message );
    }

    // verify UTC with current time (+/-) 10sec
    long current.UTC = DateTimeOffset.Now.ToUnixTimeMilliseconds() / 1000; // get current UTC in seconds
    if ( Math.Abs( current.UTC - decrypted.UTC ) > 10 )
    {
        System.Console.WriteLine( "Invalid UTC value has sent, deny login & all services for this client..." );
        return false;
    }
    System.Console.WriteLine( "*** UTC verified : " + decrypted.UTC + " (current = " + current.UTC + ", diff = " +
Math.Abs(current.UTC - decrypted.UTC ) + ")" );

    // get AES KEY2
    byte[] aes_key2 = new byte[16];
    Array.Copy( decrypted_buffer, 0, aes_key2, 0, 8 );
    byte[] xor = StringToByteArray( ORG_CREDENTIAL.Substring( 52, 16 ));

```

```
        Array.Copy( xor, 0, aes_key2, decrypted_buffer.Length, xor.Length );

        for(int i = 0; i < 16; i++)
            aes_key2[i] ^= ( byte )StringToByteArray( AES_IV.Substring( i* 2, 2 ))[0];

        // decrypt credential
        byte[] decrypted_credential = null;
        try
        {
            byte[] encrypted_credential = StringToByteArray( credential.Substring( 32, 256 ));
            decrypted_credential = new byte[encrypted_credential.Length];
            AesCtrTransform( aes_key2, StringToByteArray( AES_IV ), new MemoryStream( encrypted_credential ), new
MemoryStream( decrypted_credential ));
        }
        catch(Exception e )
        {
            System.Console.WriteLine( "[Error] " + e.Message );
        }

        // verify credential with CREDENTIAL(ADC)
        // return if fail
        if ( !ORG_CREDENTIAL.Equals( BitConverter.ToString( decrypted_credential ).Replace( "-", "" ),
StringComparison.InvariantCultureIgnoreCase ))
        {
            System.Console.WriteLine( "Invalid credential value has sent, deny login & all services for this
client..." );
            return false;
        }

        System.Console.WriteLine( "** Credential verified : PASS" );
        return true;
    }
}
```

[서버 코드가 Python으로 구성되어 있는 경우]

아래의 코드를 기존 코드에 추가하고 자격증명 데이터 검증을 위해 사용하십시오. 기존 서버 코드의 로그인 또는 인증 함수에서 파라미터로 함께 올라온 credential 값을 아래 코드에서 제공되는 verifyAppSealingCredential 함수에 넘겨 진위 여부를 판단할 수 있습니다. (아래 코드는 SDK 내에 appsealing_credential.py 이름의 파일로도 포함되어 있습니다)

**** 주의: 아래 코드 중 ORG_CREDENTIAL과 AES_IV, AES_KEY는 반드시 ADC의 해당 프로젝트의 "Check Credential" 기능을 통해 가져온 값으로 대체해야 합니다. 예제 코드를 그냥 가져다 바로 사용할 경우 제대로 검증이 되지 않습니다.**

```
from Crypto.Cipher import AES
from Crypto.Util import Counter
import time

#=====
def verifyAppSealingCredential( credential ):

    # Need to Change : Get From ADC (via 'Check Credential') -----
    ORG_CREDENTIAL =
    "572E0E1459453F2078D6576FF71ECD0DBCA0484430C7FA7FE45B788A37DE3A04204F5A55FEA83AC9AFBA2C688594F75A3828B23972DB34858EC4F6CC
    3202533E44121E5F2614B227E18B6419A83810F7511D5E51FCACD5175A1CC550F83CB874A7378ACDAFE78EB2E329CD5D3C384061C4669674F1EE6B1B5
    9FB7D91835DB7EE"
    AES_IV = "055772B7434A4174749AFE09B1413472"
    AES_KEY = "71CA94A64A4DEBF5566495AB03F6798F"
    #-----

    # decrypt UTC
    try:
        counter = Counter.new( 128, initial_value = int.from_bytes( bytes.fromhex( AES_IV ), "big" ))
        cipher = AES.new( bytes.fromhex( AES_KEY ), AES.MODE_CTR, counter = counter )
        decrypted_buffer = cipher.decrypt( bytes.fromhex( credential[:32] ))

        decrypted.UTC = int.from_bytes( decrypted_buffer[:8], "little" )
    except Exception as e:
        print( '[Error] ', e )
        return 0;

    # verify UTC with current time (+/-) 10sec
    current.UTC = round( time.time() * 1000 ); # get current UTC in seconds

    if abs( current.UTC - decrypted.UTC ) > 10:
        print( "Invalid UTC value has sent, deny login & all services for this client..." )
        #return 0

    print( "** UTC verified : ", decrypted.UTC, " (current = ", current.UTC, ", diff = ", abs( current.UTC -
decrypted.UTC ), ")" )

    # get AES KEY2
    aes_key2 = bytearray( 16 )
    aes_key2[0:8] = decrypted_buffer[0:8]
```

```
aes_key2[8:8] = bytes.fromhex( ORG_CREDENTIAL[52:68] )
print( aes_key2[:16].hex() )

for i in range( 0, 16 ):
    aes_key2[i] ^= bytes.fromhex( AES_IV[i * 2:i * 2 + 2] )[0]
print( aes_key2[:16].hex() )

# decrypt credential
try:
    counter2 = Counter.new( 128, initial_value = int.from_bytes( bytes.fromhex( AES_IV ), "big" ))
    cipher2 = AES.new( aes_key2[:16], AES.MODE_CTR, counter = counter2 )
    print( credential[32:288] )
    decrypted_credential = cipher2.decrypt( bytes.fromhex( credential[32:288] ))
except Exception as e:
    print( '[Error] ', e )
    return 0;

# verify credential with CREDENTIAL(ADC)
# return if fail
if ORG_CREDENTIAL.casefold() != decrypted_credential.hex().casefold():
    print( "Invalid credential value has sent, deny login & all services for this client..." )
    return 0

print( "** Credential verified : PASS" )
return 1
```

[서버 코드가 Ruby 스크립트로 구성되어 있는 경우]

아래의 코드를 기존 코드에 추가하고 자격증명 데이터 검증을 위해 사용하십시오. 기존 서버 코드의 로그인 또는 인증 함수에서 파라미터로 함께 올라온 credential 값을 아래 코드에서 제공되는 verifyAppSealingCredential 함수에 넘겨 진위 여부를 판단할 수 있습니다. (아래 코드는 SDK 내에 appsealing_credential.rb 이름의 파일로도 포함되어 있습니다)

**** 주의: 아래 코드 중 ORG_CREDENTIAL과 AES_IV, AES_KEY는 반드시 ADC의 해당 프로젝트의 "Check Credential" 기능을 통해 가져온 값으로 대체해야 합니다. 예제 코드를 그냥 가져다 바로 사용할 경우 제대로 검증이 되지 않습니다.**

```
require 'securerandom'
require 'net/https'
require 'json'

# Need to Change : Get From ADC (via 'Check Credential') -----
ORG_CREDENTIAL =
"572E0E1459453F2078D6576FF71ECD0DBCA0484430C7FA7FE45B788A37DE3A04204F5A55FEA83AC9AFBA2C688594F75A3828B23972DB34858EC4F6CC
3202533E44121E5F2614B227E18B6419A83810F7511D5E51FCACD5175A1CC550F83CB874A7378ACDAFE78EB2E329CD5D3C384061C4669674F1EE6B1B5
9FB7D91835DB7EE"
AES_IV = "055772B743A4174749AFE09B1413472"
AES_KEY = "71CA94A64A4DEBF5566495AB03F6798F"
#-----

def verifyAppSealingCredential( credential )

  # decrypt UTC
  begin
    aes = OpenSSL::Cipher::AES.new( "128-CTR" )
    aes.decrypt
    aes.key = [AES_KEY].pack( 'H*' )
    aes.iv = [AES_IV].pack( 'H*' )
    decrypted_buffer = aes.update( [credential[0..31]].pack( 'H*' )) + aes.final

    decrypted.UTC = decrypted_buffer.slice( 0, 8 ).unpack( 'V' ).first
  rescue => e
    puts '[Error] ' + e.to_s
    return 0
  end

  # verify UTC with current time (+/-) 10sec
  current.UTC = Time.now.strftime( '%s%L' ).to_i;          # get current UTC in seconds

  if ( current.UTC - decrypted.UTC ).abs > 10
    puts "Invalid UTC value has sent, deny login & all services for this client..."
    return 0
  end

  puts "*** UTC verified : " + decrypted.UTC.to_s + " (current = " + current.UTC.to_s + ", diff = " + ( current.UTC -
decrypted.UTC ).abs.to_s + ")"
```

```
# get AES KEY2
aes_key2 = decrypted_buffer.bytes.slice( 0, 8 ) + [ORG_CREDENTIAL[52..67]].pack( 'H*' ).bytes
for i in 0..15 do
  aes_key2[i] ^= [AES_IV[i * 2..i * 2 + 1]].pack( 'H*' ).bytes[0]
end

# decrypt credential
begin
  aes = OpenSSL::Cipher::AES.new( "128-CTR" )
  aes.decrypt
  aes.key = aes_key2.pack( 'C*' )
  aes.iv = [AES_IV].pack( 'H*' )
  decrypted_credential = aes.update( [credential[32..287]].pack( 'H*' ) ) + aes.final
rescue => e
  puts '[Error] ' + e.to_s
  return 0
end

# verify credential with CREDENTIAL(ADC)
# return if fail
if ORG_CREDENTIAL.casecmp( decrypted_credential.unpack( 'H*' ).first ) != 0
  print( "Invalid credential value has sent, deny login & all services for this client..." )
  return 0
end

print( "*** Credential verified : PASS" )
return 1
```

end

[서버 코드가 C++로 구성되어 있는 경우]

아래의 코드를 기존 코드에 추가하고 자격증명 데이터 검증을 위해 사용하십시오. 기존 서버 코드의 로그인 또는 인증 함수에서 파라미터로 함께 올라온 credential 값을 아래 코드에서 제공되는 verifyAppSealingCredential 함수에 넘겨 진위 여부를 판단할 수 있습니다. (아래 코드는 SDK 내에 appsealing_credential.cpp 이름으로 aes.hpp/ase.cpp 파일과 함께 포함되어 있습니다)

**** 주의: 아래 코드 중 ORG_CREDENTIAL과 AES_IV, AES_KEY는 반드시 ADC의 해당 프로젝트의 "Check Credential" 기능을 통해 가져온 값으로 대체해야 합니다. 예제 코드를 그냥 가져다 바로 사용할 경우 제대로 검증이 되지 않습니다.**

```
#include <iostream>
#include <vector>
#include <time.h>
#include "aes.cpp"

typedef std::vector<unsigned char> bytes;
bytes HexToBytes( const std::string& hex )
{
    std::vector<unsigned char> bytes;
    for( unsigned int i = 0; i < hex.length(); i += 2 )
    {
        std::string byteString = hex.substr( i, 2 );
        unsigned char byte = ( unsigned char )strtol( byteString.c_str(), NULL, 16 );
        bytes.push_back( byte );
    }
    return bytes;
}

static bool verifyAppSealingCredential( const char* credential )
{
    // Need to Change : Get From ADC (via 'Check Credential') -----
    const char* ORG_CREDENTIAL =
    "572E0E1459453F2078D6576FF71ECD0BCA0484430C7FA7FE45B788A37DE3A04204F5A55FEA83AC9AFBA2C688594F75A3828B23972DB34858EC4F6CC
    3202533E44121E5F2614B227E18B6419A83810F7511D5E51FCACD5175A1CC550F83CB874A7378ACDAFE78EB2E329CD5D3C384061C4669674F1EE6B1B5
    9FB7D91835DB7EE";
    const char* AES_IV = "055772B7434A4174749AFE09B1413472";
    const char* AES_KEY = "71CA94A64A4DEBF5566495AB03F6798F";
    //-----

    unsigned char decrypted_buffer[16], decrypted_credential[128];

    // decrypt UTC
    struct AES_ctx ctx;
    try
    {
        memcpy( decrypted_buffer, HexToBytes( std::string( credential ).substr( 0, 32 ) ).data(), 16 );

        AES_init_ctx_iv( &ctx, HexToBytes( AES_KEY ).data(), HexToBytes( AES_IV ).data() );
        AES_CTR_xcrypt_buffer( &ctx, decrypted_buffer, 16 );
    }
}
```

```

    long decrypted.UTC = *(( long* )decrypted_buffer );

    std::cout << "*** UTC = " << decrypted.UTC << "\n";

    // verify UTC with current time (+/-) 10sec
    time_t current.UTC = time( 0 );
    current.UTC = 1664439768; //REMOVE
    if ( abs( current.UTC - decrypted.UTC ) > 10 )
    {
        std::cout << "Invalid UTC value has sent, deny login & all services for this client...\n";
        return false;
    }
    std::cout << "*** UTC verified : " << decrypted.UTC << " (current = " << current.UTC << ", diff = " <<
abs(current.UTC - decrypted.UTC ) << ") \n";

    // get AES KEY2
    unsigned char aes_key2[16];
    memcpy( aes_key2, decrypted_buffer, 8 );
    bytes XOR = HexToBytes( std::string( ORG_CREDENTIAL ).substr( 52, 16 ));
    memcpy( aes_key2 + 8, XOR.data(), XOR.size() );

    for( int i = 0; i < 16; i++ )
        aes_key2[i] ^= HexToBytes( std::string( AES_IV ).substr( i * 2, 2 )).data()[0];

    // decrypt credential
    memcpy( decrypted_credential, HexToBytes( std::string( credential ).substr( 32, 256 )).data(), 128 );

    AES_init_ctx_iv( &ctx, aes_key2, HexToBytes( AES_IV ).data() );
    AES_CTR_xcrypt_buffer( &ctx, decrypted_credential, 128 );
}
catch( const std::out_of_range& e )
{
    std::cout << "pos exceeds string size\n";
}

// verify credential with CREDENTIAL(ADC)
// return if fail
if ( memcmp( HexToBytes( ORG_CREDENTIAL ).data(), decrypted_credential, 128 ) != 0 )
{
    std::cout << "Invalid credential value has sent, deny login & all services for this client...";
    return false;
}
std::cout << "*** Credential verified : PASS";
}

```

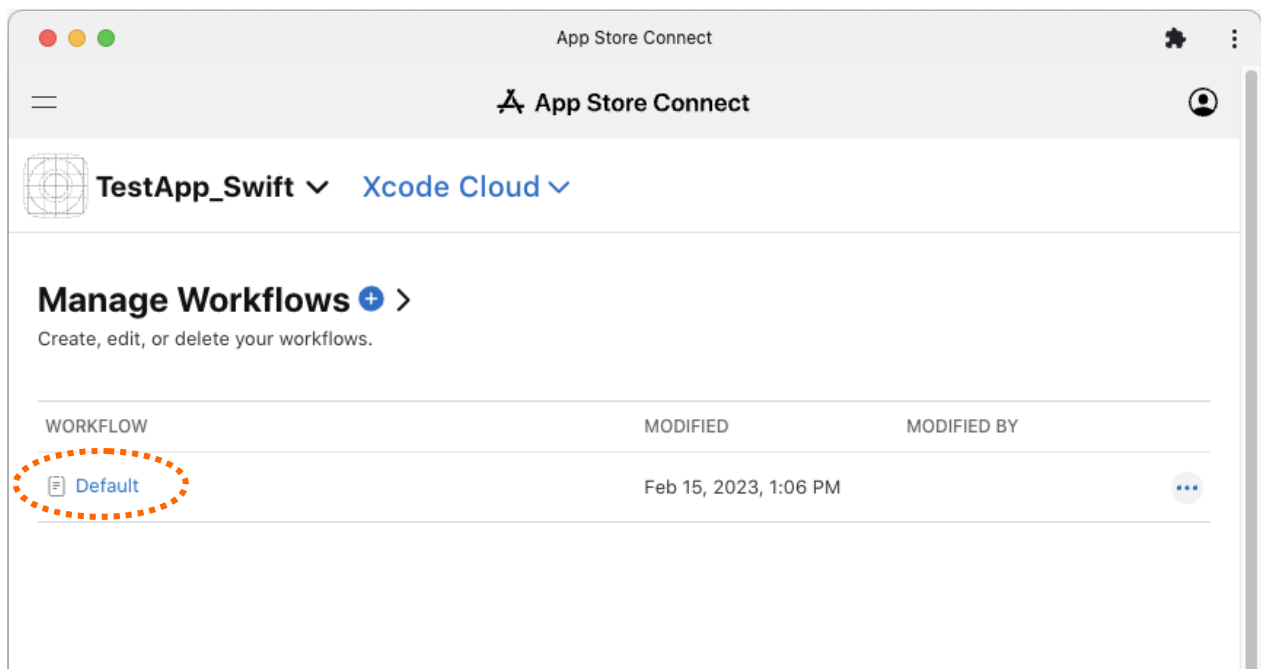
Part 7. Xcode Cloud에서 재서명 및 업로드

AppSealing SDK는 Xcode Cloud를 사용하는 프로젝트에 적용할 수 있습니다. SDK에 필요한 파일들을 추가하고 Git 저장소에 푸시하면 Xcode Cloud의 동작 방식으로 앱을 보관(archive)하고 IPA로 export 한 후 재서명 스크립트를 수행하여 보안 관련 기능이 추가된 IPA를 App Store Connect로 업로드 하게 됩니다.

다만 이 과정에서 원래의 Cloud 동작과 다른 몇 가지 액션들을 수행하게 되므로 추가적인 설정과 준비 과정이 필요합니다. 다음에 설명하는 단계대로 준비 과정을 수행하세요.

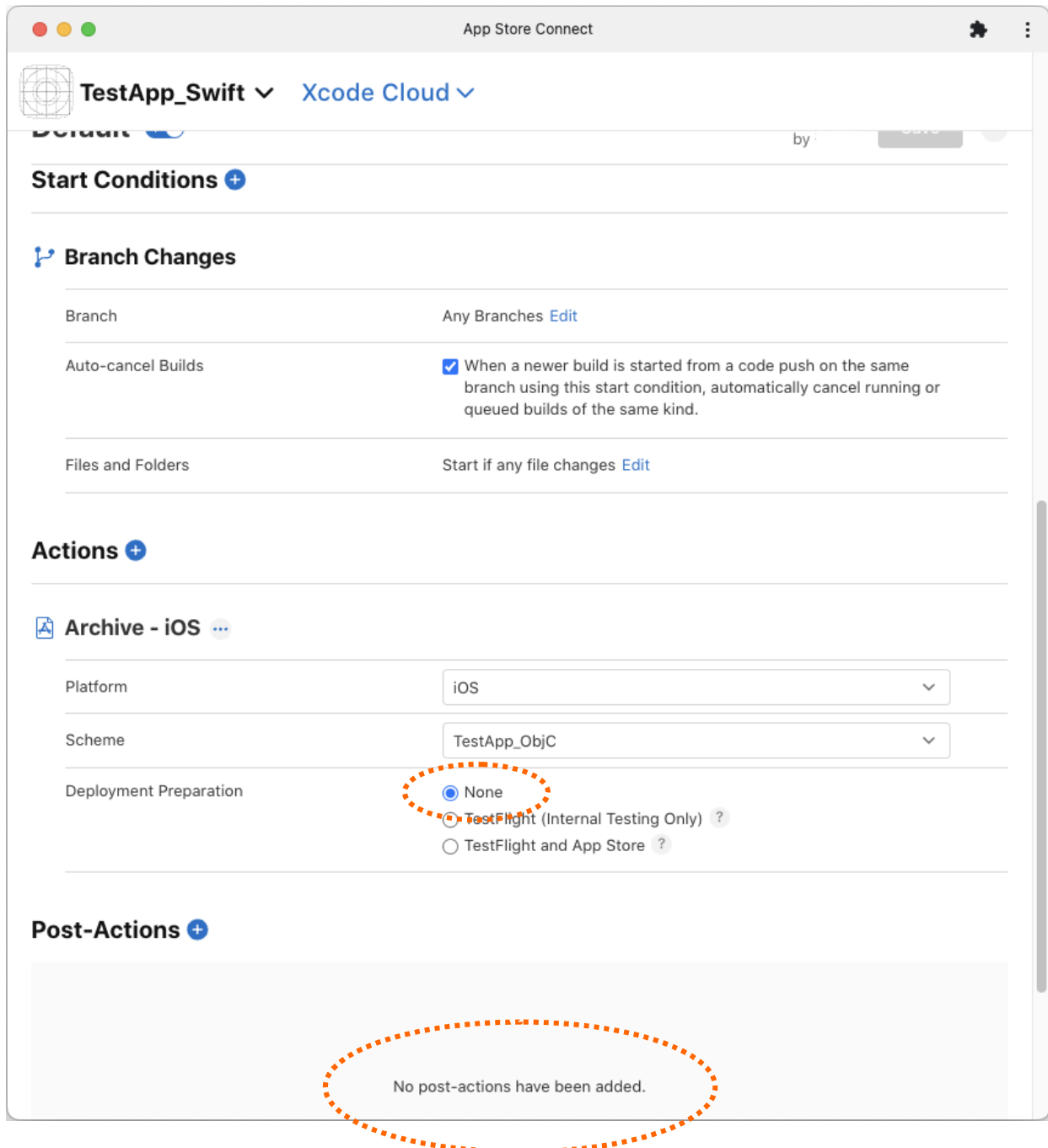
7-1 Xcode Cloud 설정

Xcode Cloud와 프로젝트 Git 저장소의 연동은 Xcode Cloud 문서 설명대로 완료되어 있는 것으로 간주합니다. 먼저 Xcode Cloud의 설정을 변경해야 합니다. 이 문서에서 사용된 프로젝트명은 "TestApp_Swift", 워크플로우 이름은 "Default"입니다. App Store Connect에 접속해서 해당 프로젝트의 "Xcode Cloud" 페이지로 이동합니다.



워크플로우 이름을 클릭하여 설정 화면으로 이동합니다.

설정 화면이 표시되면 다른 항목은 그대로 두고 페이지를 맨 아래로 스크롤 시킨 후 "Actions – Deployment Preparation" 항목의 값을 "None"으로 선택하고 "Post-Actions" 항목은 아무 액션도 들어가지 않도록 비워 둡니다.



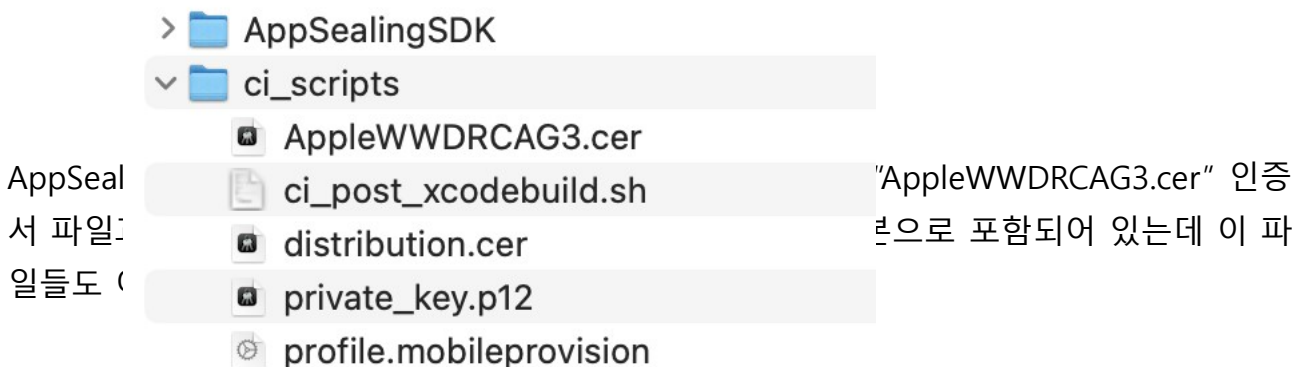
Deployment Preparation 항목의 값을 "None"으로 선택하지 않으면 빌드된 IPA가 재서명 과정을 거치지 않고 그대로 App Store Connect로 업로드 되기 때문에 앱이 정상적으로 실행되지 않습니다. Post-Actions도 마찬가지로 이 두 항목은 반드시 비워 두거나 "None"으로 설정해야 합니다.

7-2 ci_scripts 준비 사항

Xcode Cloud 환경에서 빌드된 IPA에 보안 설정 파일을 추가하고 재서명 한 뒤, 수정된 IPA를 App Store Connect에 업로드 하기 위해서는 커스텀 스크립트 파일과 서명용 인증서 및 개인키, 배포용 provisioning profile이 추가로 필요합니다.

이 파일들은 모두 프로젝트 폴더의 ci_scripts 폴더 안에 포함되어야 합니다. AppSealing SDK에서 제공하는 ci_scripts 구성 항목 외에 추가로 아래의 항목들이 필요합니다.

- App Store 배포용 인증서 : distribution.cer
- 배포용 인증서 개인키 : private_key.p12 (password: " [empty])
- App Store 배포용 프로비저닝 프로파일 : profile.mobileprovision
- Apple ID 및 App-Specific password (스크립트에 기록됨)



이 기본 파일에 앞에서 언급된 세 개의 파일(distribution.cer, private_key.p12, profile.mobileprovision) 이 추가되어 위 그림과 같이 모두 다섯 개의 파일이 포함되게 됩니다.

App Store 배포용 인증서

ci_scripts 폴더에 반드시 배포용 인증서가 포함되어야 합니다. 이 인증서는 Apple Developer 사이트에서 다운로드 받은 CER 형식의 파일이어야 하고 파일명은 반드시 "distribution.cer"로 고정되어 있어야 합니다. App Store 배포용 인증서가 아닌 인증서를 추가하거나 파일 이름이 다르게 되어 있을 경우 재서명이 정상적으로 이루어지지 않습니다.

배포용 인증서 개인키

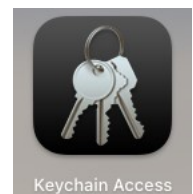
앞서 포함시킨 배포용 인증서에 대응하는 개인키 파일도 ci_scripts 폴더에 포함되어야 합니다. 개인키 파일은 맥 키체인 앱에서 export할 수 있으며 형식은 PKCS#12 (확장자 .p12)로 설정하고 개인키 파일에 대한 패스워드는 빈 문자열로 비워 두어야 합니다. 개인키 파일의 파일명도 "private_key.p12"로 고정되어 있어야 합니다.

파일명이 맞지 않거나 패스워드에 값이 설정되어 있을 경우 재서명이 정상적으로 이루어지지 않습니다. 만약 개인키 파일에 패스워드를 설정하고자 할 경우 아래 스크립트 수정 항목에서 추가적인 작업을 진행하세요.

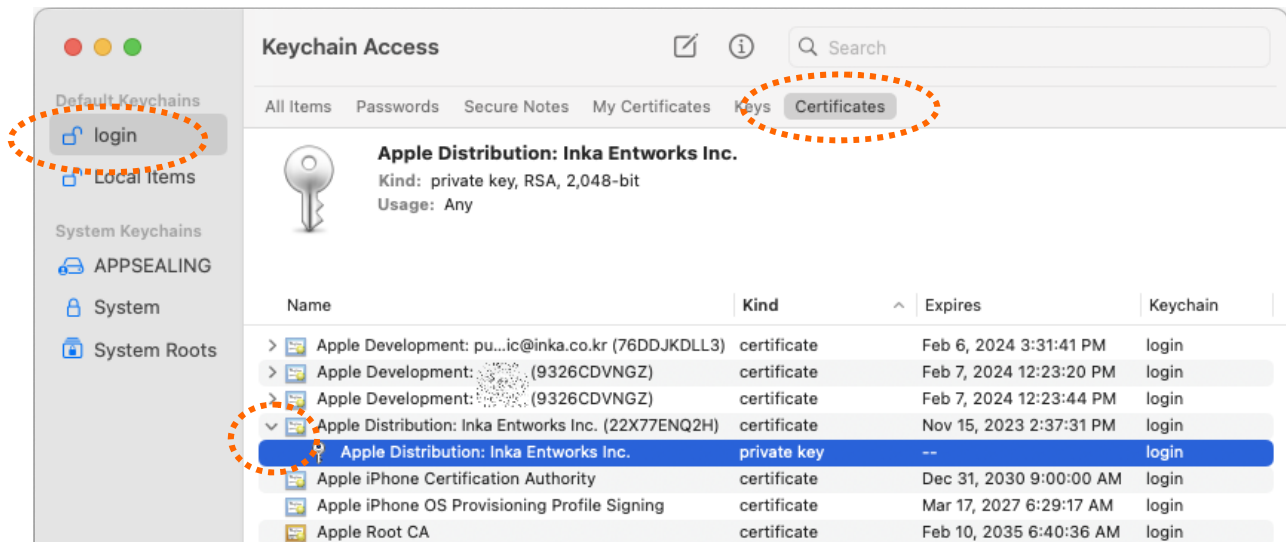
아래는 개인키 파일 생성 과정에 대한 단계별 설명입니다. 이미 개인키 파일을 생성한 경우 이 설명을 건너 뛰세요.

STEP 맥 키체인 앱에서 개인키 파일 생성 과정

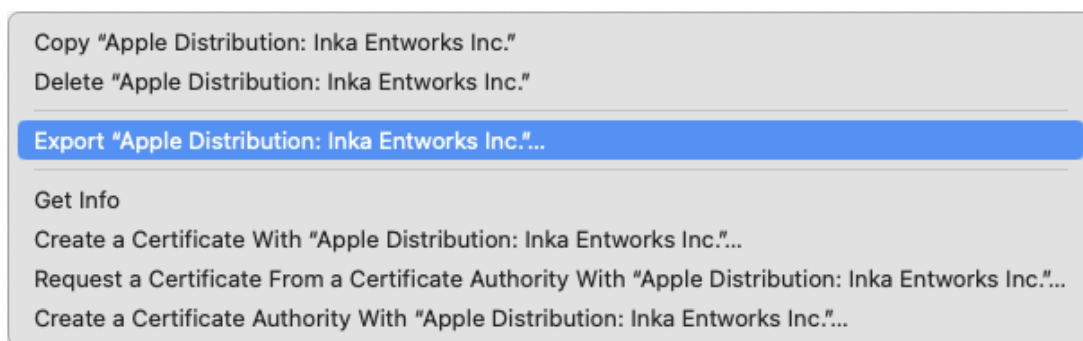
맥의 launchpad에서 기타 항목으로 들어가면 "Keychain Access" 앱이 포함되어 있습니다. 이 앱을 실행하고 아래와 같은 화면이 표시되면 좌측 탭에서 "login" 항목을 선택합니다.



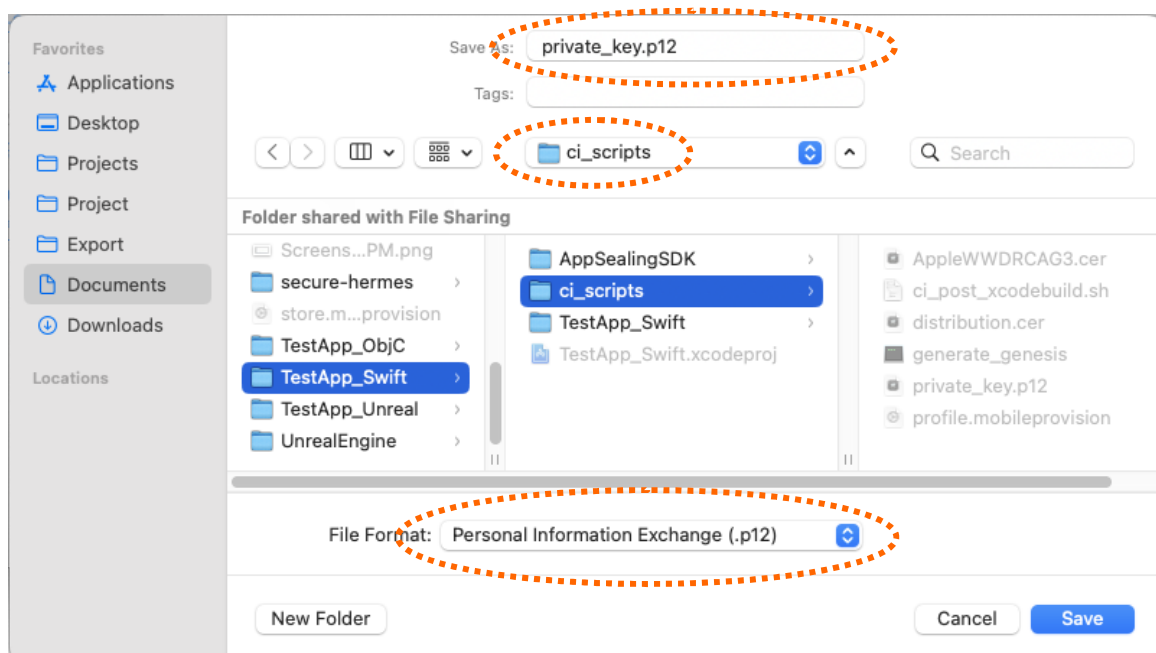
오른 쪽 창의 상단 탭 메뉴에서 "Certificates"를 선택하고 키체인에 등록된 배포용 인증서 좌측의 오른쪽 꺾쇠(>)를 클릭하여 해당 인증서의 개인키가 표시되도록 합니다.



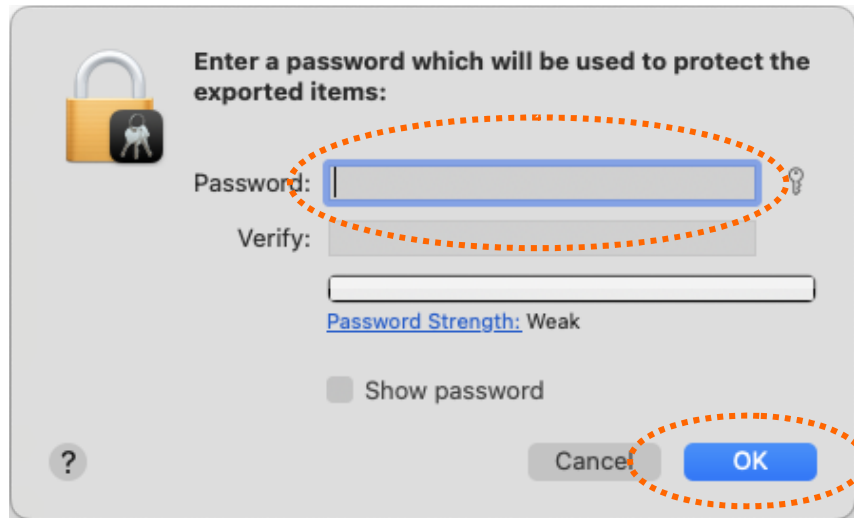
개인키를 선택한 후 마우스를 우클릭하여 컨텍스트 메뉴가 표시되면 "Export ..." 항목을 선택합니다.



저장 위치는 프로젝트의 ci_scripts 폴더, 파일 이름은 "private_key.p12", 파일 형식은 "Personal Information Exchange" 로 선택하세요.



패스워드 입력창이 나타나면 패스워드는 비워 둔 상태로 저장합니다.



만약 패스워드를 지정할 경우는 ci_post_xcodebuild.sh 파일을 수정해 주어야 합니다. 에디터로 파일을 열고 445 라인으로 이동합니다. 이 줄의 명령어에 개인키 파일의 패스워드가 공백으로 되어 있는 것을 볼 수 있습니다.

```

441 system( 'security create-keychain -p 0000 ' + APPSEALING_KEYCHAIN )
442 system( 'security list-keychains -d user -s login.keychain ' + APPSEALING_KEYCHAIN )
443 system( 'security import ./AppleWDRACAG3.cer -k ' + APPSEALING_KEYCHAIN + ' -t cert -A -P "" ' )
444 system( 'security import ./distribution.cer -k ' + APPSEALING_KEYCHAIN + ' -t cert -A -P "" ' )
445 system( 'security import ./private_key.p12 -k ' + APPSEALING_KEYCHAIN + ' -t priv -A -P "" ' )
446 system( 'security default-keychain -d user -s ' + APPSEALING_KEYCHAIN )
447 system( 'security unlock-keychain -p 0000 ' + APPSEALING_KEYCHAIN )
448 system( 'security set-keychain-settings ' + APPSEALING_KEYCHAIN )
449 system( 'security set-key-partition-list -S apple-tool:,apple:,codesign: -s -k 0000 ' + APPSEALING_KEYCHAIN + ' > /dev/null' )

```

이 위치에 지정한 패스워드를 기록하고 파일을 저장합니다. 만약 패스워드를 "your_password"라는 문자열로 저장했다면 아래와 같이 수정을 해야 합니다.

```

441 system( 'security create-keychain -p 0000 ' + APPSEALING_KEYCHAIN )
442 system( 'security list-keychains -d user -s login.keychain ' + APPSEALING_KEYCHAIN )
443 system( 'security import ./AppleWDRACAG3.cer -k ' + APPSEALING_KEYCHAIN + ' -t cert -A -P "" ' )
444 system( 'security import ./distribution.cer -k ' + APPSEALING_KEYCHAIN + ' -t cert -A -P "" ' )
445 system( 'security import ./private_key.p12 -k ' + APPSEALING_KEYCHAIN + ' -t priv -A -P "your_password" ' )
446 system( 'security default-keychain -d user -s ' + APPSEALING_KEYCHAIN )
447 system( 'security unlock-keychain -p 0000 ' + APPSEALING_KEYCHAIN )
448 system( 'security set-keychain-settings ' + APPSEALING_KEYCHAIN )
449 system( 'security set-key-partition-list -S apple-tool:,apple:,codesign: -s -k 0000 ' + APPSEALING_KEYCHAIN + ' > /dev/null' )

```

App Store 배포용 프로비저닝 프로파일

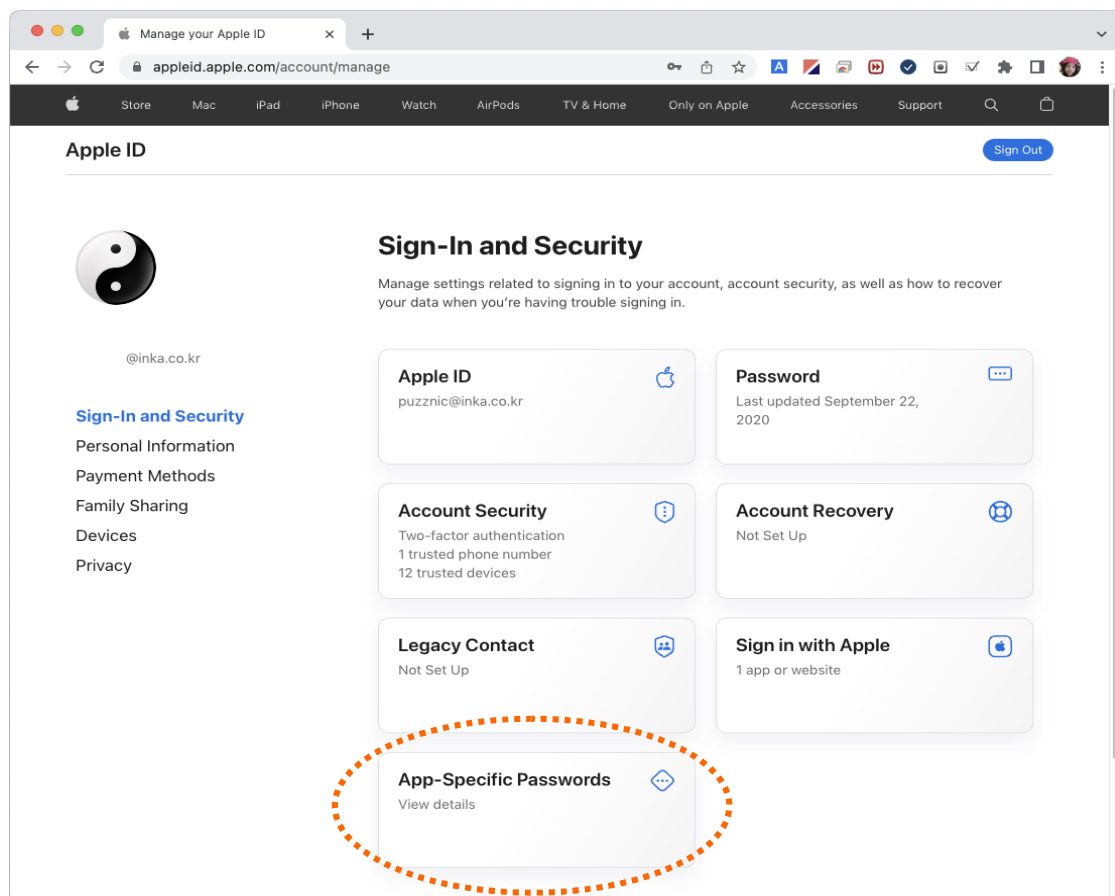
ci_scripts 폴더에 App store 배포를 위한 프로비저닝 프로파일이 포함되어야 합니다. 이 파일은 Apple Developer 사이트에서 다운로드 한 프로파일을 "profile.mobileprovision" 이라는 이름으로 포함시켜야 합니다. 파일명이 맞지 않거나 올바른 프로파일이 아닐 경우 재서명에 실패하거나 앱이 정상적으로 설치/실행이 되지 않을 수 있습니다.

빌드 후처리 과정용 커스텀 스크립트

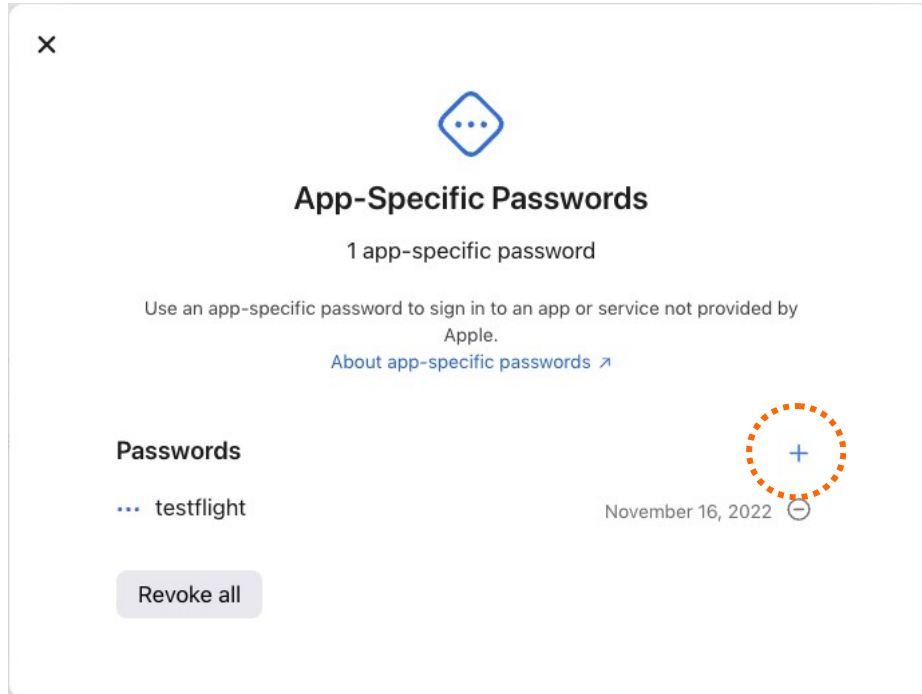
ci_scripts 폴더에는 기본으로 ci_post_xcodebuild.sh 스크립트 파일이 포함되어 있습니다. 스크립트의 내용 중 재서명한 IPA를 App Store Connect에 업로드 하기 위한 과정이 포함되어 있는데 이 동작을 수행하기 위해서 앱 개발자의 Apple 계정과 앱 전용 비밀번호가 필요합니다. (Apple 계정 패스워드가 아닙니다!)

앱 전용 비밀번호가 없을 경우 아래 주소의 Apple 페이지에서 새로 생성할 수 있습니다.

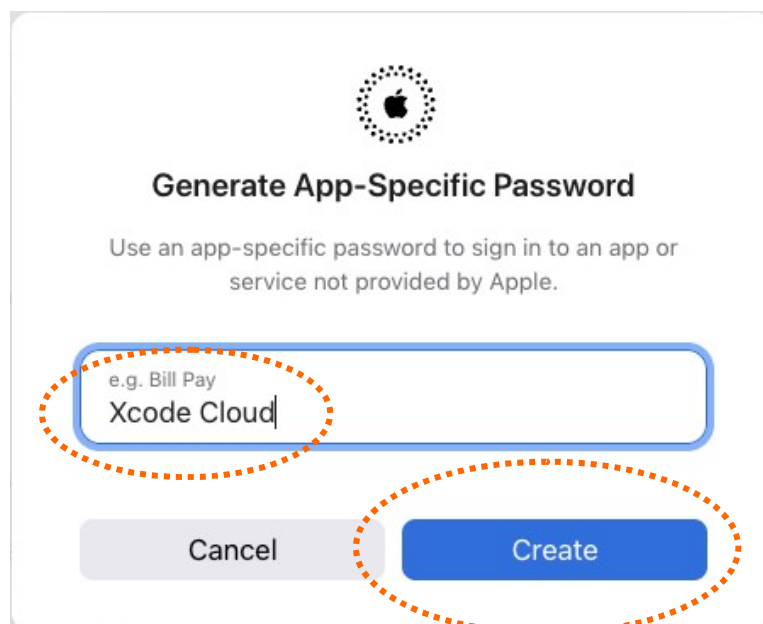
<https://appleid.apple.com/account/manage>



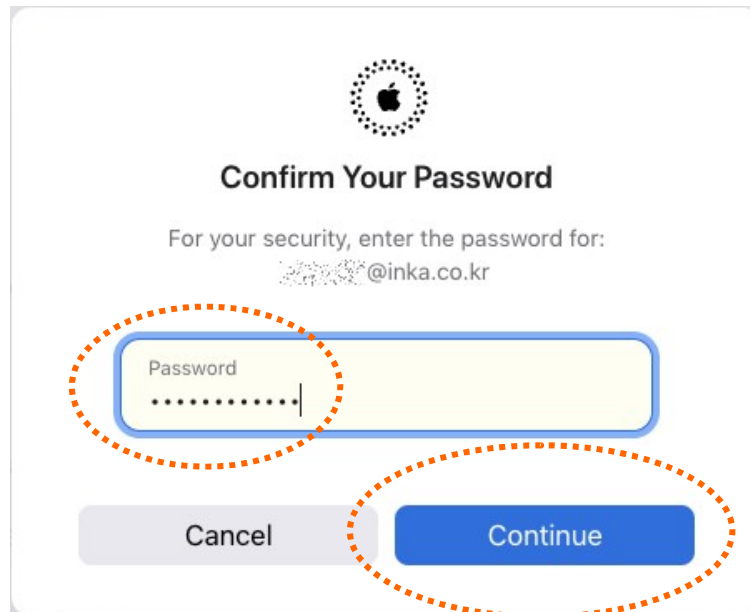
“Sign-In and Security” 페이지에서 “App-Specific Passwords” 항목을 선택하면 다음과 같이 “App-Specific Passwords” 창이 표시됩니다. 여기서 + 버튼을 클릭하여 새로운 앱 전용 비밀번호 생성을 시작합니다.



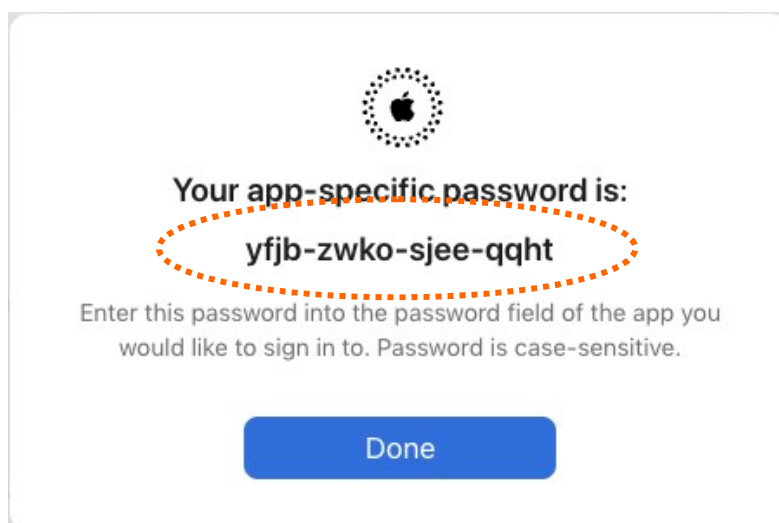
+ 버튼을 클릭하면 앱 전용 비밀번호의 이름을 입력하는 창이 나타납니다. 이름은 용도에 맞게 알아보기 편한 이름을 적당히 입력하면 됩니다. 여기서는 임의로 “Xcode Cloud”라는 이름을 입력했습니다. 이제 “Create” 버튼을 클릭합니다.



이제 Apple 계정 패스워드를 입력하라는 창이 나타나면 Apple ID의 계정 패스워드를 입력하고 "Continue" 버튼을 클릭합니다.



이제 아래 창과 같이 새로 생성된 16자리의 앱 전용 비밀번호가 표시됩니다. 이렇게 생성된 패스워드는 이후 **값을 확인할 수 없고 폐기하는 것만 가능하므로 잘 기록해 두어야 합니다.**



이제 Apple 계정과 앱 전용 비밀번호를 ci_post_xcodebuild.sh 파일의 상단 부분 20~21 라인위치에 기록하고 파일을 저장합니다.

만약 Apple 계정이 "myappleid@company.com" 이고 발급받은 앱 전용 비밀번호가 "aaaa-bbbb-cccc-dddd" 라면 스크립트 파일의 20~21라인은 다음과 같이 변경되어야 합니다.

```

13  #-----
14  APPLE_ID = "myappleid@company.com" → # replace with your apple developer ID
15  APPLE_APP_PASSWORD = "yfjb-zwko-sjee-qght" → # replace with your apple application password
16  # NOT ACCOUNT PASSWORD !
17  #-----

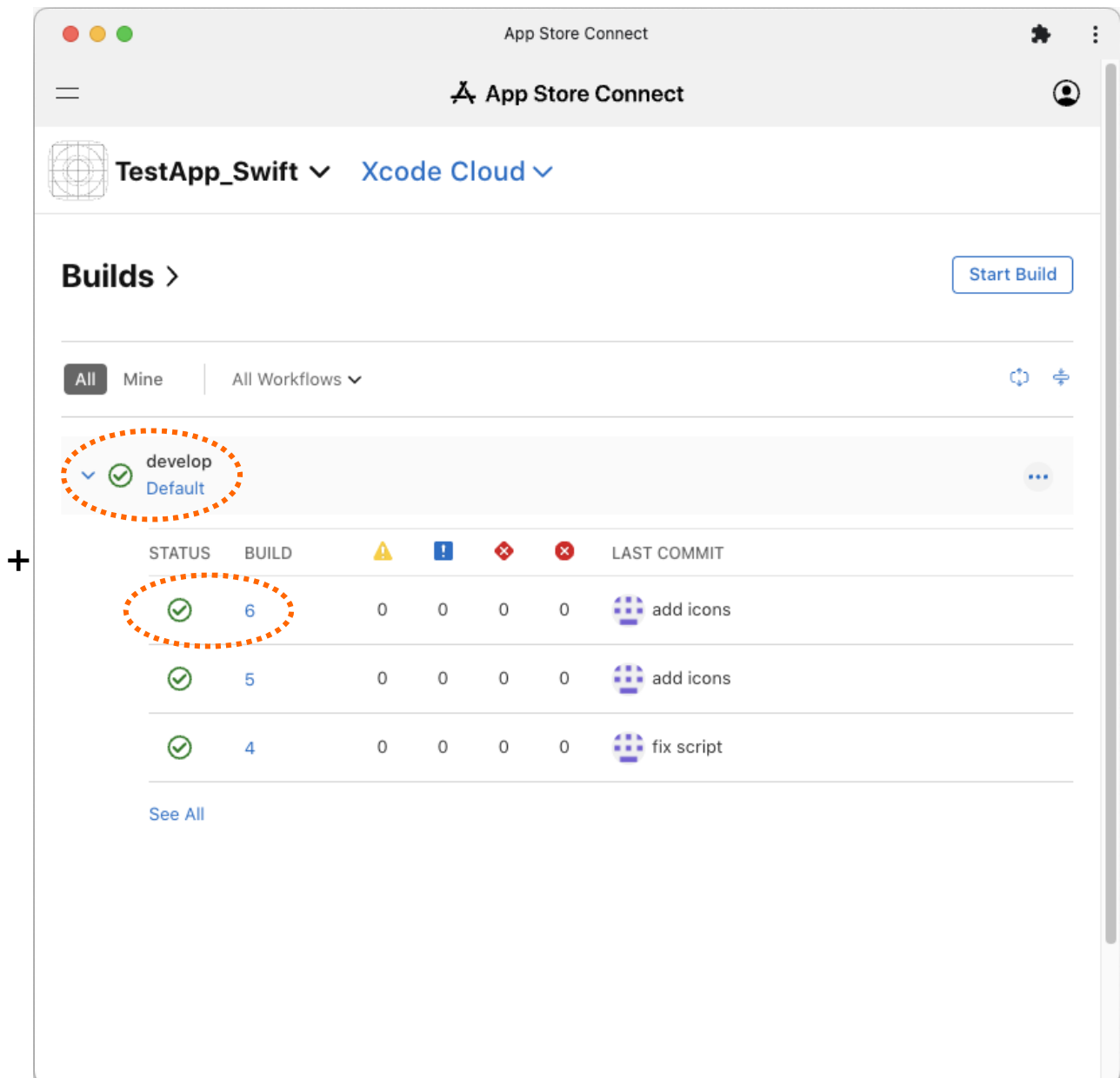
1  #!/usr/bin/env ruby
2
3  =begin
4  =====
5  | AppSealing iOS SDK Hash Generator V1.2.2 |
6  | |
7  | * presented by Inka Entworks |
8  =====
9  =end
10
11 require 'pathname'
12 require 'tmpdir'
13 require 'securerandom'
14 require 'net/https'
15 require 'json'
16 require 'io/console'
17 require 'open-uri'
18
19 #-----
20 APPLE_ID = "support@inka.co.kr" # replace with your apple developer ID
21 APPLE_APP_PASSWORD = "aaaa-bbbb-cccc-dddd" # replace with your apple application password (https://appleid.apple.com/)
22 # NOT ACCOUNT PASSWORD !
23 #-----
24

```

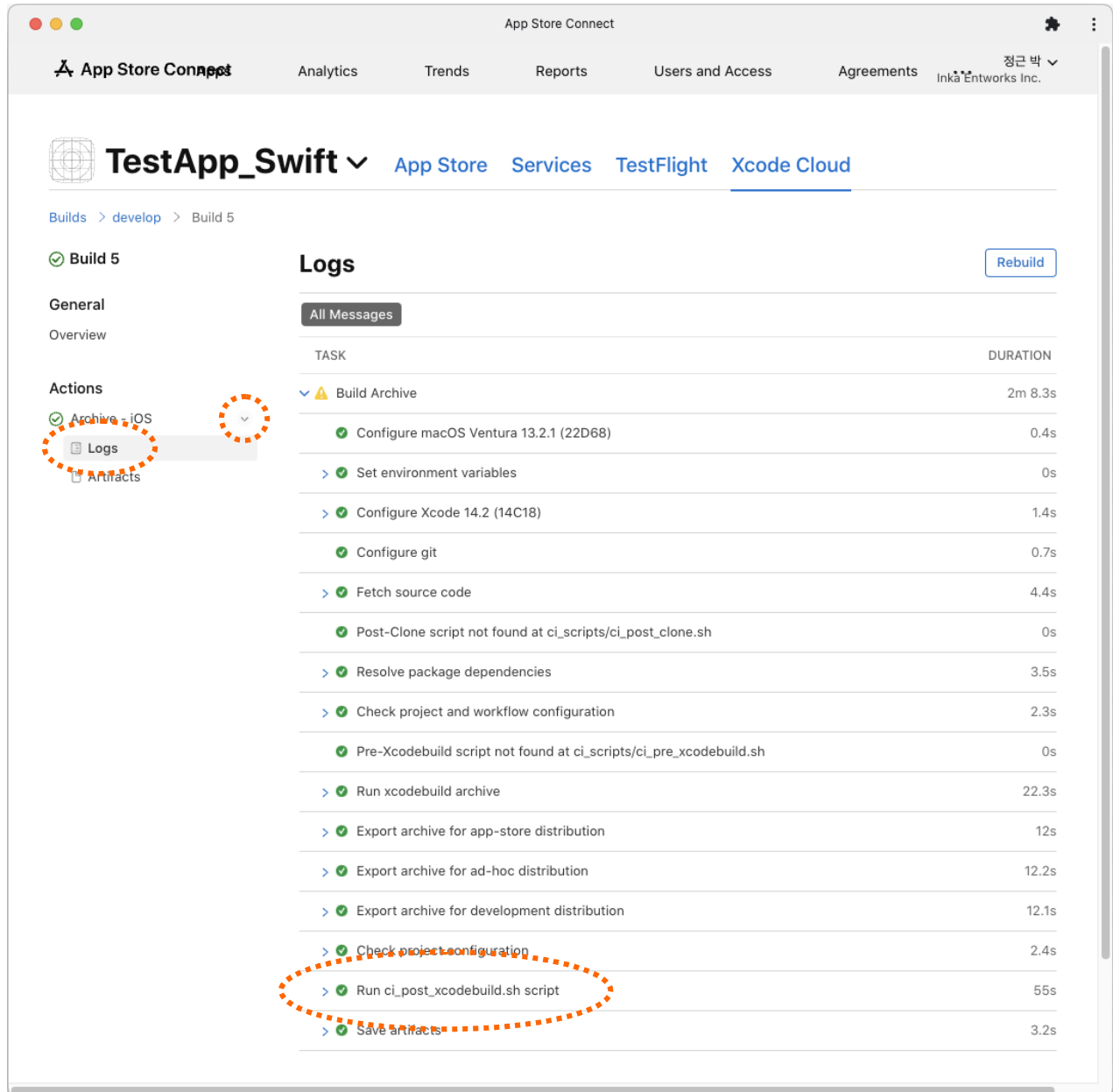
7-3 빌드 및 업로드 과정 확인

Xcode Cloud 환경 설정 및 ci_scripts 폴더에 필요한 파일 준비가 모두 완료되면 ci_scripts에 추가된 파일들을 git 저장소에 푸시합니다.

이제 App Store Connect에 접속하여 타겟 브랜치(이 문서에서는 develop)의 빌드가 완료된 것을 확인합니다.



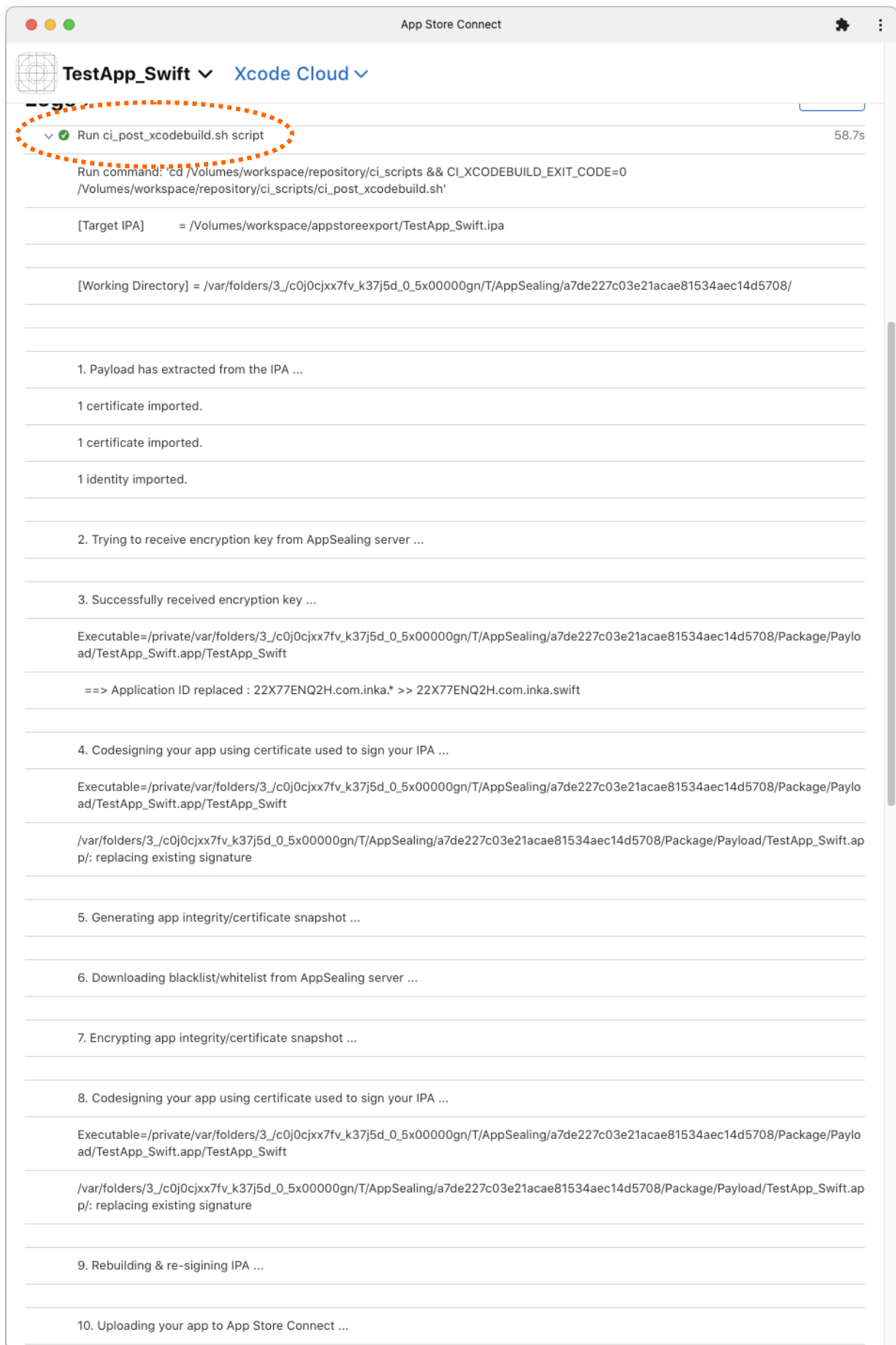
빌드 번호를 클릭하여 빌드 Overview 화면으로 이동합니다. 좌측의 "Actions" 아래 있는 "Archive – iOS" 항목의 우측 화살표를 클릭하여 메뉴를 확장하고 로그를 확인합니다.

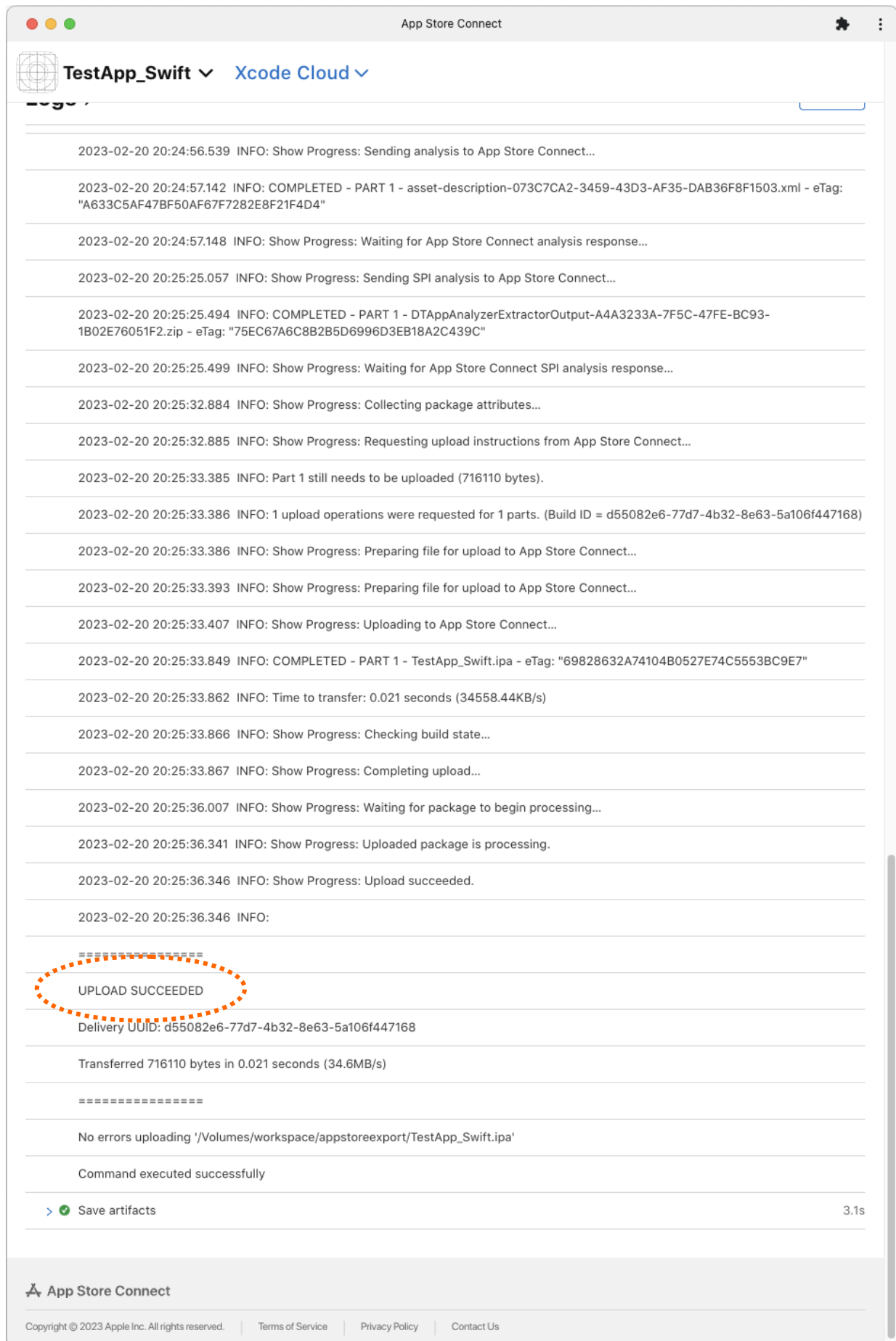


위 화면처럼 "Build Archive" 아래의 작업들이 모두 정상적으로 수행되어야 합니다.

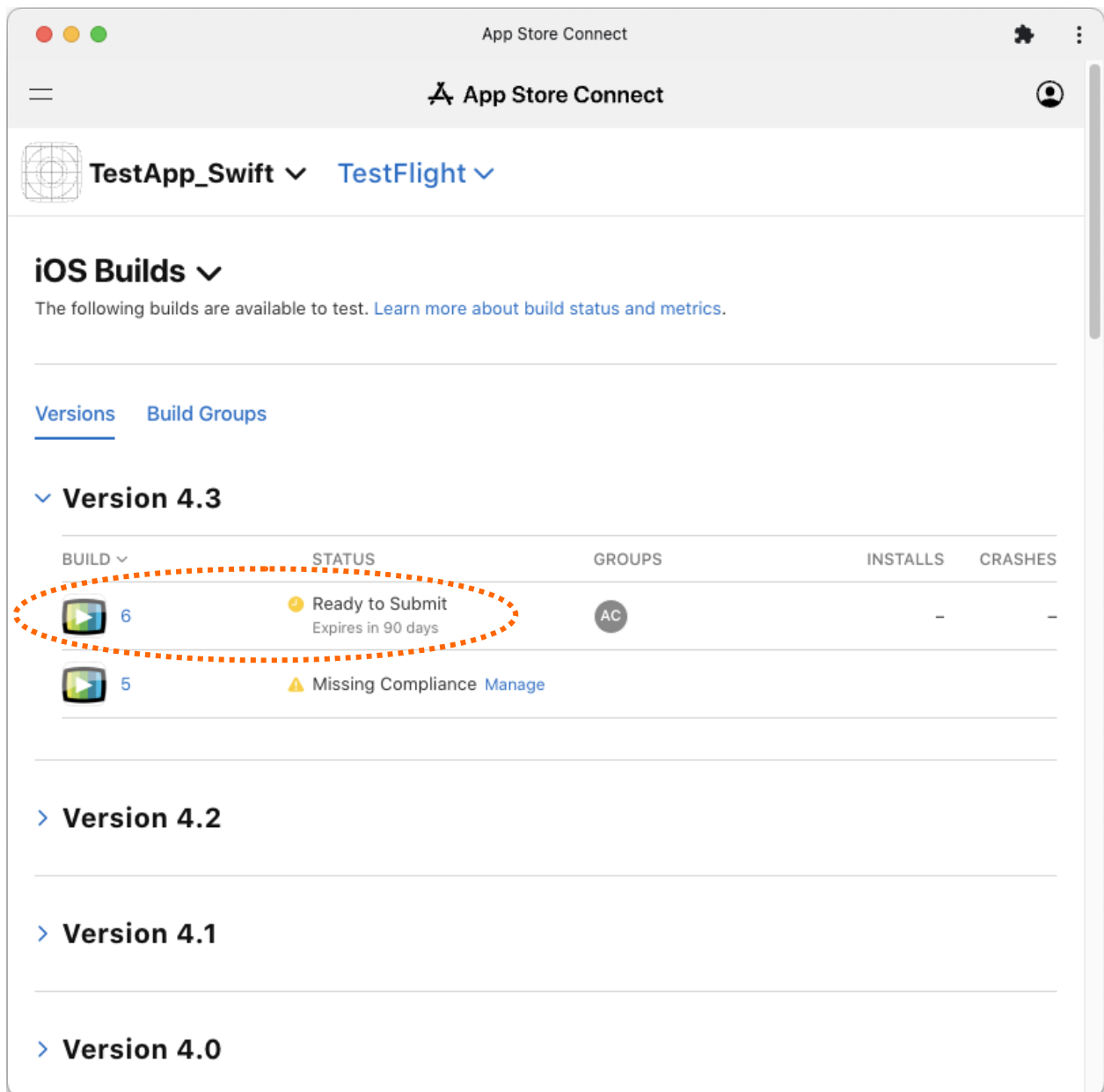
특히 "Run ci_post_xcodebuild.sh script"와 "Save artifacts" 이후에 다른 작업이 더 수행되어서는 안 됩니다. 만약 다른 작업의 진행 로그가 있다면 7-1 섹션의 내용을 참고하여 워크플로우 설정을 수정한 후 다시 빌드를 진행해야 합니다.

이제 "Run ci_post_xcodebuild.sh script" 항목의 왼쪽 꺾쇠(>)를 클릭하여 스크립트 로그를 펼쳐 봅니다. Exported 된 IPA에 대해 ci_post_xcodebuild.sh 스크립트가 실행된 로그를 확인할 수 있습니다. 로그를 맨 아래쪽으로 스크롤 하여 IPA가 정상적으로 업로드 된 내용을 확인합니다.





이제 TestFlight로 이동해 보면 방금 빌드 된 버전(6)이 등록되어 있는 것을 확인할 수 있습니다.



Part 8. 소스 코드 문자열 암호화 적용

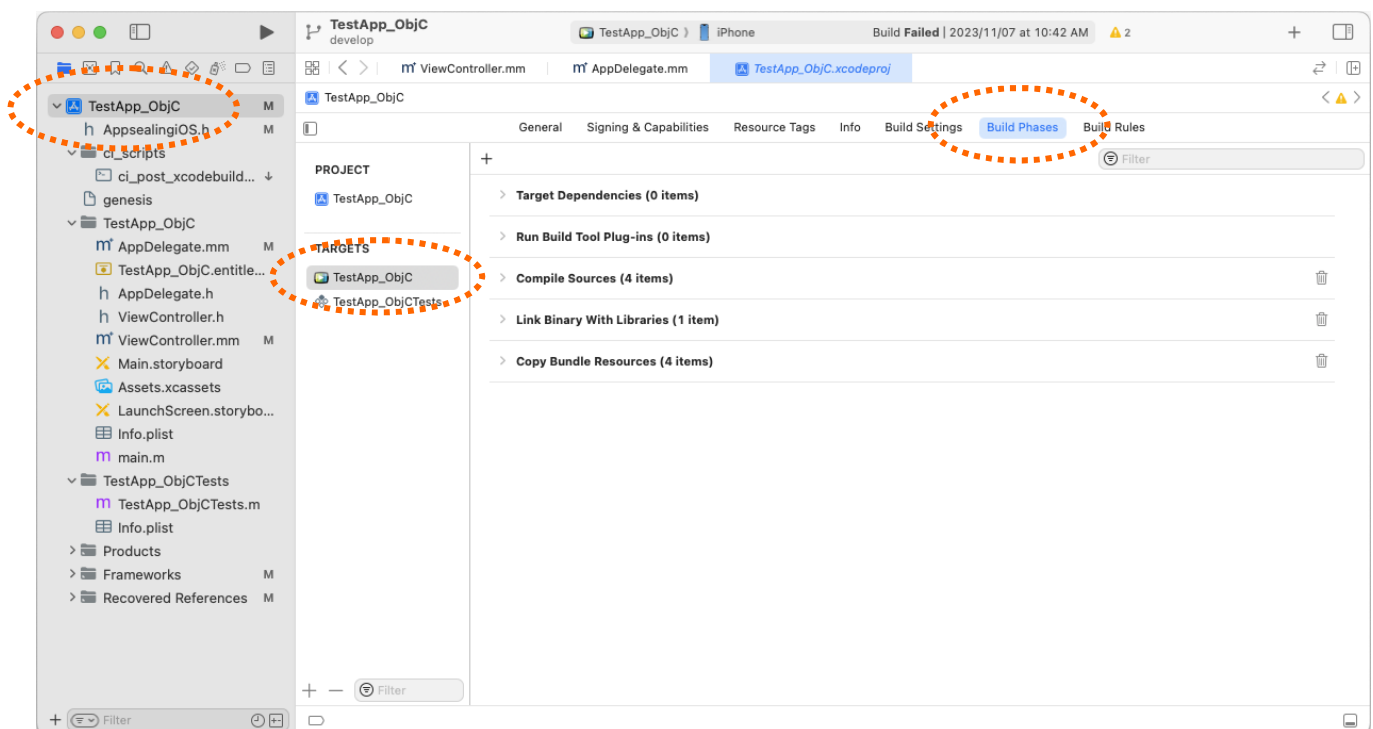
8-1 프로젝트 Build Phase 설정

AppSealing SDK는 부가 기능으로 소스 코드 문자열을 암호화하여 실행 파일에서 문자열을 완전히 숨길 수 있는 기능을 제공합니다. 일반적으로 소스 코드의 모든 문자열을 암호화 할 필요는 없지만 일련번호나 중요 정보를 포함한 문자열의 경우, 또는 문자열 복사를 하지 못하도록 막을 필요가 있을 경우 암호화를 통해 문자열을 보호할 수 있습니다.

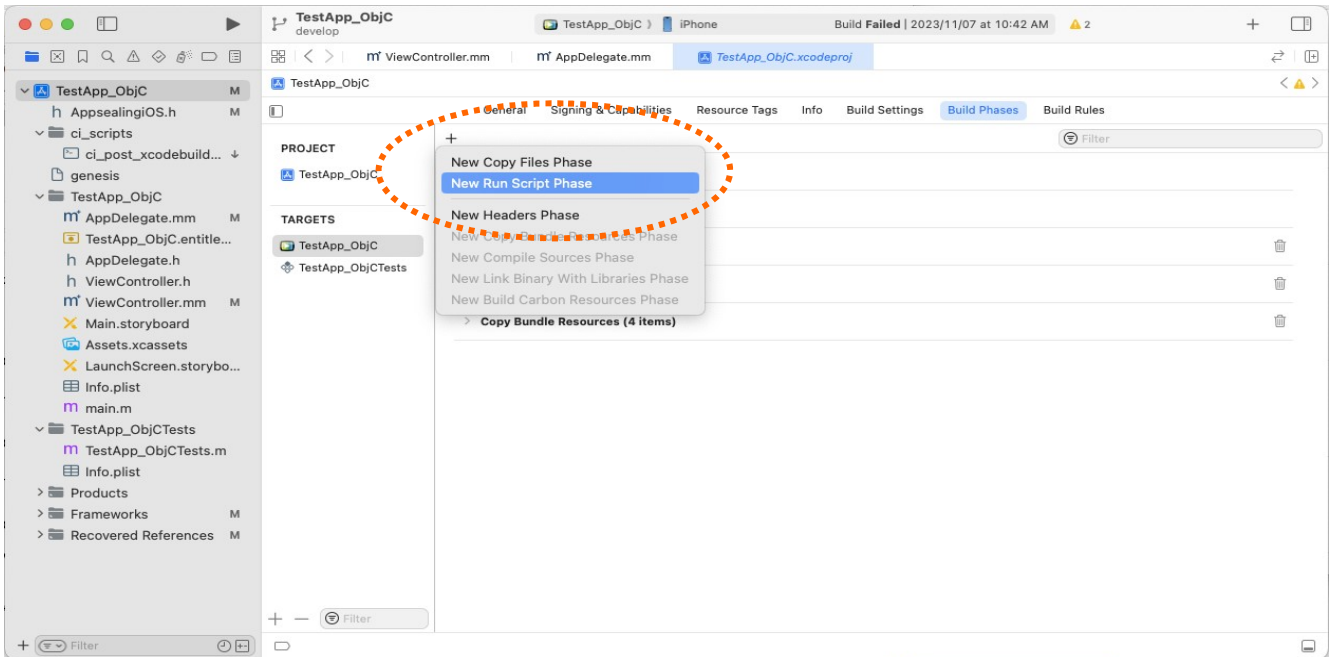
따라서 AppSealing의 문자열 암호화 기능은 소스 코드의 모든 문자열을 암호화 하는 것이 아니라 소스 코드 중에서 보호가 필요한 문자열에 한해 개발자가 직접 지정된 태그를 추가하는 방식으로 진행됩니다. 이렇게 태그가 추가된 문자열은 원래의 문자열 형태를 그대로 보존하고 있다가 컴파일 직전 암호화된 문자열로 교체되고 컴파일 되며 컴파일이 완료되면 다시 태그가 추가된 원래 문자열 형태로 복구됩니다. 다만 이를 위해 Xcode 빌드 과정에 추가적인 설정이 필요합니다. 아래 설명된 단계를 따라 빌드 스크립트를 설정해 줍니다.

이 장에서는 샘플 프로젝트로 Objective-C 기반의 TestApp_ObjC 라는 이름의 프로젝트를 사용합니다.

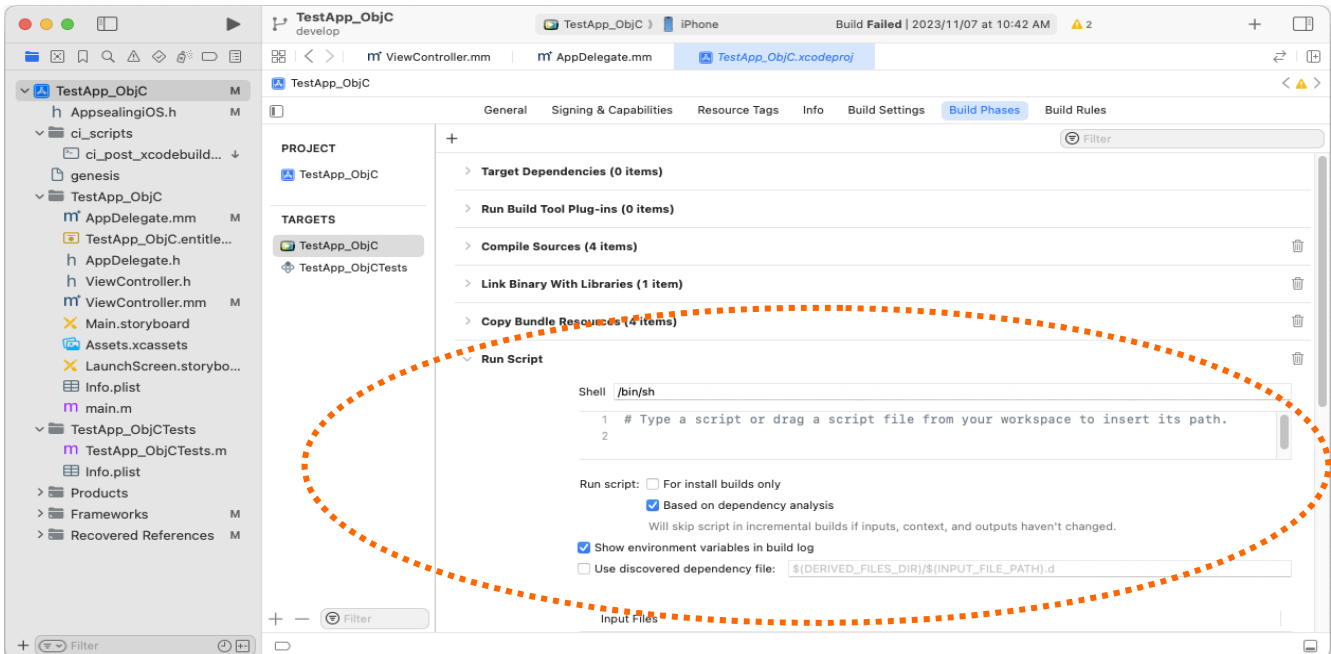
먼저 프로젝트 네비게이터에서 프로젝트를 선택하고 TARGETS에서 메인 타겟 "TestApp_ObjC"를 선택합니다. 우측에 표시되는 탭 중에서 "Build Phase" 탭을 선택합니다.



"Build Phase" 탭 화면이 표시되면 좌측 상단의 "+" 아이콘을 클릭하여 메뉴를 표시하고 여기서 "New Run Script Phase"를 선택합니다.

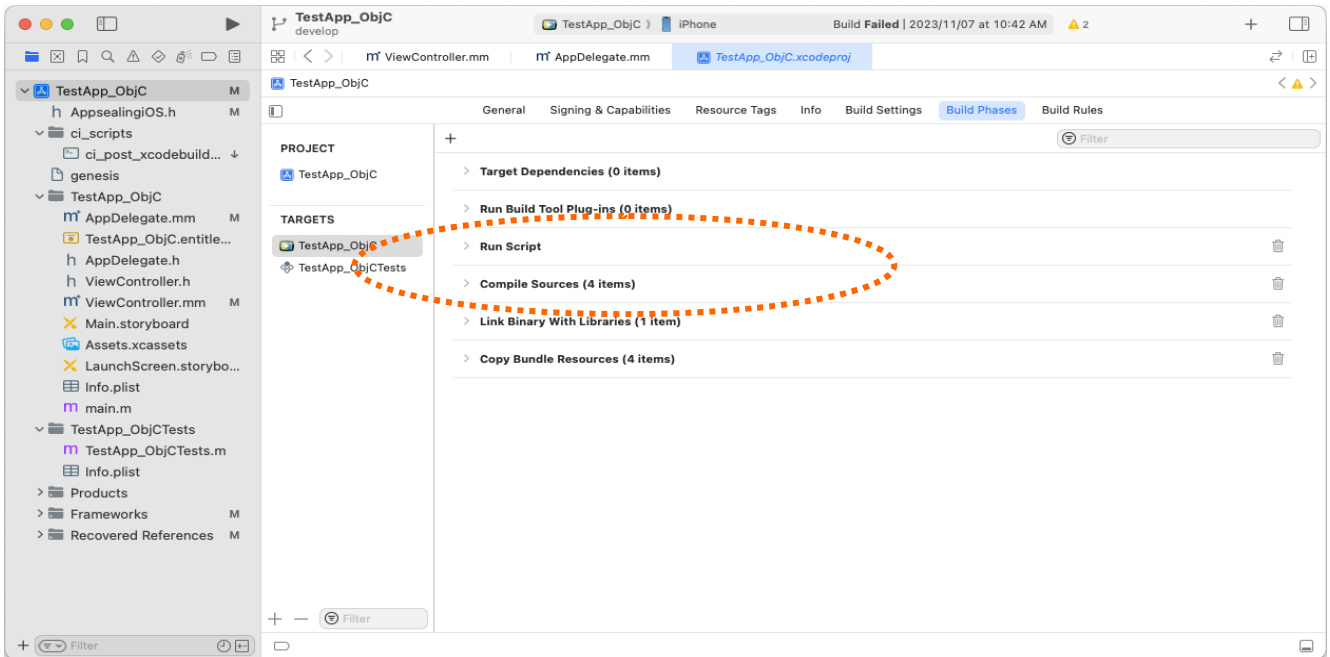


아래 화면과 같이 "Run Script" 단계가 추가되고 그 안에 스크립트 명령을 추가할 수 있는 박스가 표시됩니다.



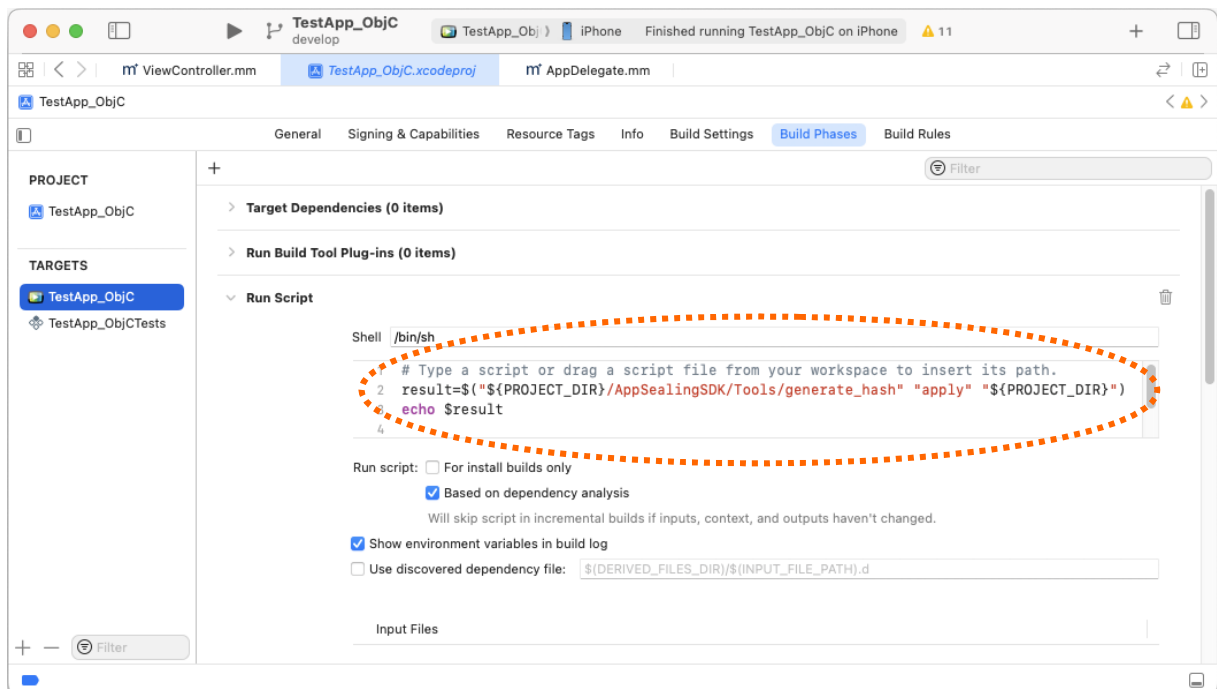
"Run Script" 좌측의 뒤집힌 캐럿(v) 아이콘을 클릭해 펼쳐진 스크립트 내용을 접은 후 마우스

왼쪽 버튼으로 "Run Script" 항목을 잡아 끌어 "Compile Sources (## items)" 항목 위로 이동시킵니다. 아래 화면과 같이 되어야 합니다.

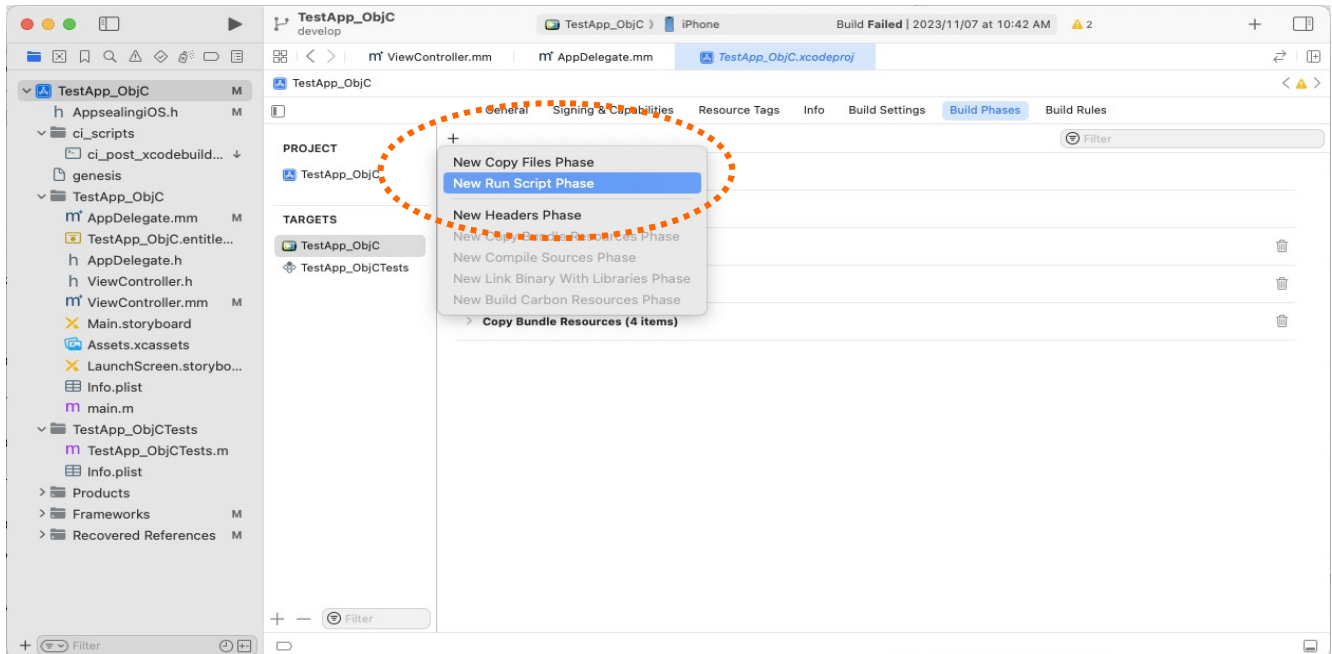


다시 "Run Script" 좌측의 ">" 아이콘을 클릭해 스크립트 항목을 펼친 후에 스크립트 박스에 아래 내용을 복사하여 붙여넣기로 입력합니다.

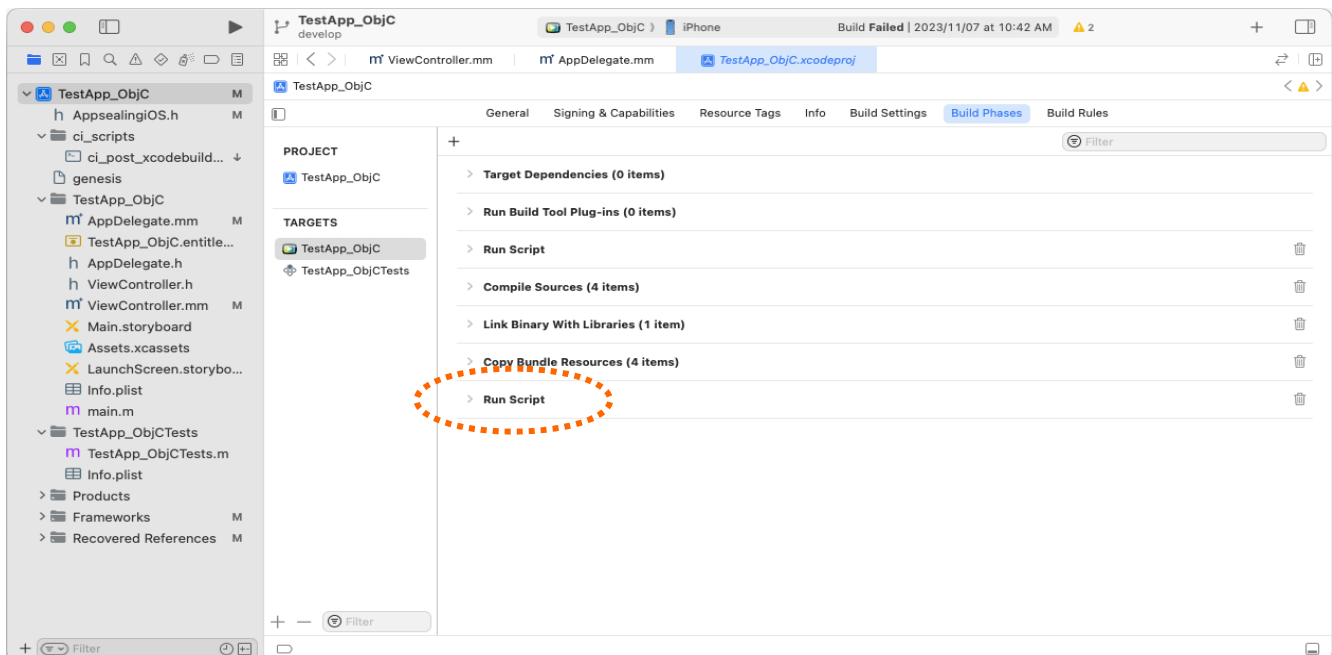
```
result=$(("${PROJECT_DIR}/AppSealingSDK/Tools/generate_hash" "apply" "${PROJECT_DIR}")
echo $result
```



다음은 좌측 상단의 "+" 아이콘을 한번 더 클릭하여 메뉴를 표시하고 "New Run Script Phase" 항목을 추가로 선택합니다.

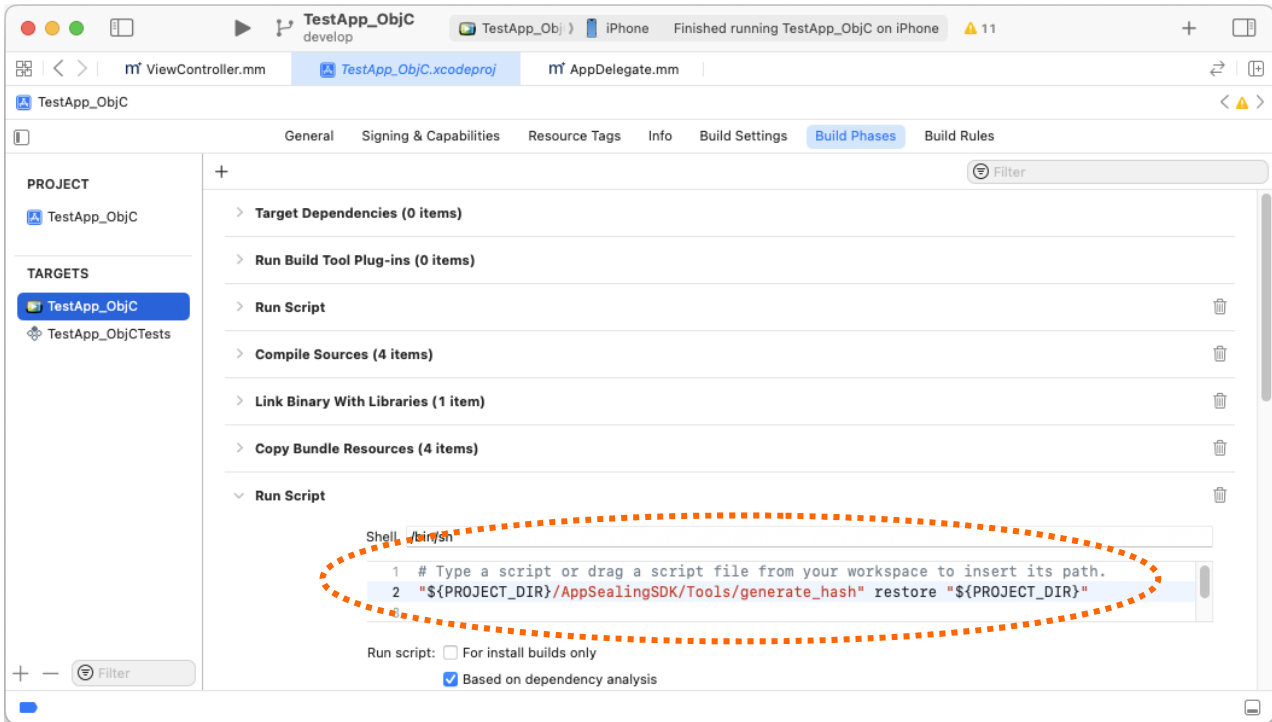


이제 아래 화면과 같이 맨 아래 쪽에 "Run Script" 항목이 하나 더 추가가 되어야 합니다.



“Run Script” 좌측의 “>” 아이콘을 클릭해 스크립트 항목을 펼친 후 스크립트 박스에 아래 내용을 복사하여 붙여넣기로 입력합니다.

```
"${PROJECT_DIR}/AppSealingSDK/Tools/generate_hash " "restore" "${PROJECT_DIR}"
```



8-2 문자열 암호화 태그 추가

이제 문자열 암호화를 위한 Build Phase 설정 작업은 모두 완료됐습니다. 지금부터는 실제 소스 코드에서 암호화를 적용할 문자열에 태그를 추가하는 작업을 진행해야 합니다. 암호화 태그는 “\0x1\0x1\0x1\0x1”가 사용되며 문자열 맨 끝 부분에 추가해야 됩니다.

e.g)

“This is my string” → “This is my string\0x1\0x1\0x1\0x1”

“Special char !@#\$%^&'\\" → “Special char !@#\$%^&'\\"

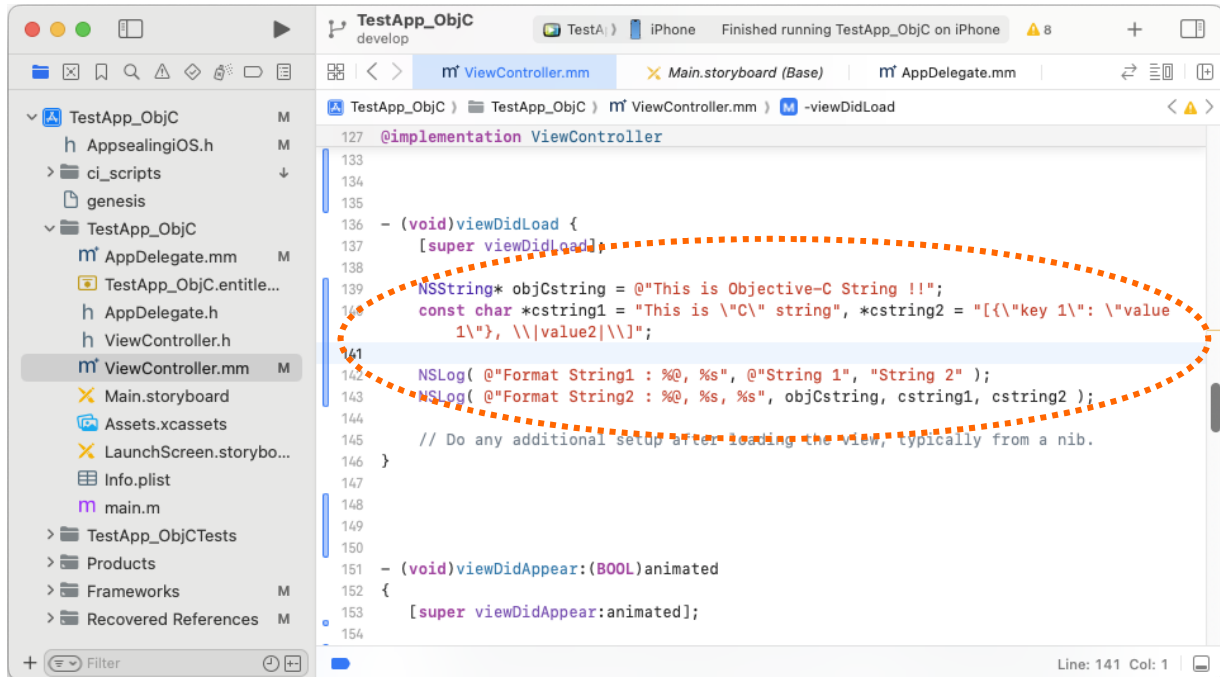
“문자열을 암호화 합니다” → “문자열을 암호화 합니다\0x1\0x1\0x1\0x1”

“文字列を暗号化します” → “文字列を暗号化します\0x1\0x1\0x1\0x1”

“加密字符串” → “加密字符串\0x1\0x1\0x1\0x1”

"Cifrar la cadena" → "Cifrar la cadena\0x1\0x1\0x1\0x1"

실제 소스 코드에서 문자열에 태그를 적용해 보겠습니다. TestApp_ObjC 프로젝트의 경우 Objective-C 언어를 기반으로 하고 있어서 소스 코드 상에 C/C++ 문자열과 Objective-C 문자열을 모두 포함할 수 있습니다.



위 코드의 viewDidLoad 함수에 있는 4개의 Objective-C 문자열과 3개의 C 문자열에 암호화를 적용해 보겠습니다.

[Objective-C 문자열]

@\"This is Objective-C String !\" → @\"This is Objective-C String !\0x1\0x1\0x1\0x1\"

@\"Format String1 : %@, %s\" → @\"Format String1 : %@, %s\0x1\0x1\0x1\0x1\"

@\"String 1\" → @\"String 1\0x1\0x1\0x1\0x1\"

@\"Format String2 : %@, %s, %s\" → @\"Format String2 : %@, %s, %s\0x1\0x1\0x1\0x1\"

[C 문자열]

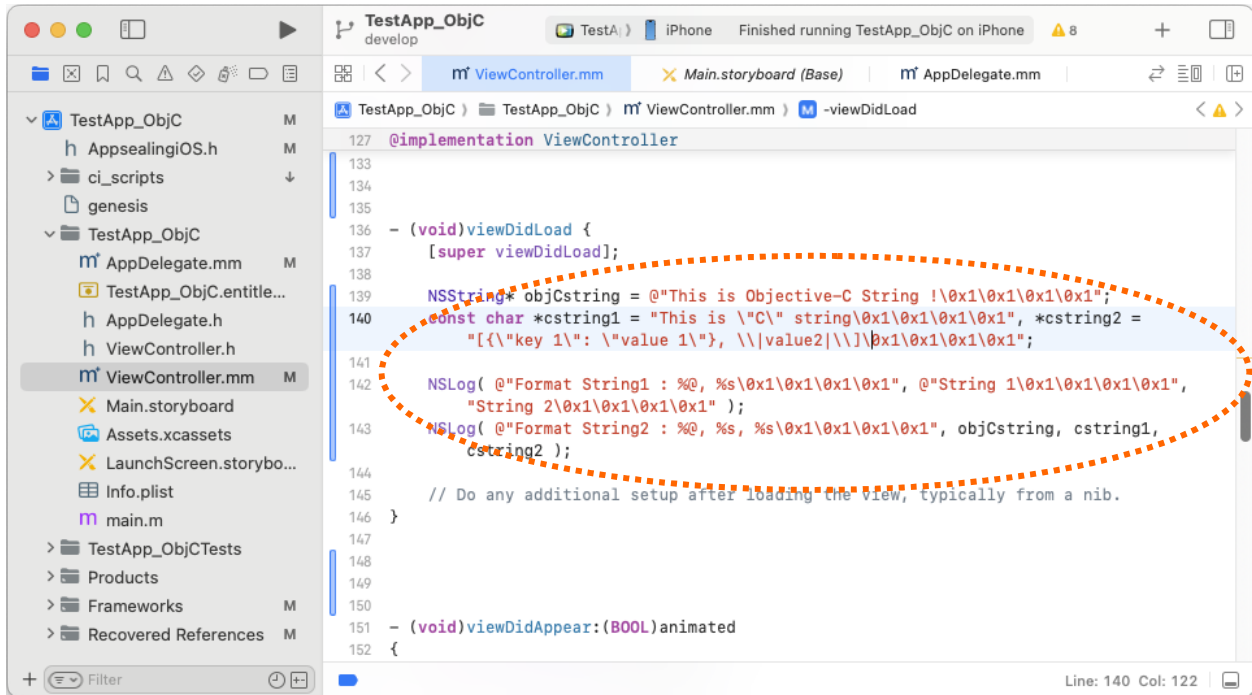
\"This is \\\"C\\\" string\" → \"This is \\\"C\\\" string\0x1\0x1\0x1\0x1\"

\"[{\\\"key 1\\\": \\\"value 1\\\"}, \\\"value2\\\"]\" →

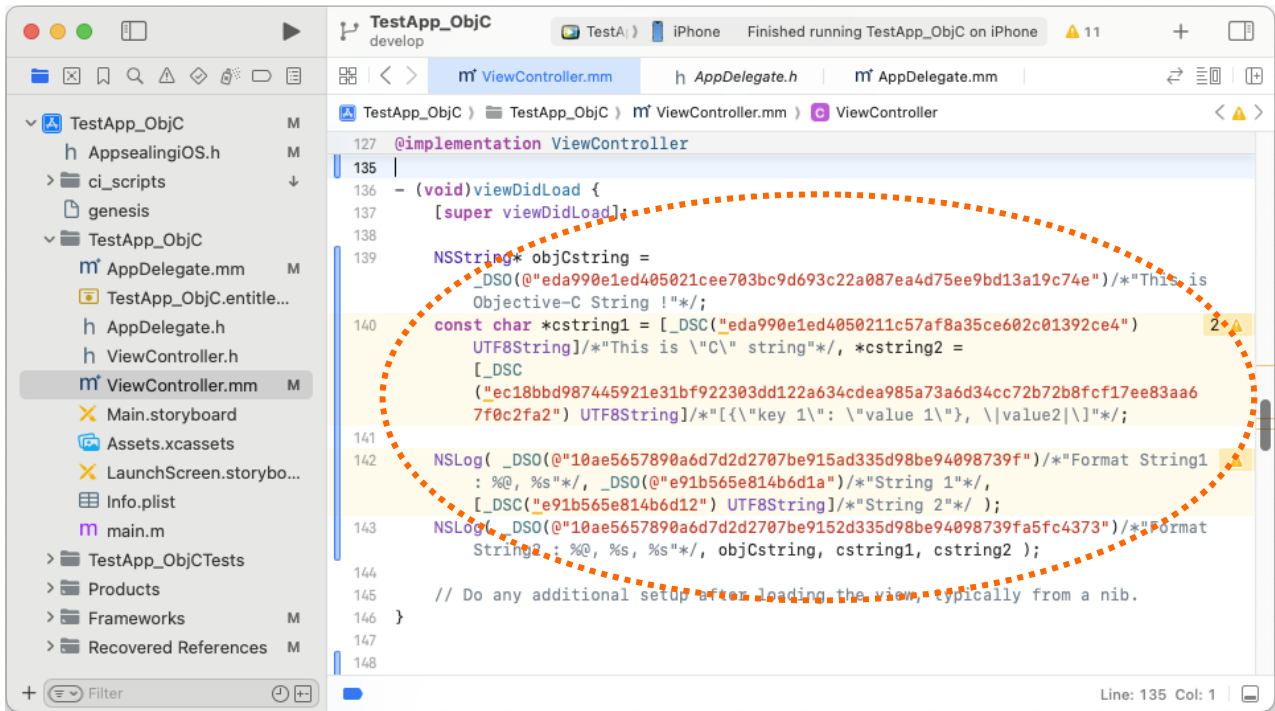
```
"[{\"key 1\": \"value 1\"}, \\\|value2|\\|\\|\\0x1\\0x1\\0x1\\0x1"
```

```
"String 2" → "String 2\\0x1\\0x1\\0x1\\0x1"
```

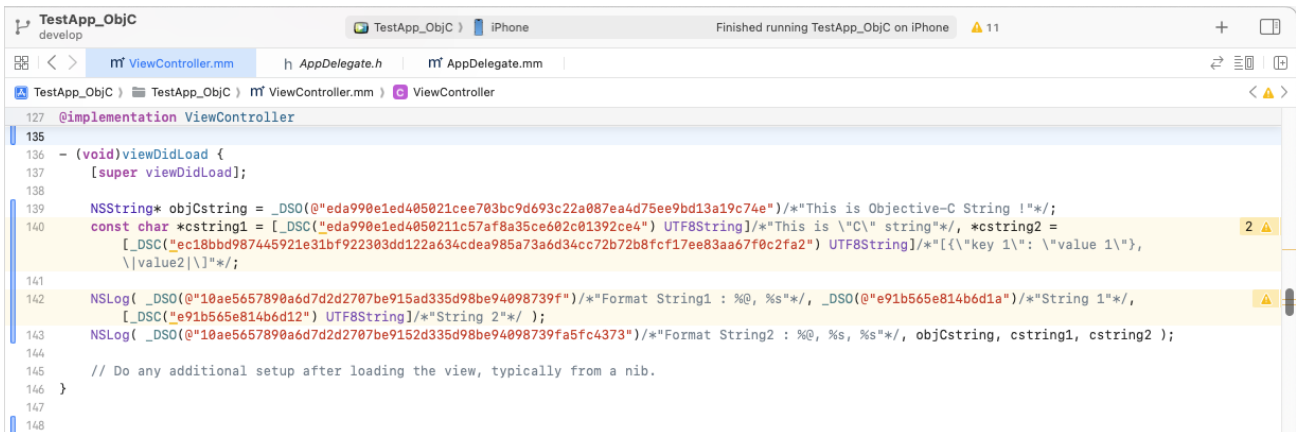
태그를 모두 적용한 코드의 모습은 아래와 같습니다.



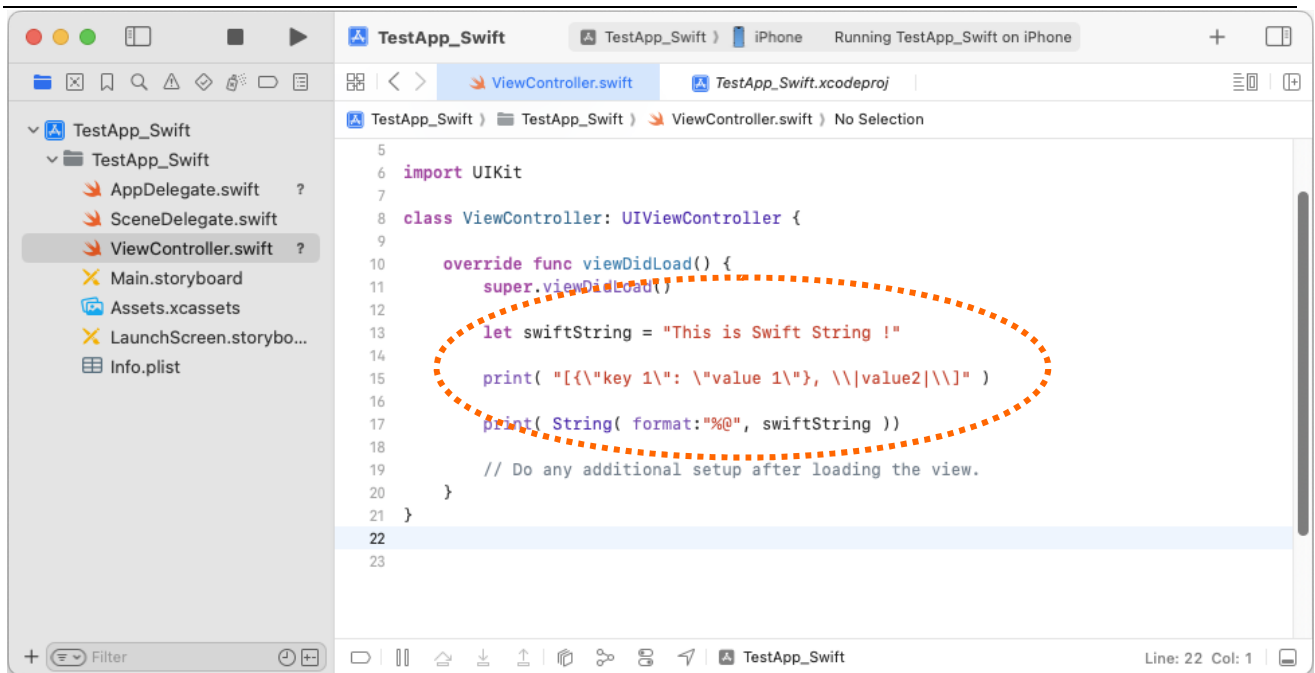
이렇게 태그가 적용된 Objective-C 코드는 컴파일 시점에 아래와 같이 암호화 된 문자열로 변환되어 컴파일 되고 컴파일이가 완료되면 다시 위 그림과 같은 원래의 태그 달린 문자열로 복원 됩니다. (따라서 아래와 같은 암호화 된 문자열이 포함된 Objective-C 코드는 일반적인 상황에서는 볼 수 없습니다)



아래는 줄바꿈이 최소화 되도록 화면을 넓게 확대한 코드의 모습입니다.



Swift 소스 코드의 경우에도 태그 적용은 동일한 방법으로 이루어 집니다.

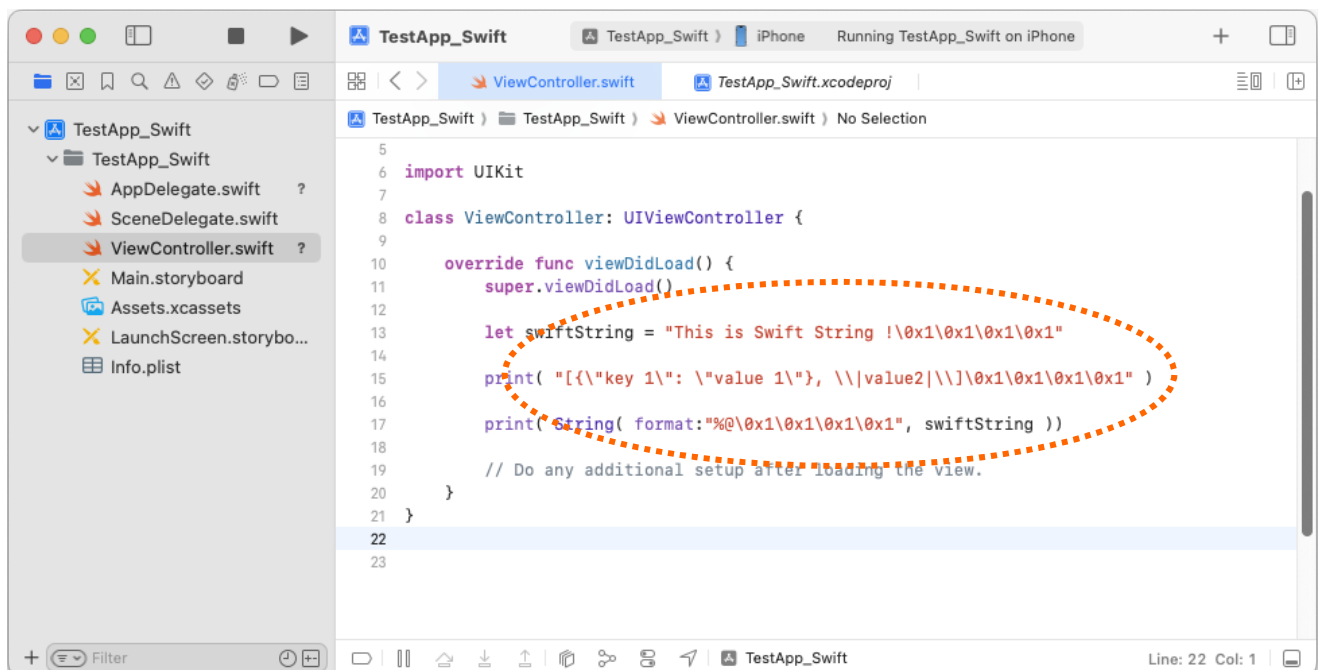


"This is Swift string" → "This is Swift string\\0x1\\0x1\\0x1\\0x1"

"[{"key 1": "value 1"}, "\\value2\\"]" →

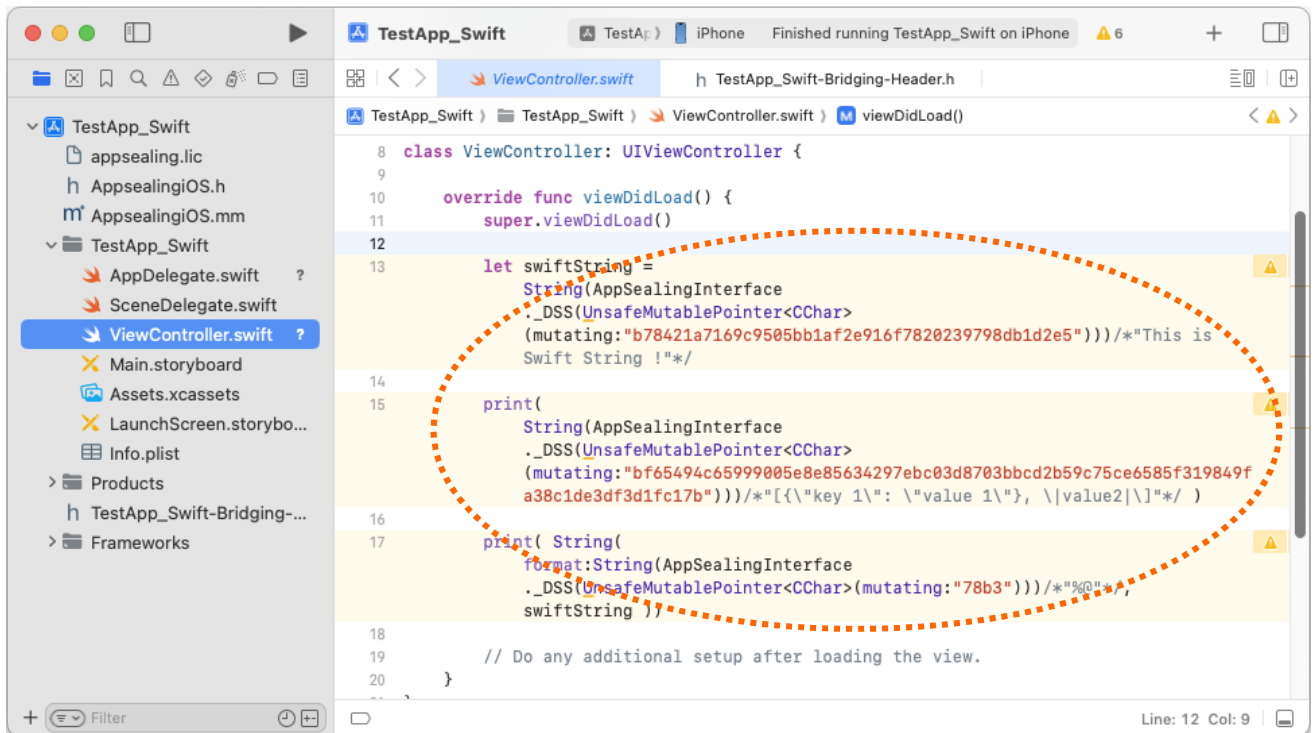
"[{"key 1": "value 1"}, "\\value2\\"\\0x1\\0x1\\0x1\\0x1"

"%@" → "%@\\0x1\\0x1\\0x1\\0x1"



태그가 적용된 Swift 코드는 컴파일 시점에 아래와 같이 암호화 된 문자열로 변환되어 컴파일 되고 컴파일이 완료되면 다시 위 그림과 같은 원래의 태그 달린 문자열로 복원됩니다. (마찬가

지로 아래와 같은 암호화 된 문자열이 포함된 Swift 코드도 일반적인 상황에서는 볼 수 없습니다)



8-3 Swift 문자열 암호화 주의 사항

- 다음과 같은 문자열 열거형을 사용할 경우 원시값으로 지정된 문자열에는 태그를 추가할 수 없습니다. 태그를 추가할 경우 컴파일 오류가 발생하게 됩니다.

```
enum StringEnum: String
{
    case first = "FIRST"
    case second = "SECOND"
    case third = "THIRD"
}
```



```
enum StringEnum: String
{
    case first = "FIRST\0x1\0x1\0x1\0x1"
    case second = "SECOND\0x1\0x1\0x1\0x1"
    case third = "THIRD\0x1\0x1\0x1\0x1"
}
```

- 다음과 같은 멀티라인 문자열에는 태그를 추가할 수 없습니다.

```
let str = ""
    This is multi
    line string,
    do not use tag
    ""
```



```
let str = ""
    This is multi\0x1\0x1\0x1\0x1
    line string,\0x1\0x1\0x1\0x1
    do not use tag\0x1\0x1\0x1\0x1
    ""
```

- 다음과 같이 문자열 보간법이 적용된 문자열에는 태그를 추가할 수 없습니다.

```
var city = "Seoul"
var message = "Welcome to \$(city)!"
print( "Message : \$(message)" )
```



```
var city = "Seoul"
var message = "Welcome to \$(city)!\0x1\0x1\0x1\0x1"
print( "Message : \$(message)\0x1\0x1\0x1\0x1" )
```

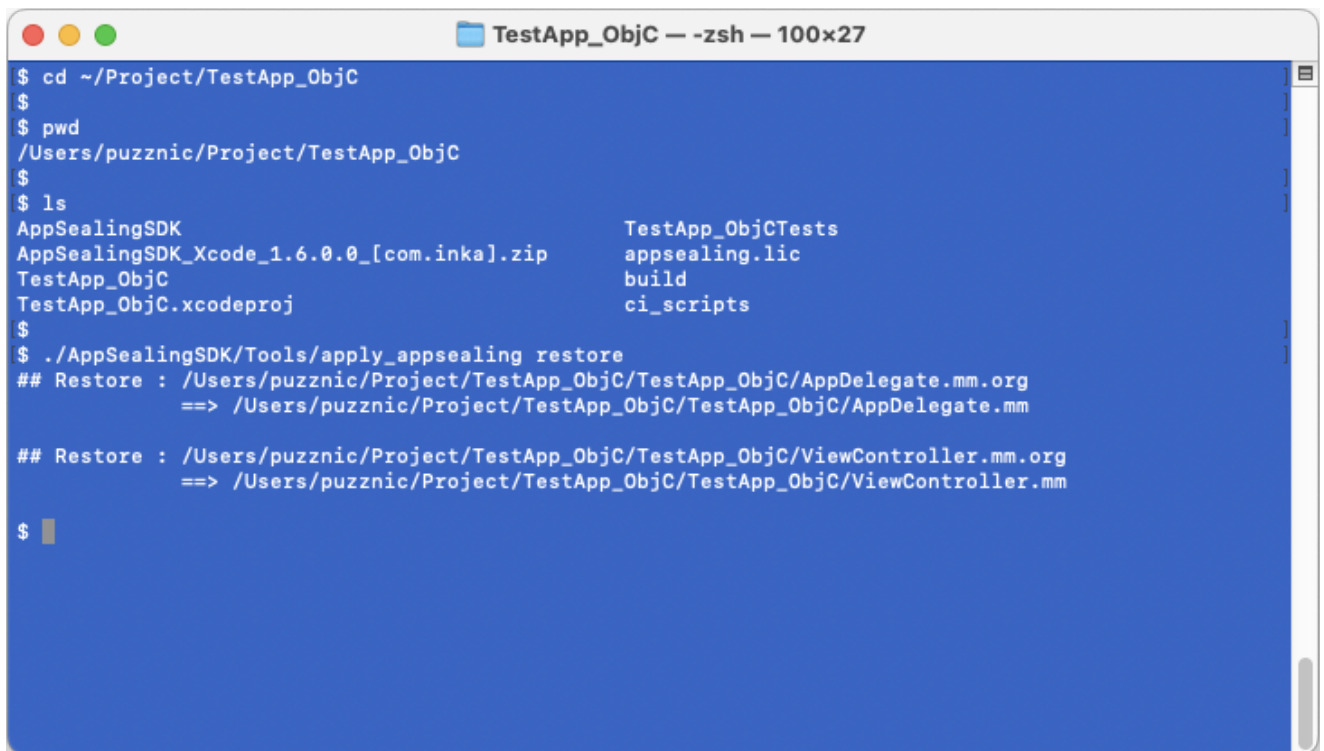
8-4 컴파일 오류 대응 : 암호화 된 문자열 복원

태그가 추가된 문자열은 컴파일 직전 암호화 된 문자열로 변경되고 컴파일 이후 원래의 문자열로 복원됩니다(8-1 Run Script). 그러나 컴파일 오류가 발생하거나 어떤 원인으로 컴파일 과정이 중단 되었을 경우 컴파일 이후의 스크립트가 실행되지 않고 이는 암호화 된 문자열이 복원되지 않은 채 소스 코드 상에 그대로 남아 있는 문제로 이어지게 됩니다.

이 경우 암호화 된 문자열을 원래대로 복원하는 방법은 두가지가 있습니다.

첫번째는 스크립트 실행을 통한 복원입니다. 터미널 앱을 실행하고 프로젝트 디렉토리로 이동합니다. 그리고 나서 AppSealingSDK/Tools/generate_hash 스크립트를 “restore” 파라미터와 함께 실행하면 원본 코드가 복원됩니다.

```
$ ./AppSealingSDK/Tools/generate_hash restore
```

A screenshot of a macOS terminal window titled "TestApp_ObjC — zsh — 100x27". The terminal shows the following commands and output:

```
$ cd ~/Project/TestApp_ObjC
$
$ pwd
/Users/puzznic/Project/TestApp_ObjC
$
$ ls
AppSealingSDK                               TestApp_ObjCTests
AppSealingSDK_Xcode_1.6.0.0_[com.inka].zip   appsealing.lic
TestApp_ObjC                                 build
TestApp_ObjC.xcodeproj                       ci_scripts
$
$ ./AppSealingSDK/Tools/apply_appsealing restore
## Restore : /Users/puzznic/Project/TestApp_ObjC/TestApp_ObjC/AppDelegate.mm.org
==> /Users/puzznic/Project/TestApp_ObjC/TestApp_ObjC/AppDelegate.mm

## Restore : /Users/puzznic/Project/TestApp_ObjC/TestApp_ObjC/ViewController.mm.org
==> /Users/puzznic/Project/TestApp_ObjC/TestApp_ObjC/ViewController.mm

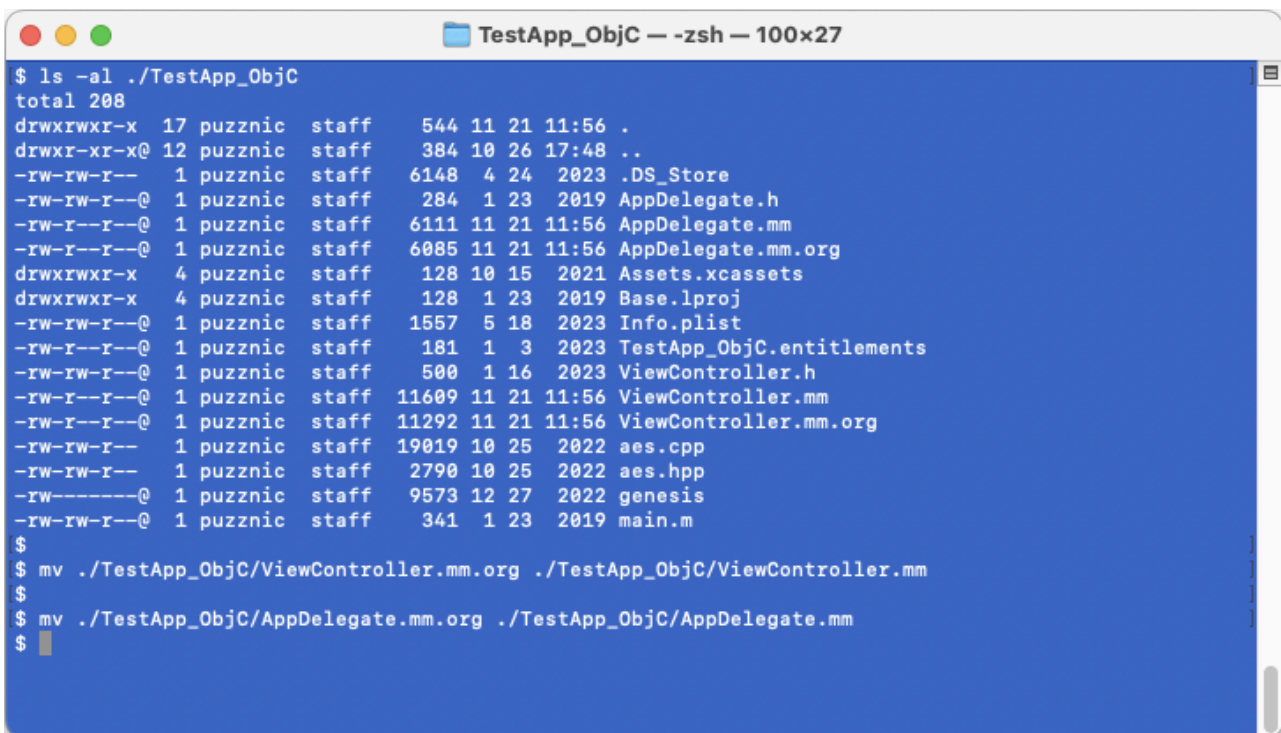
$
```

스크립트를 실행하면 예제로 사용된 TestApp_ObjC 프로젝트의 AppDelegate.mm 및 ViewController.mm 두 개의 소스코드가 복원되었다는 메시지가 출력되는 것을 볼 수 있습니다.

두번째 방법은 터미널에서 쉘 코맨드를 직접 입력해 백업되어 있는 코드 파일을 원래 이름으로 되돌리는 것입니다. TestApp_ObjC 프로젝트를 예로 들면 TestApp_ObjC 폴더 아래 있는 TestApp_ObjC 서브 폴더 내용을 출력해 보면 아래 화면과 같이 두 개의 소스 코드가 .org 라는 확장자를 이용해 백업되어 있는 것을 확인할 수 있습니다.

이제 아래의 커맨드를 입력해서 두 개의 파일을 원래 이름으로 되돌립니다.

```
$ mv ./TestApp_ObjC/ViewController.mm.org ./TestApp_ObjC/ViewController.mm
$ mv ./TestApp_ObjC/AppDelegate.mm.org ./TestApp_ObjC/AppDelegate.mm
```



```
TestApp_ObjC — zsh — 100x27
$ ls -al ./TestApp_ObjC
total 208
drwxrwxr-x  17 puzznic  staff   544 11 21 11:56 .
drwxr-xr-x@  12 puzznic  staff   384 10 26 17:48 ..
-rw-rw-r--   1 puzznic  staff  6148  4 24  2023 .DS_Store
-rw-rw-r--@   1 puzznic  staff   284  1 23  2019 AppDelegate.h
-rw-r--r--@   1 puzznic  staff  6111 11 21 11:56 AppDelegate.mm
-rw-r--r--@   1 puzznic  staff  6085 11 21 11:56 AppDelegate.mm.org
drwxrwxr-x   4 puzznic  staff   128 10 15  2021 Assets.xcassets
drwxrwxr-x   4 puzznic  staff   128  1 23  2019 Base.lproj
-rw-rw-r--@   1 puzznic  staff  1557  5 18  2023 Info.plist
-rw-r--r--@   1 puzznic  staff   181  1  3  2023 TestApp_ObjC.entitlements
-rw-rw-r--@   1 puzznic  staff   500  1 16  2023 ViewController.h
-rw-r--r--@   1 puzznic  staff 11609 11 21 11:56 ViewController.mm
-rw-r--r--@   1 puzznic  staff 11292 11 21 11:56 ViewController.mm.org
-rw-rw-r--   1 puzznic  staff 19019 10 25  2022 aes.cpp
-rw-rw-r--   1 puzznic  staff  2790 10 25  2022 aes.hpp
-rw-----@   1 puzznic  staff  9573 12 27  2022 genesis
-rw-rw-r--@   1 puzznic  staff   341  1 23  2019 main.m

$
$ mv ./TestApp_ObjC/ViewController.mm.org ./TestApp_ObjC/ViewController.mm
$
$ mv ./TestApp_ObjC/AppDelegate.mm.org ./TestApp_ObjC/AppDelegate.mm
$
```

Part 9. Fastlane 프로젝트에 적용

9-1 프로젝트 Fastfile 설정

AppSealing SDK는 Fastlane을 적용한 프로젝트에도 사용할 수 있습니다. 일반적으로 Fastlane 툴을 Xcode 프로젝트에 사용하는 이유는 앱 빌드 및 패키징 과정을 자동화하고 이후의 배포 과정도 자동화 하기 위한 것입니다. Xcode 프로젝트에 AppSealing SDK를 적용한 경우 앱 빌드 이후에 추가로 진행해야 되는 과정들이 있지만 Fastlane이 적용된 경우 Fastfile 스크립트에 AppSealing 적용 과정을 추가함으로써 기존의 자동화 과정을 그대로 유지할 수 있습니다.

이 장에서는 Fastlane의 빌드 및 배포 자동화를 그대로 유지하기 위해 스크립트를 어떻게 수정하고 반영해야 하는지를 설명합니다.

다음은 TestApp_Swift라는 프로젝트에 Fastlane을 적용한 상태의 Fastfile 코드입니다. (주석 제외). 앱을 빌드하고 서명해서 Testflight로 업로드하는 과정이 포함되어 있습니다. Apple Store에 업로드 하는 스크립트도 동일한 구조를 가지므로 Testflight에 대해서만 설명합니다.

프로젝트 기본 Fastfile

```
default_platform(:ios)

platform :ios do
  desc "Push a new beta build to TestFlight"
  lane :beta do
    increment_build_number(xcodeproj: "TestApp_Swift.xcodeproj")
    build_app(scheme: "TestApp_Swift")
    upload_to_testflight
  end
end
```

일반적인 경우는 이 파일에 크게 수정을 하지 않은 상태로 "fastlane beta" 명령어를 실행해서 빌드와 배포를 하게 됩니다. 여기에 AppSealing SDK를 적용한 경우는 빌드와 배포 사이에 추가 작업이 필요하기 때문에 fastfile 스크립트를 아래와 같이 수정해야 합니다. 원본 스크립트는 프로젝트 이름과 파일명이 문자열 값으로 직접 입력되어 있지만 수정할 스크립트는 모든 프로젝트에 적용할 수 있도록 직접 값을 지정하는 대신 프로젝트 폴더에서 필요한 값을 추출해서 사용하도록 되어 있습니다. 따라서 프로젝트 명이 TestApp_Swift가

아니더라도 다른 프로젝트에 모두 동일한 스크립트를 적용할 수 있습니다.

Fastfile 을 문서 편집기로 열고 모든 코드를 아래 코드로 대체합니다. 다만 아래 코드 중에서 `FASTLANE_USER`, `FASTLANE_APPLE_APPLICATION_SPECIFIC_PASSWORD`, `PROFILE` 값은 실제 사용되는 값으로 바뀌어서 적용해야 합니다. `FASTLANE_USER` 은 Apple 계정, `FASTLANE_APPLE_APPLICATION_SPECIFIC_PASSWORD` 은 앱 전용 비밀번호를 입력해야 합니다. 앱 전용 비밀번호 발급 방법에 대해서는 64 페이지의 내용을 참고하면 됩니다.

`PROFILE` 값은 배포 단계에 사용하는 프로비저닝 프로파일의 이름을 입력해야 합니다.

AppSealing SDK 적용 시의 Fastfile

```
default_platform(:ios)

before_all do
  ENV["FASTLANE_USER"] = "#YOUR_APPLE_ID#"
  ENV["FASTLANE_APPLE_APPLICATION_SPECIFIC_PASSWORD"] = "#YOUR_APP_SPECIFIC_PASSWORD#"
  ENV['PROFILE'] = "#YOUR_DISTRIBUTION_PROVISIONING_PROFILE_NAME#"
end

platform :ios do
  desc "Push a new beta build to TestFlight"
  lane :beta do
    # 프로젝트 루트 경로 가져오기
    project_root = File.expand_path("../", __dir__)

    # Xcode 프로젝트 또는 워크스페이스 파일 동적으로 검색
    xcode_file_path = Dir.glob(File.join(project_root, "*.xcworkspace")).first ||
Dir.glob(File.join(project_root, "*.xcodeproj")).first
    if xcode_file_path.nil?
      UI.user_error!("[AppSealing] No Xcode project or workspace found in the project
root directory: #{project_root}")
    end

    # 프로젝트 이름 및 스킴 자동 추출
    project_name = File.basename(xcode_file_path, File.extname(xcode_file_path)) #
확장자를 제거한 프로젝트 이름

    # 스킴 자동 검색
    scheme_name = ""
    Dir.chdir(File.dirname(xcode_file_path)) do
      scheme_name = sh("xcodebuild -list -#{xcode_file_path.end_with?('xcworkspace')} ?
'workspace' : 'project'} #{File.basename(xcode_file_path)} | grep 'Schemes:' -A 1 |
tail -n 1").strip
    end
  end
end
```

```
    if scheme_name.empty?  
      UI.user_error!("[AppSealing] Failed to retrieve scheme name from Xcode project or  
workspace.")  
    end  
  
    # 번들 ID 동적으로 추출  
    bundle_id = ""  
    Dir.chdir(File.dirname(xcode_file_path)) do  
      bundle_id_command = "xcodebuild -showBuildSettings -scheme #{scheme_name} | grep  
'PRODUCT_BUNDLE_IDENTIFIER' | awk -F ' = ' '{print $2}'"  
      bundle_id_output = sh(bundle_id_command).lines.map(&:strip).reject { |line|  
line.include?("WARNING") || line.empty? }  
      bundle_id = bundle_id_output.last.strip  
    end  
  
    if bundle_id.empty?  
      UI.user_error!("[AppSealing] Failed to retrieve Bundle ID from Xcode project.")  
    end  
  
    # Team ID 동적으로 추출  
    team_id = ""  
    Dir.chdir(File.dirname(xcode_file_path)) do  
      team_id_command = "xcodebuild -showBuildSettings -scheme #{scheme_name} | grep  
'DEVELOPMENT_TEAM' | awk -F ' = ' '{print $2}'"  
      team_id_output = sh(team_id_command).lines.map(&:strip).reject { |line|  
line.include?("WARNING") || line.empty? }  
      team_id = team_id_output.last.strip  
    end  
  
    if team_id.empty?  
      UI.user_error!("[AppSealing] Failed to retrieve Team ID from Xcode project.")  
    end  
  
    UI.message "[AppSealing] Project Name: #{project_name}"  
    UI.message "[AppSealing] Scheme Name: #{scheme_name}"  
    UI.message "[AppSealing] Bundle ID: #{bundle_id}"  
    UI.message "[AppSealing] Team ID: #{team_id}"  
  
    # 빌드 및 IPA 파일 생성  
    archive_path = File.join(project_root, "build", "#{project_name}.xcarchive")  
    ipa_output_path = File.join(project_root, "build")  
  
    # build_ios_app 호출 시 workspace 와 project 옵션 분리 처리  
    build_options = {  
      scheme: scheme_name,  
      export_method: "app-store",  
      clean: true,  
      output_directory: ipa_output_path,  
      output_name: "#{project_name}.ipa",  
      export_options: {  
        provisioningProfiles: {
```

```

        bundle_id => ENV["PROFILE"]
    }
}
}

if xcode_file_path.end_with?(".xcworkspace")
    build_options[:workspace] = xcode_file_path
    build_options[:project] = nil
else
    build_options[:project] = xcode_file_path
    build_options[:workspace] = nil
end

build_ios_app(build_options)

# .ipa 파일 경로 설정
ipa_path = File.join(ipa_output_path, "#{project_name}.ipa")

unless File.exist?(ipa_path)
    UI.user_error!("[AppSealing] IPA file not found at path: #{ipa_path}")
end

UI.message "[AppSealing] IPA Path: #{ipa_path}"

# generate_hash 스크립트 경로 동적 검색 및 실행
generate_hash_script = Dir.glob(File.join(project_root, "**/generate_hash")).first
|| File.join(project_root, "AppSealingSDK", "Tools", "generate_hash")

unless File.exist?(generate_hash_script)
    UI.user_error!("[AppSealing] generate_hash script not found at path:
#{generate_hash_script}")
end

unless File.executable?(generate_hash_script)
    sh("chmod +x '#{generate_hash_script}'")
end

sh("#{generate_hash_script} #{File.absolute_path(ipa_path)}")

# IPA 파일을 TestFlight 로 업로드
begin
    upload_to_testflight(
        ipa: ipa_path,
        skip_waiting_for_build_processing: true, # 빌드 처리 대기 시간 생략
    )
    UI.success("[AppSealing] Upload to TestFlight completed successfully!")

rescue => e
    UI.error("[AppSealing] Upload to TestFlight failed with error: #{e.message}")
    raise e
end

```

```

ensure
  # 빌드 디렉토리 삭제
  if Dir.exist?(ipa_output_path)
    UI.message("[AppSealing] Deleting build directory: #{ipa_output_path}")
    FileUtils.rm_rf(ipa_output_path)
    UI.message("[AppSealing] Build directory deleted.")
  else
    UI.message("[AppSealing] Build directory not found or already deleted.")
  end
end
end
end
end

```

위 코드는 SDK 에 포함된 CodeSamples.txt 파일에도 포함되어 있으므로 해당 파일에서 복사해서 사용할 수 있습니다.

코드를 수정하고 기존과 동일하게 “fastlane beta” 명령을 실행하면 앱을 빌드하고 IPA 로 export 한 후 generate_hash 스크립트를 자동으로 실행하고 AppSealing 이 적용된 IPA 를 testflight 로 업로드 하게 됩니다.

9-2 Github Action과 Fastlane 동시 적용

Xcode 프로젝트 코드를 Github에 올려 두고 Github Action을 사용하여 코드 푸시가 이루어진 경우 자동으로 빌드와 배포가 이루어지도록 배포 파이프라인을 구축할 수 있습니다. 이 경우 Action 정의를 위해 아래와 같은 yaml 형식의 빌드 스크립트 파일을 추가하게 됩니다

이 파일에는 소스코드 브랜치 이름이 develop으로 정의되어 있고 프로젝트에 포함된 fastfile을 실행하기 위해 Fastlane 설치 단계가 포함되어 있습니다. 또한 fastfile 실행에 필요한 환경 변수를 설정하기 위해 APP_STORE_CONNECT_KEY_ID, APP_STORE_CONNECT_ISSUER_ID, APP_STORE_CONNECT_PRIVATE_KEY값을 env에서 지정하고 있습니다. 로컬 프로젝트에 Fastlane을 적용하는 단계 혹은 Xcode Cloud를 사용하는 과정에서는 앱 전용 비밀번호를 사용 했으나 이 단계에서는 App Store Connect API를 사용합니다. App Store Connect API를 사용하기 위해서는 Apple account 페이지에서 API Key를 발급받아야 하며 여기서 발급된 개인키(APP_STORE_CONNECT_PRIVATE_KEY)와 키 ID(APP_STORE_CONNECT_KEY_ID), 발급자 ID(APP_STORE_CONNECT_ISSUER_ID)를 fastfile 스크립트로 전달해 주어야 합니다. 전달하는 실제 값들은 스크립트에 직접 입력하지 않고 Github Action의 secrets 변수로 저장한 다음 해당 변수를 참조해서 전달하는 방식으로 작성되어 있습니다.

프로젝트 기본 Github Action 빌드 스크립트 (ios_build.yml)

```
name: iOS Build & Deploy
on:
  push:
    branches:
      - develop

jobs:
  release-ios:
    name: Build and release iOS app
    runs-on: macos-latest
    steps:
      - uses: actions/checkout@v2

      - uses: actions/setup-ruby@v1
        with:
          ruby-version: '3.1.2'

      - name: Install Fastlane
        run: cd ios && bundle install

      - name: Install pods
        run: cd ios && pod install

      - name: Execute Fastlane
        env:
          APP_STORE_CONNECT_KEY_ID: ${ secrets.KEY_ID }
          APP_STORE_CONNECT_API_KEY: ${ secrets.API_KEY }
          APP_STORE_CONNECT_PRIVATE_KEY: ${ secrets.PRIVATE_KEY }
        run: cd ios && fastlane release
```

아래 코드는 이 환경 변수들을 넘겨 받아 실제 fastlane 작업을 수행하게 될 Fastfile 코드입니다. 여기에는 App Store Connect API를 사용하기 위한 설정 단계 및 앱 빌드, 업로드를 위한 과정이 포함되어 있습니다.

프로젝트 기본 Fastfile

```
default_platform(:ios)

platform :ios do
  desc "Release to App Store"
```

```
lane :release do
  # 앱 스토어 커넥트 API 인증
  app_store_connect_api_key(
    key_id: ENV["APP_STORE_CONNECT_KEY_ID"],
    issuer_id: ENV["APP_STORE_CONNECT_ISSUER_ID"],
    key_content: ENV["APP_STORE_CONNECT_PRIVATE_KEY"]
  )

  # 앱 빌드
  build_app(
    scheme: "프로젝트스킴명",
    export_method: "app-store"
  )

  # 업로드
  upload_to_testflight(
    skip_waiting_for_build_processing: true,
    distribute_external: true
  )
end
end
```

기존에 App Store Connect API Key가 발급되어 있지 않은 상태라면 다음 링크의 내용을 참조하여 키를 발급받은 후 아래 과정을 진행합니다.

(<https://developer.apple.com/documentation/appstoreconnectapi/creating-api-keys-for-app-store-connect-api>)

이 상태에서 AppSealing SDK를 적용할 경우의 변경 사항에 대해 설명하겠습니다. AppSealing SDK를 적용할 경우 archive와 export 단계를 거쳐 생성된 IPA에 대해 스크립트 작업을 수행해야 하고 이 과정에서 IPA에 대한 재서명이 이루어지기 때문에 추가로 서명용 인증서와 프로비저닝 프로파일이 필요합니다. 인증서는 스토어 업로드를 위한 배포용 인증서여야 하고 프로파일 역시 배포용 프로파일이 있어야 합니다. 따라서 프로젝트에 인증서를 PKCS#12 형식으로 변환하여 포함시키고 프로비저닝 프로파일을 포함시킨 상태로 github에 푸시를 해야 합니다. 이 때 인증서의 파일명은 "certificate.p12"가 되어야 하고 프로비저닝 프로파일의 파일명은 "distribution.mobileprovision"이 되어야 합니다. 파일명을 다르게 할 경우 아래 스크립트에서 파일명을 그에 맞게 수정해야 합니다. 프로비저닝 프로파일의 이름은 **PROFILE** 환경 변수에 직접 값을 입력하여 수정합니다.

인증서를 PKCS#12 형식으로 저장할 때 비밀번호는 "123456"으로 설정한 것으로 간주합니다. 만약 비밀번호가 다르다면 아래 스크립트에서 역시 비밀번호 문자열을 올바르게 수정해야 합니다.

Github에 업로드 된 인증서를 키체인에 설치하고 프로비저닝 프로파일을 설치하기 위해 ios_build.yml 스크립트를 다음과 같이 수정합니다.

Github Action + AppSealing SDK 적용 시의 ios_build.yml

```
name: iOS Build with Xcode 16

on:
  push:
    branches:
      - develop

jobs:
  build-ios:
    runs-on: macos-15

    steps:
      - name: Checkout code
        uses: actions/checkout@v3

      - name: Setup Xcode 16
        uses: maxim-lobanov/setup-xcode@v1
        with:
          xcode-version: '16.0'

      - name: Set up Signing Credentials
        run: |
          security create-keychain -p "temp_password" temp.keychain
          security import ./certificate.p12 -k temp.keychain -P "123456" -T
/usr/bin/codesign
          security set-key-partition-list -S apple-tool:,apple:,codesign: -s -k
"temp_password" temp.keychain
          security list-keychains -s temp.keychain
          security unlock-keychain -p "temp_password" temp.keychain
          security set-keychain-settings -lut 3600 temp.keychain

      - name: Set up Provisioning Profile from Project Folder
        run: |
          mkdir -p ~/Library/Developer/Xcode/UserData/Provisioning\ Profiles
          cp ./distribution.mobileprovision
~/Library/Developer/Xcode/UserData/Provisioning\ Profiles/

      # 5. Install Fastlane using Homebrew
      - name: Install Fastlane
        run: |
          brew install fastlane

      # 6. Run Fastlane lane for TestFlight upload
      - name: Run Fastlane Lane
        env:
          PROFILE: "#YOUR_DISTRIBUTION_PROVISIONING_PROFILE_NAME#"
          APP_STORE_CONNECT_KEY_ID: ${ secrets.APP_STORE_CONNECT_KEY_ID }
```

```

APP_STORE_CONNECT_ISSUER_ID: ${ secrets.APP_STORE_CONNECT_ISSUER_ID }
APP_STORE_CONNECT_PRIVATE_KEY:
${ secrets.APP_STORE_CONNECT_PRIVATE_KEY }
run: |
    fastlane beta --verbose

```

이 스크립트에는 "runs-on: macos-15"와 "xcode-version: '16.0'" 항목이 포함되어 있는데 이는 프로비저닝 프로파일 저장 경로가 Xcode 16 버전부터 변경되었기 때문입니다. Xcode 버전을 16으로 지정하지 않으면 프로파일이 잘못된 경로로 복사되고 이로 인해 IPA 후처리 단계에서 재서명에 실패하게 됩니다.

이제 앱 빌드 및 재서명, 업로드를 위한 작업에 AppSealing SDK 관련 작업이 반영되도록 Fastfile도 다음과 같이 수정합니다.

Github Action + AppSealing SDK 적용 시의 Fastfile

```

default_platform(:ios)

platform :ios do
  desc "Push a new beta build to TestFlight"
  lane :beta do
    require 'net/http'
    require 'uri'
    require 'json'
    require 'jwt'

    # 프로젝트 루트 경로 가져오기 (Fastfile 기준 상위 디렉토리)
    project_root = File.expand_path("../", __dir__)
    fastlane_path = File.expand_path(".", __dir__)

    KEY_ID = ENV['APP_STORE_CONNECT_KEY_ID'] # GitHub Actions Secret 에서 가져오기
    ISSUER_ID = ENV['APP_STORE_CONNECT_ISSUER_ID'] # GitHub Actions Secret 에서
    가져오기
    PRIVATE_KEY_CONTENT = ENV['APP_STORE_CONNECT_PRIVATE_KEY'] # 개인 키 내용을 환경
    변수로 가져오기

    unless KEY_ID && ISSUER_ID && PRIVATE_KEY_CONTENT
      UI.user_error!("Missing required App Store Connect credentials. Ensure
      APP_STORE_CONNECT_KEY_ID, APP_STORE_CONNECT_ISSUER_ID, and
      APP_STORE_CONNECT_PRIVATE_KEY are set as environment variables.")
    end

    # Xcode 프로젝트(.xcodproj) 또는 워크스페이스(.xcworkspace) 파일 자동 검색
    xcode_file_path = Dir.glob(File.join(project_root, "*.xcworkspace")).first ||
    Dir.glob(File.join(project_root, "*.xcodproj")).first
    if xcode_file_path.nil?

```

```

    UI.user_error!("[AppSealing] No Xcode project or workspace found in the
project root directory: #{project_root}")
end

# 프로젝트 이름 및 스킴 자동 추출
project_name = File.basename(xcode_file_path, File.extname(xcode_file_path)) #
확장자를 제거한 프로젝트 이름

# 스킴 자동 검색
scheme_name = ""
Dir.chdir(File.dirname(xcode_file_path)) do
  scheme_name = sh("xcodebuild -list -
#{xcode_file_path.end_with?('.xcworkspace') ? 'workspace' : 'project'}
#{File.basename(xcode_file_path)} | grep 'Schemes:' -A 1 | tail -n 1").strip
end

if scheme_name.empty?
  UI.user_error!("[AppSealing] Failed to retrieve scheme name from Xcode
project or workspace.")
end

UI.message "[AppSealing] Project Name: #{project_name}"
UI.message "[AppSealing] Scheme Name: #{scheme_name}"

# 번들 ID 동적으로 추출
bundle_id = ""
Dir.chdir(File.dirname(xcode_file_path)) do
  bundle_id_command = "xcodebuild -showBuildSettings -scheme #{scheme_name} |
grep 'PRODUCT_BUNDLE_IDENTIFIER' | awk -F ' = ' '{print $2}'"
  bundle_id_output = sh(bundle_id_command).lines.map(&:strip).reject { |line|
line.include?("WARNING") || line.empty? }
  bundle_id = bundle_id_output.last.strip
end

if bundle_id.empty?
  UI.user_error!("[AppSealing] Failed to retrieve Bundle ID from Xcode
project.")
end

UI.message "[AppSealing] Bundle ID: #{bundle_id}"

# Team ID 동적으로 추출
team_id = ""
Dir.chdir(File.dirname(xcode_file_path)) do
  team_id_command = "xcodebuild -showBuildSettings -scheme #{scheme_name} |
grep 'DEVELOPMENT_TEAM' | awk -F ' = ' '{print $2}'"
  team_id_output = sh(team_id_command).lines.map(&:strip).reject { |line|
line.include?("WARNING") || line.empty? }
  team_id = team_id_output.last.strip
end

```

```

if team_id.empty?
  UI.user_error!("[AppSealing] Failed to retrieve Team ID from Xcode project.")
end

UI.message "[AppSealing] Team ID: #{team_id}"

# JWT 토큰 생성 함수
def generate_jwt_token(private_key_content)
  private_key = OpenSSL::PKey::EC.new(private_key_content)
  payload = {
    iss: ISSUER_ID,
    exp: Time.now.to_i + 20 * 60, # 20 분 유효기간
    aud: "appstoreconnect-v1"
  }
  header = { kid: KEY_ID }
  JWT.encode(payload, private_key, 'ES256', header)
end

# Xcode 프로젝트에서 CFBundleShortVersionString (마케팅 버전) 가져오기 함수
def fetch_version_from_xcode(xcodeproj_path, scheme_name)
  version_command = "xcodebuild -project #{xcodeproj_path} -scheme
#{scheme_name} -showBuildSettings | grep MARKETING_VERSION | sed
's/[ ]*MARKETING_VERSION = //'
  version = `#{version_command}`.strip

  if version.empty?
    UI.user_error!("Failed to retrieve MARKETING_VERSION from Xcode project.")
  end

  UI.message("[AppSealing] Retrieved MARKETING_VERSION: #{version}")
  version
end

# 최신 빌드 번호 가져오기 함수
def fetch_latest_build_number jwt_token, bundle_id, version)
  # App ID 가져오기 (필요 시)
  app_id = fetch_app_id(jwt_token, bundle_id)

  # API 요청 URL
  uri =
URI("https://api.appstoreconnect.apple.com/v1/builds?filter[app]=#{app_id}&filter[p
reReleaseVersion.version]=#{version}&sort=-version")
  request = Net::HTTP::Get.new(uri)
  request['Authorization'] = "Bearer #{jwt_token}"

  # API 요청 실행
  response = Net::HTTP.start(uri.hostname, uri.port, use_ssl: true) do |http|
    http.request(request)
  end

  # 응답 처리

```

```

    if response.code.to_i == 200
      builds = JSON.parse(response.body)['data']
      if builds && !builds.empty?
        latest_build_number = builds.first['attributes']['version'].to_i
        return latest_build_number
      else
        UI.message("[AppSealing] No builds found for version #{version}.")
      end
    else
      UI.error("[AppSealing] Failed to fetch builds: #{response.code} -
#{response.body}")
    end

    nil # 빌드를 찾지 못한 경우 nil 반환
  end

  # App ID 가져오기 함수 (필요 시 사용)
  def fetch_app_id(jwt_token, bundle_id)
    uri =
URI("https://api.appstoreconnect.apple.com/v1/apps?filter[bundleId]=#{bundle_id}")
    request = Net::HTTP::Get.new(uri)
    request['Authorization'] = "Bearer #{jwt_token}"

    response = Net::HTTP.start(uri.hostname, uri.port, use_ssl: true) do |http|
      http.request(request)
    end

    if response.code.to_i == 200
      apps = JSON.parse(response.body)['data']
      if apps && !apps.empty?
        app_id = apps.first['id']
        return app_id
      else
        UI.user_error!("[AppSealing] No app found with bundle ID: #{bundle_id}")
      end
    else
      UI.error("[AppSealing] Failed to fetch app ID: #{response.code} -
#{response.body}")
      UI.user_error!("[AppSealing] Unable to retrieve app ID for bundle ID:
#{bundle_id}")
    end

    nil # 앱 ID를 찾지 못한 경우 nil 반환
  end

  # JWT 토큰 생성 및 최신 빌드 번호 가져오기
  jwt_token = generate_jwt_token(PRIVATE_KEY_CONTENT)
  version = fetch_version_from_xcode(xcode_file_path, scheme_name)
  latest_build_number = fetch_latest_build_number(jwt_token, bundle_id, version)

  if latest_build_number.nil?

```

```

    UI.message("[AppSealing] No builds found for version #{version}. Starting
with build number 1.")
    new_build_number = 1
  else
    new_build_number = latest_build_number + 1
    UI.message("[AppSealing] Latest Build Number: #{latest_build_number}")
  end

  # 새로운 빌드 번호 설정 (Xcode 프로젝트에 적용)
  increment_build_number(
    build_number: new_build_number,
    xcodeproj: xcode_file_path # Xcode 프로젝트 경로 명시
  )

  UI.message "[AppSealing] Updated Build Number: #{new_build_number}"

  # 빌드 및 IPA 파일 생성
  archive_path = File.join(project_root, "build", "#{project_name}.xcarchive")
  ipa_output_path = File.join(project_root, "build")

  build_ios_app(
    scheme: scheme_name,
    export_method: "app-store",
    clean: true,
    output_directory: ipa_output_path,
    output_name: "#{project_name}.ipa",
    workspace: xcode_file_path.end_with?(".xcworkspace") ? xcode_file_path : nil,
    project: xcode_file_path.end_with?(".xcodeproj") ? xcode_file_path : nil,
    export_options: {
      provisioningProfiles: {
        bundle_id => ENV["PROFILE"]
      }
    },
    xcargs: "CODE_SIGN_STYLE=Manual DEVELOPMENT_TEAM=#{team_id}
PROVISIONING_PROFILE_SPECIFIER=\"#{ENV['PROFILE']}\"",
    codesigning_identity: "Apple Distribution"
  )

  # .ipa 파일 경로 설정
  ipa_path = File.join(ipa_output_path, "#{project_name}.ipa")

  unless File.exist?(ipa_path)
    UI.user_error!("[AppSealing] IPA file not found at path: #{ipa_path}")
  end

  UI.message "[AppSealing] IPA Path: #{ipa_path}"

  # generate_hash 스크립트 경로 동적 검색
  generate_hash_script = File.join(project_root, "AppSealingSDK", "Tools",
"generate_hash")
  unless File.exist?(generate_hash_script)

```

```

    UI.user_error!("[AppSealing] generate_hash script not found at path:
#{generate_hash_script}")
  end

  # 실행 권한 확인 및 설정
  unless File.executable?(generate_hash_script)
    sh("chmod +x '#{generate_hash_script}'")
  end
  sh("'#{generate_hash_script}' '#{File.absolute_path(ipa_path)}'")

  # Step 2: Upload the IPA to TestFlight with callback
  begin
    upload_to_testflight(
      ipa: ipa_path,
      api_key: {
        key_id: KEY_ID,
        issuer_id: ISSUER_ID,
        key: PRIVATE_KEY_CONTENT
      },
      skip_waiting_for_build_processing: true, # 빌드 처리 대기 시간 생략
    )
    UI.success("[AppSealing] Upload to TestFlight completed successfully!")

  rescue => e
    UI.error("[AppSealing] Upload to TestFlight failed with error: #{e.message}")
    raise e

  ensure
    # Always delete the build directory, even if an error occurs
    if Dir.exist?(ipa_output_path)
      UI.message("[AppSealing] Deleting build directory: #{ipa_output_path}")
      FileUtils.rm_rf(ipa_output_path)
      UI.message("[AppSealing] Build directory deleted.")
    else
      UI.message("[AppSealing] Build directory not found or already deleted.")
    end
  end
end
end
end
end

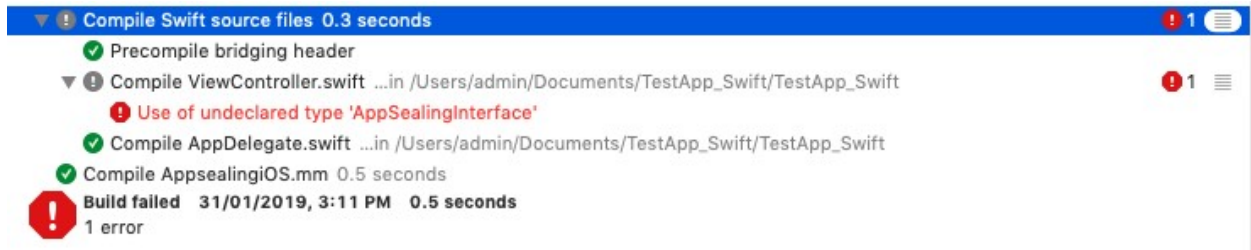
```

이 스크립트는 App Store Connect의 앱 정보를 가져와 자동으로 빌드 번호를 증가시킨 후 앱을 빌드하고 IPA로 내보낸 다음 generate_hash 스크립트를 적용하고 재서명하여 App Store Connect로 업로드 하는 전 과정을 포함하고 있습니다.

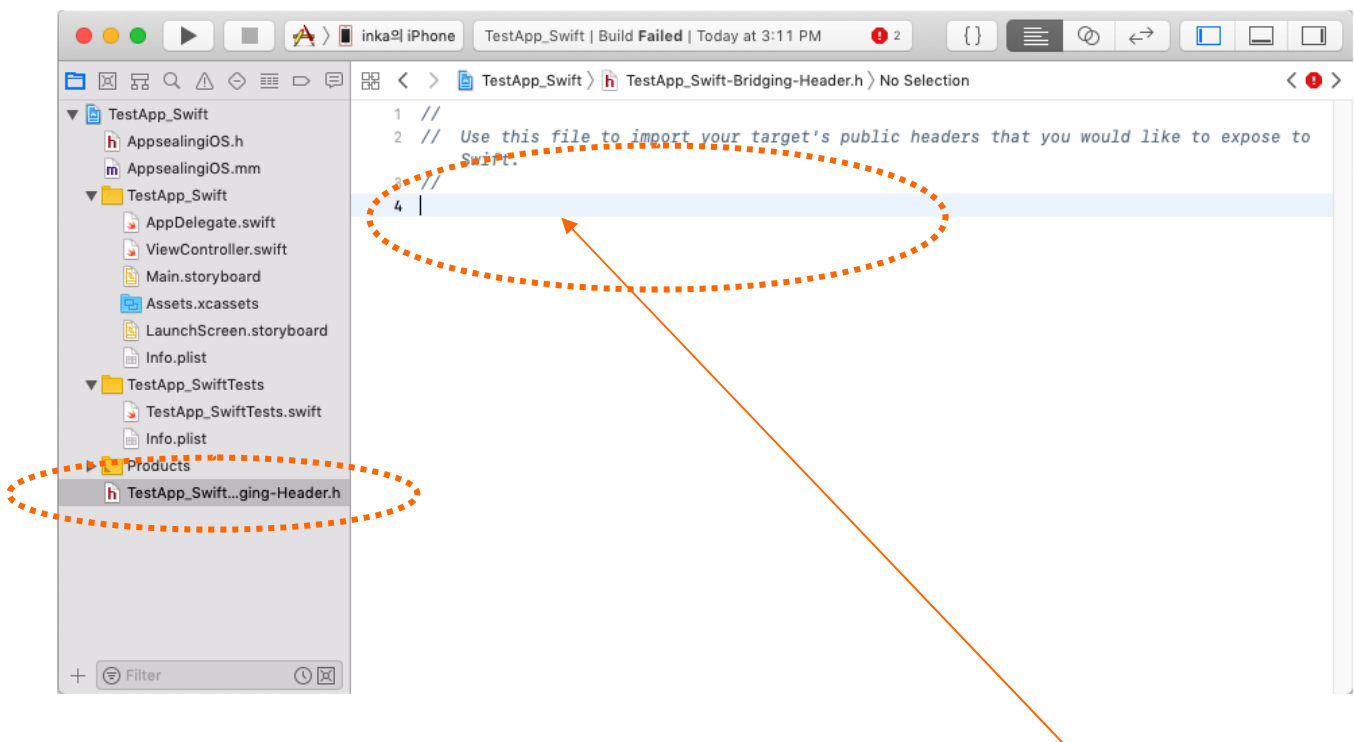
이 과정에 필요한 프로젝트 이름과 스킴, 타겟, 번들 ID 등의 모든 값은 프로젝트 파일에서 자동으로 추출하여 사용하도록 되어 있기 때문에 TestApp_Swift가 아닌 다른 프로젝트에 적용할 때도 동일한 스크립트를 그대로 사용할 수 있습니다.

Part 10. 문제 해결

10-1 Xcode Build error (1) Use of undeclared type 'AppSealingInterface'



Bridging header 파일에 '#import "AppsealingiOS.h"' 가 생략되면 위와 같은 오류가 발생합니다.



"TestApp_Swift-Bridging-Header.h"를 선택하고 '#import "AppsealingiOS.h"'를 추가하십시오. "AppSealingiOS.h" 파일이 지정된 경로에 해당 파일이 존재하는지 확인하십시오. (Xcode project/AppSealingSDK/Libraries/AppsealingiOS.h)

또한 "AppSealingiOS.h" 파일은 "AppSealingiOS.mm" 파일과 함께 프로젝트에 포함되어 있어야 합니다.

10-2 Xcode Build error (2) Undefined symbols for architecture arm64: "_OBJC_CLASS_\$_AppSealingInterface"

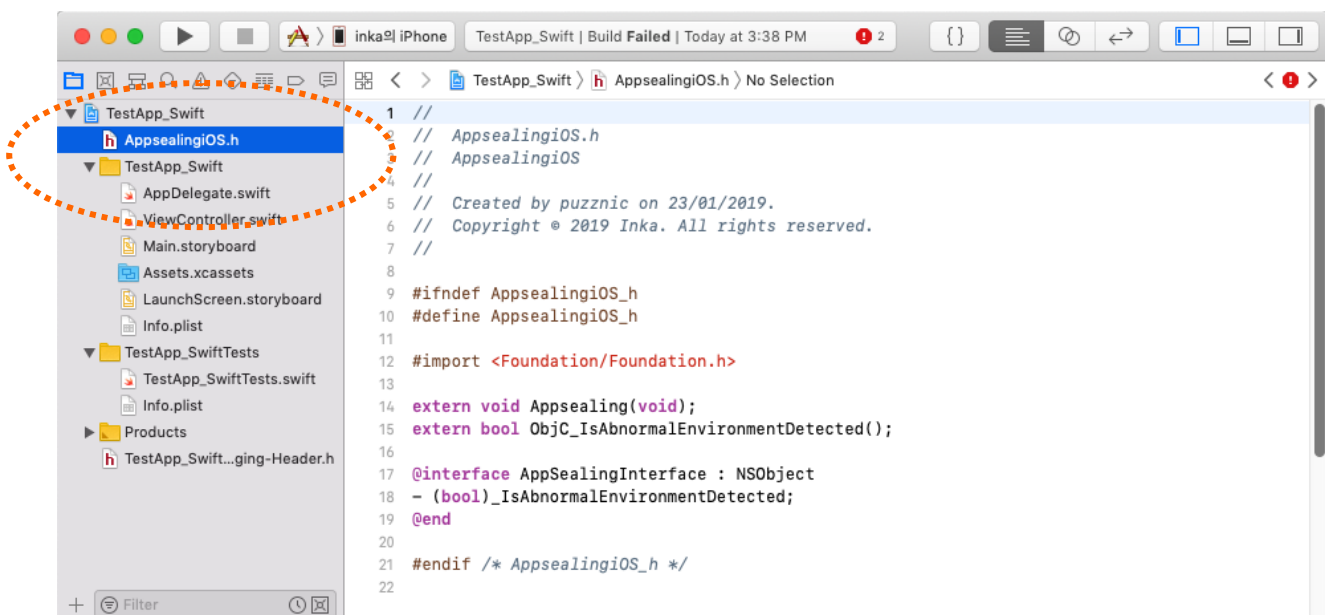
```

Ld /Users/admin/Desktop/Build/Products/Debug-iphonios/TestApp_Swift.app/TestApp_Swift normal arm64
(in target: TestApp_Swift)
cd /Users/admin/Documents/TestApp_Swift
export IPHONEOS_DEPLOYMENT_TARGET=12.1
/Applications/Xcode.app/Contents/Developer/Toolchains/XcodeDefault.xctoolchain/usr/bin/clang -
arch arm64 -isysroot /Applications/Xcode.app/Contents/Developer/Platforms/iPhoneOS.platform/
Developer/SDKs/iPhoneOS12.1.sdk -L/Users/admin/Desktop/Build/Products/Debug-iphonios -F/Users/
admin/Desktop/Build/Products/Debug-iphonios -filelist /Users/admin/Desktop/Build/
Intermediates.noindex/TestApp_Swift.build/Debug-iphonios/TestApp_Swift.build/Objects-normal/arm64/
TestApp_Swift.LinkFileList -rpath @executable_path/Frameworks -miphoneos-
version-min=12.1 -dead_strip -Xlinker -object_path_lto -Xlinker /Users/admin/Desktop/Build/
Intermediates.noindex/TestApp_Swift.build/Debug-iphonios/TestApp_Swift.build/Objects-normal/arm64/
TestApp_Swift_lto.o -Xlinker -export_dynamic -Xlinker -no_deduplicate -fembed-bitcode-marker -
fobjc-link-runtime -L/Applications/Xcode.app/Contents/Developer/Toolchains/
XcodeDefault.xctoolchain/usr/lib/swift/iphonios -Xlinker -add_ast_path -Xlinker /Users/admin/
Desktop/Build/Intermediates.noindex/TestApp_Swift.build/Debug-iphonios/TestApp_Swift.build/
Objects-normal/arm64/TestApp_Swift.swiftmodule -lStaticAppSec_Debug -L/Users/admin/Documents/
TestApp_Swift/AppSealingSDK/Libraries -Xlinker -dependency_info -Xlinker /Users/admin/Desktop/
Build/Intermediates.noindex/TestApp_Swift.build/Debug-iphonios/TestApp_Swift.build/Objects-normal/
arm64/TestApp_Swift_dependency_info.dat -o /Users/admin/Desktop/Build/Products/Debug-iphonios/
TestApp_Swift.app/TestApp_Swift

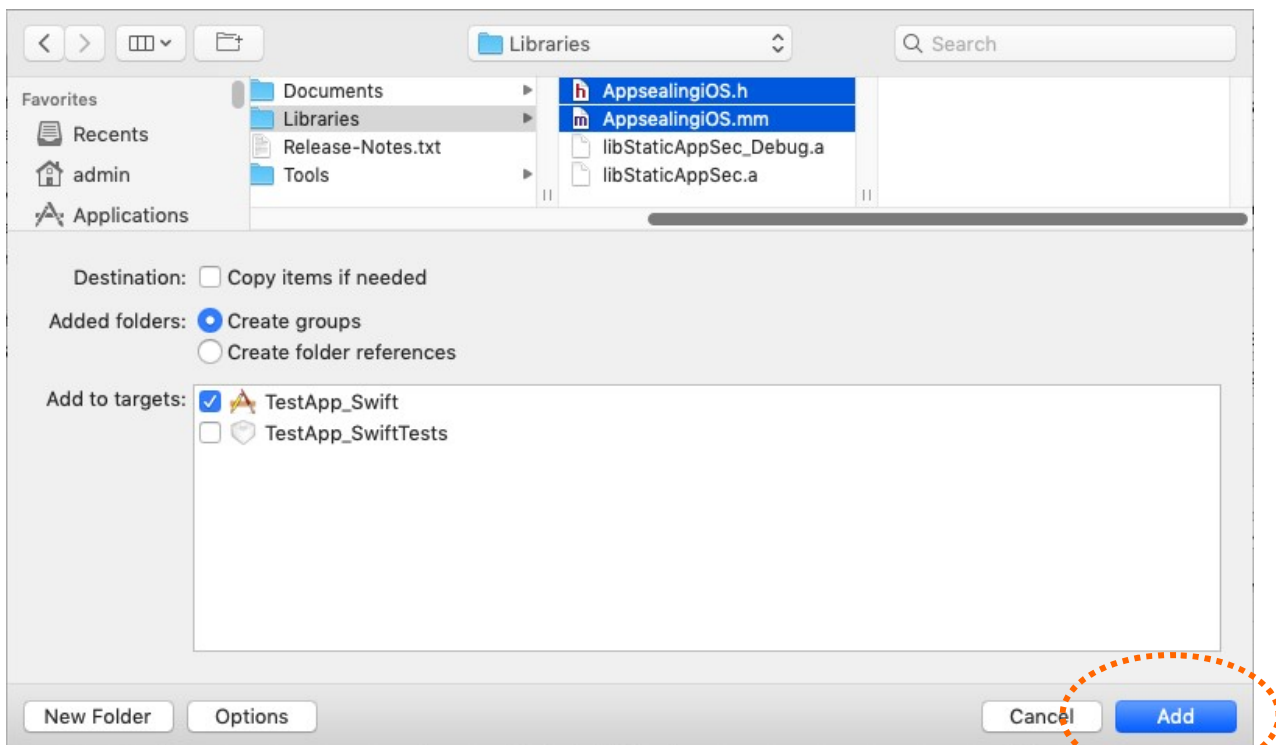
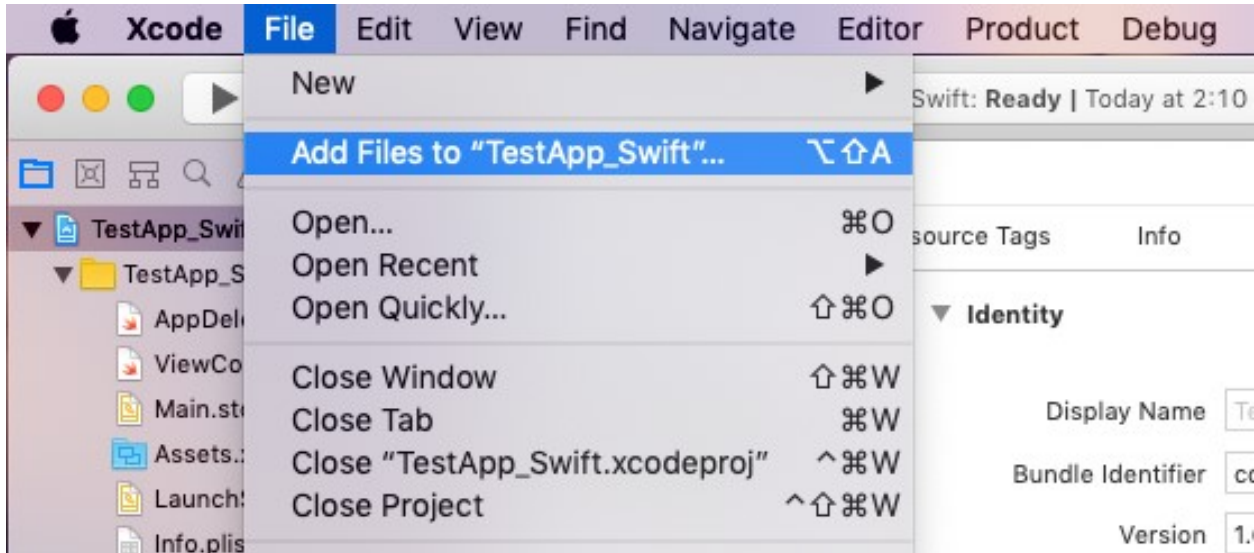
Undefined symbols for architecture arm64:
  "_OBJC_CLASS_$_AppSealingInterface", referenced from:
    objc-class-ref in ViewController.o
ld: symbol(s) not found for architecture arm64
clang: error: linker command failed with exit code 1 (use -v to see invocation)

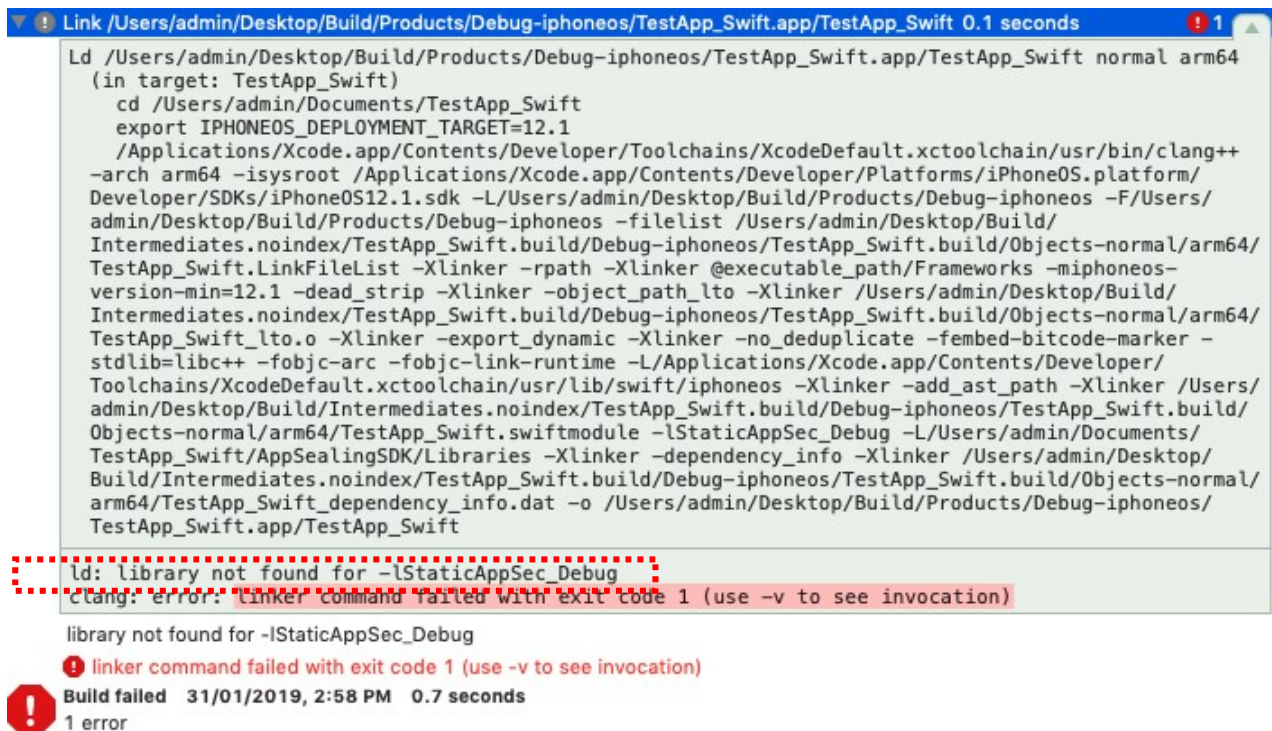
symbol(s) not found for architecture arm64
❗ linker command failed with exit code 1 (use -v to see invocation)
✅ Copy TestApp_Swift.swiftmodule 0.1 seconds
✅ Copy TestApp_Swift.swiftdoc 0.1 seconds
❗ Build failed 31/01/2019, 3:38 PM 0.9 seconds
1 error
  
```

이 링커 오류는 "AppSealingiOS.mm"이 프로젝트에 포함되어 있지 않을 때 발생합니다.
프로젝트 트리에서 해당 파일이 추가되어 있는지 확인하십시오.



"AppSealingiOS.mm" 파일이 프로젝트에 포함되어 있지 않은 경우 "File > Add Files to "TestApp_Swift" ... " 메뉴를 실행하고 AppSealing SDK 경로에서 파일을 선택하여 프로젝트에 포함 시키십시오.



10-3 Xcode Build error (3) **ld: library not found for -lStaticAppSec_Debug**


```

Link /Users/admin/Desktop/Build/Products/Debug-iphoneos/TestApp_Swift.app/TestApp_Swift 0.1 seconds
ld /Users/admin/Desktop/Build/Products/Debug-iphoneos/TestApp_Swift.app/TestApp_Swift normal arm64
(in target: TestApp_Swift)
  cd /Users/admin/Documents/TestApp_Swift
  export IPHONEOS_DEPLOYMENT_TARGET=12.1
  /Applications/Xcode.app/Contents/Developer/Toolchains/XcodeDefault.xctoolchain/usr/bin/clang++
  -arch arm64 -isysroot /Applications/Xcode.app/Contents/Developer/Platforms/iPhoneOS.platform/
  Developer/SDKs/iPhoneOS12.1.sdk -L/Users/admin/Desktop/Build/Products/Debug-iphoneos -F/Users/
  admin/Desktop/Build/Products/Debug-iphoneos -filelist /Users/admin/Desktop/Build/
  Intermediates.noindex/TestApp_Swift.build/Debug-iphoneos/TestApp_Swift.build/Objects-normal/arm64/
  TestApp_Swift.LinkFileList -Xlinker -rpath @executable_path/Frameworks -miphoneos-
  version-min=12.1 -dead_strip -Xlinker -object_path_lto -Xlinker /Users/admin/Desktop/Build/
  Intermediates.noindex/TestApp_Swift.build/Debug-iphoneos/TestApp_Swift.build/Objects-normal/arm64/
  TestApp_Swift_lto.o -Xlinker -export_dynamic -Xlinker -no_deduplicate -fembed-bitcode-marker -
  stdlib=libc++ -fobjc-arc -fobjc-link-runtime -L/Applications/Xcode.app/Contents/Developer/
  Toolchains/XcodeDefault.xctoolchain/usr/lib/swift/iphoneos -Xlinker -add_ast_path -Xlinker /Users/
  admin/Desktop/Build/Intermediates.noindex/TestApp_Swift.build/Debug-iphoneos/TestApp_Swift.build/
  Objects-normal/arm64/TestApp_Swift.swiftmodule -lStaticAppSec_Debug -L/Users/admin/Documents/
  TestApp_Swift/AppSealingSDK/Libraries -Xlinker -dependency_info -Xlinker /Users/admin/Desktop/
  Build/Intermediates.noindex/TestApp_Swift.build/Debug-iphoneos/TestApp_Swift.build/Objects-normal/
  arm64/TestApp_Swift_dependency_info.dat -o /Users/admin/Desktop/Build/Products/Debug-iphoneos/
  TestApp_Swift.app/TestApp_Swift

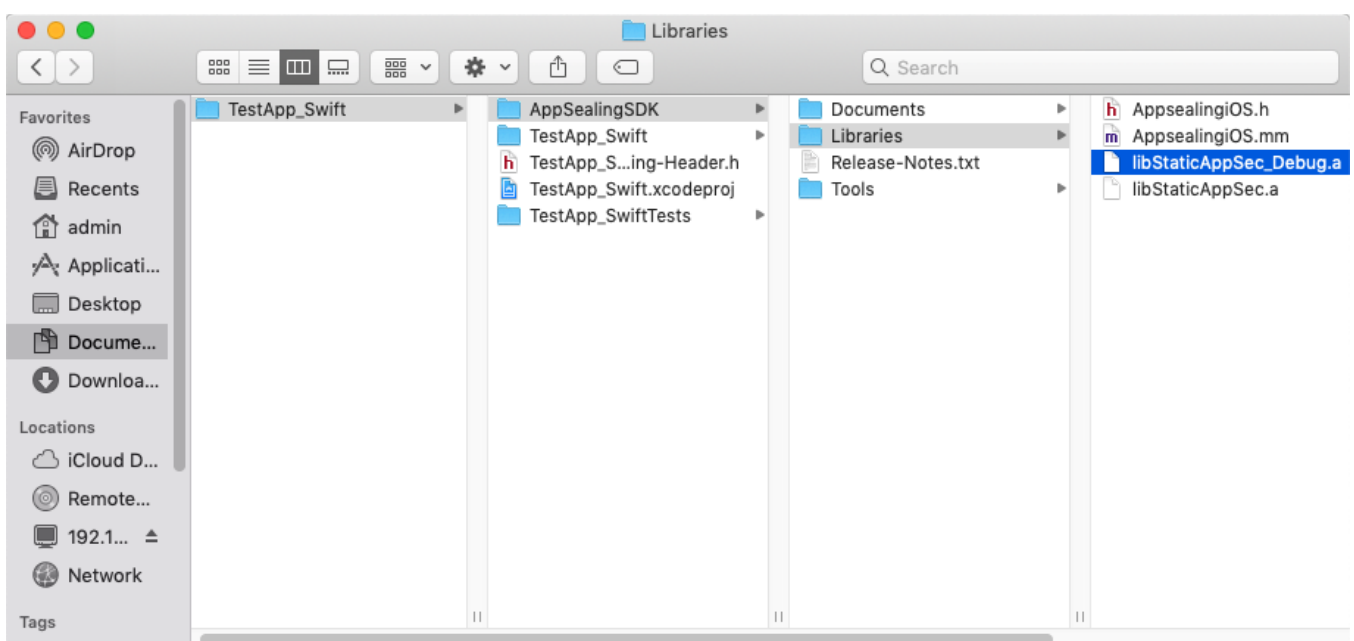
ld: library not found for -lStaticAppSec_Debug
clang: error: linker command failed with exit code 1 (use -v to see invocation)

library not found for -lStaticAppSec_Debug
! linker command failed with exit code 1 (use -v to see invocation)
Build failed 31/01/2019, 2:58 PM 0.7 seconds
1 error

```

이 링커 오류는 "libStaticAppSec_Debug.a" 파일이 지정된 폴더에 없거나 손상된 경우에 발생합니다. "libStaticAppSec_Debug.a" 파일은 아래 경로에 존재해야 합니다.

"TestApp_Swift (Project folder) > AppSealingSDK > Libraries > libStaticAppSec_Debug.a"

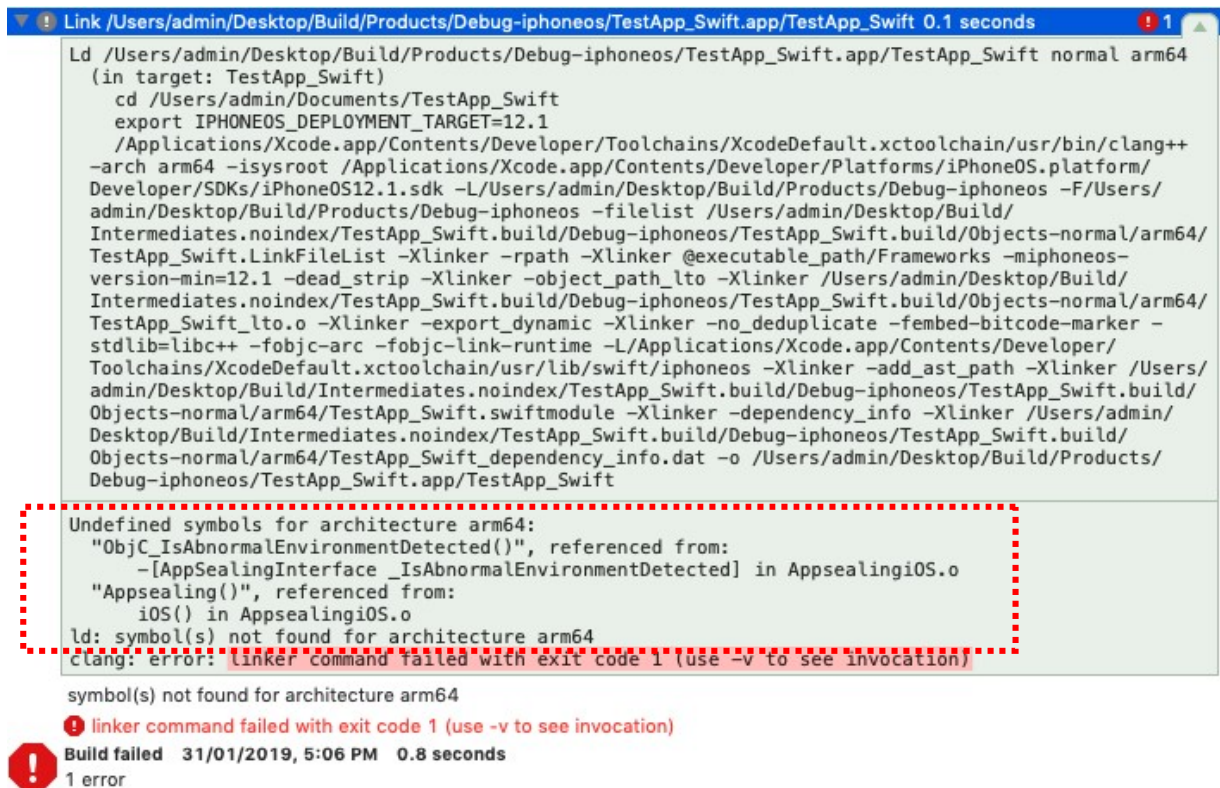


파일이 누락되었거나 손상되었을 경우 AppSealing SDK zip 파일을 다시 압축 해제하시거나 새로 다운로드하십시오.

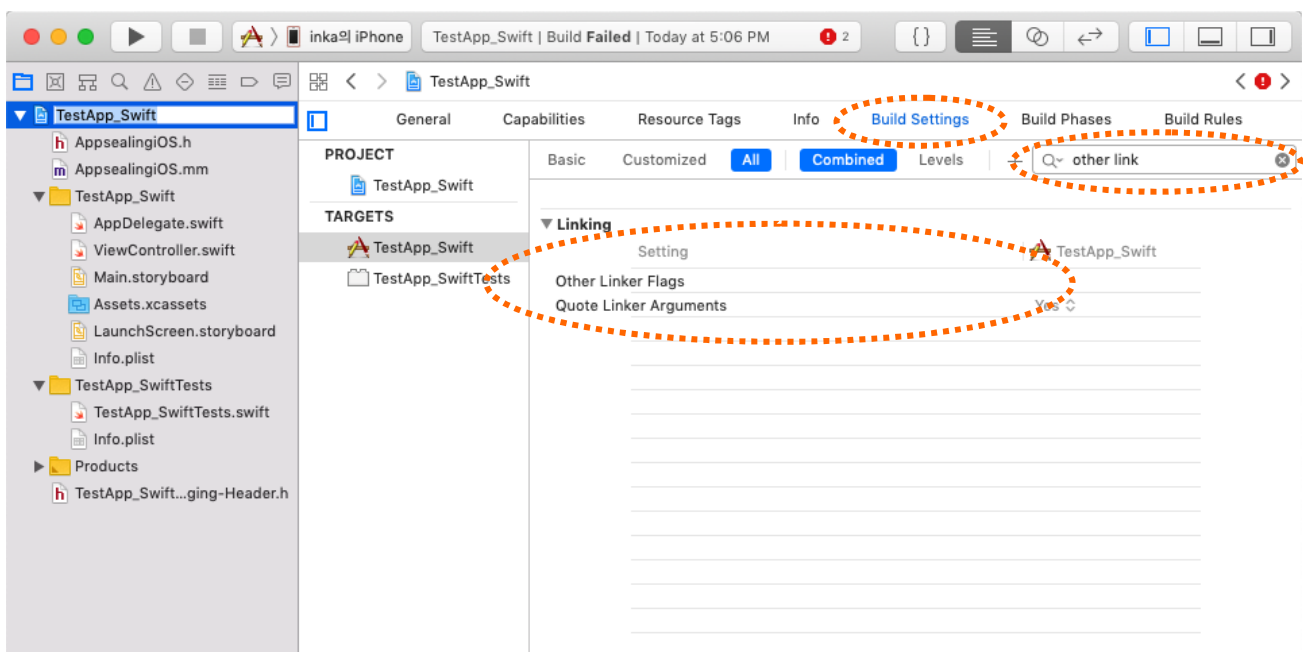
이 링커 오류는 release 빌드 시 "lStaticAppSec" 옵션을 추가했을 때에도 발생할 수 있습니다. Release 빌드를 위해서 "libStaticAppSec.a" 파일이 적절한 위치에 존재해야 합니다.

"TestApp_Swift (Project folder) > AppSealingSDK > Libraries > libStaticAppSec.a"

10-4 Xcode Build error (4) Undefined symbols for architecture arm64: "ObjC_IsAbnormalEnvironmentDetected()", "Appsealing()"

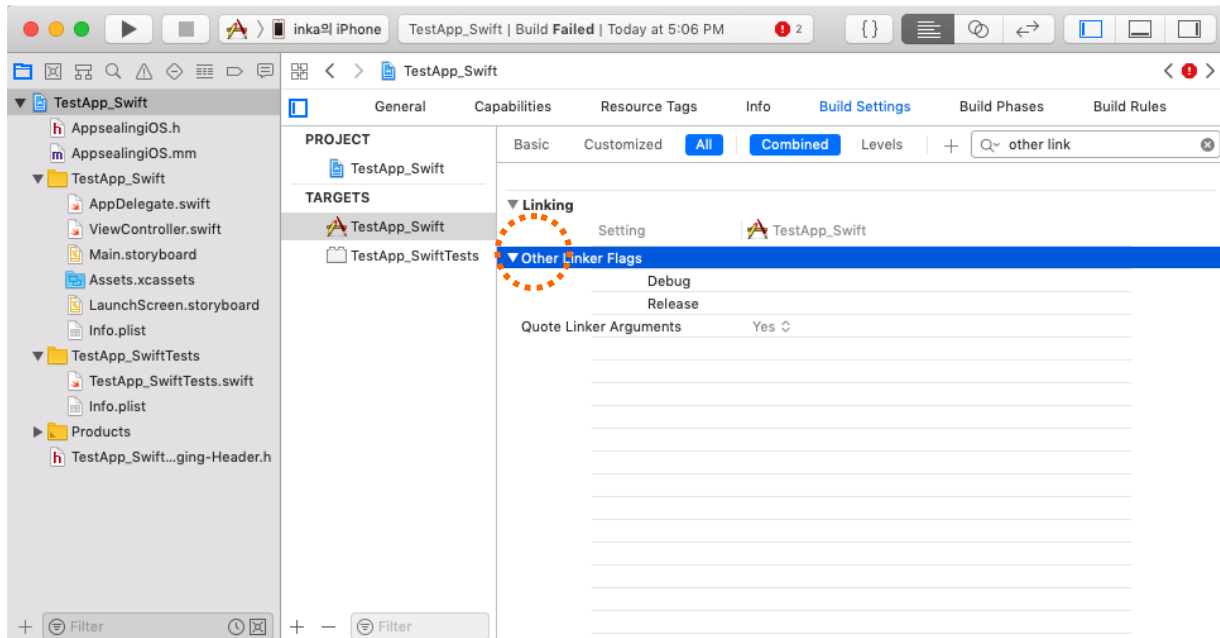


이 링커 오류는 "Build Settings"의 "Other Linker Flags" 필드 값이 없거나 잘못된 경우 발생합니다. 먼저, "Build Settings"의 **"Other Linker Flags"** 필드 값을 확인해보십시오.



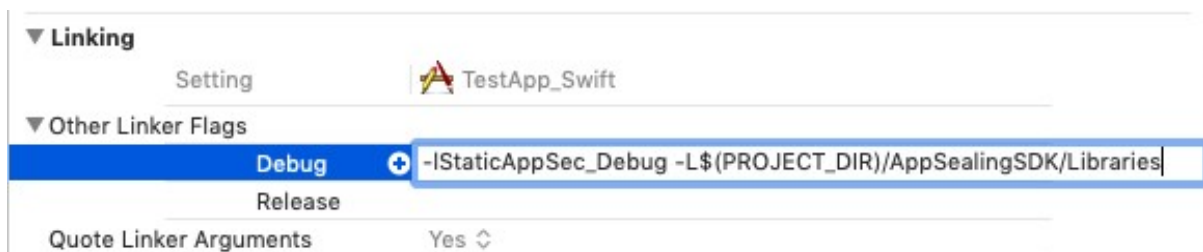
다음으로 "Other Linker Flags" 설정을 복원하십시오.

- 1) 설정값 삭제 : "Other Linker Flags" 줄을 선택하고 'Delete' 키를 눌러 삭제합니다.
- 2) "Other Linker Flags"의 왼쪽 삼각형 아이콘을 클릭하여 "Other Linker Flags"를 확장하십시오.



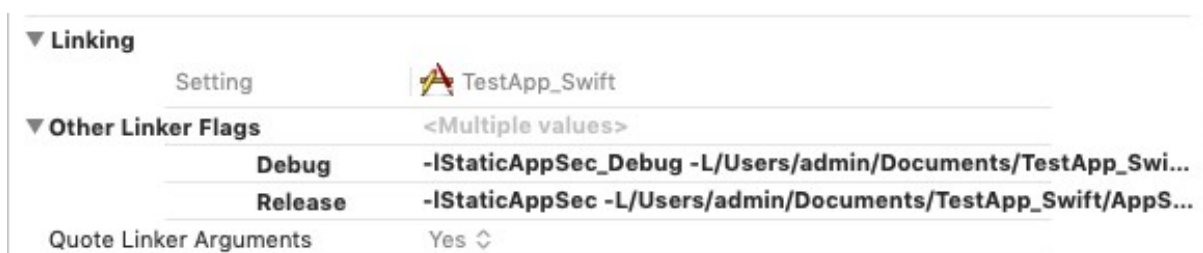
- 3) "Debug" 항목을 선택하고 클릭하여 아래 텍스트를 입력하십시오.

-IStaticAppSec_Debug -L\$(PROJECT_DIR)/AppSealingSDK/Libraries

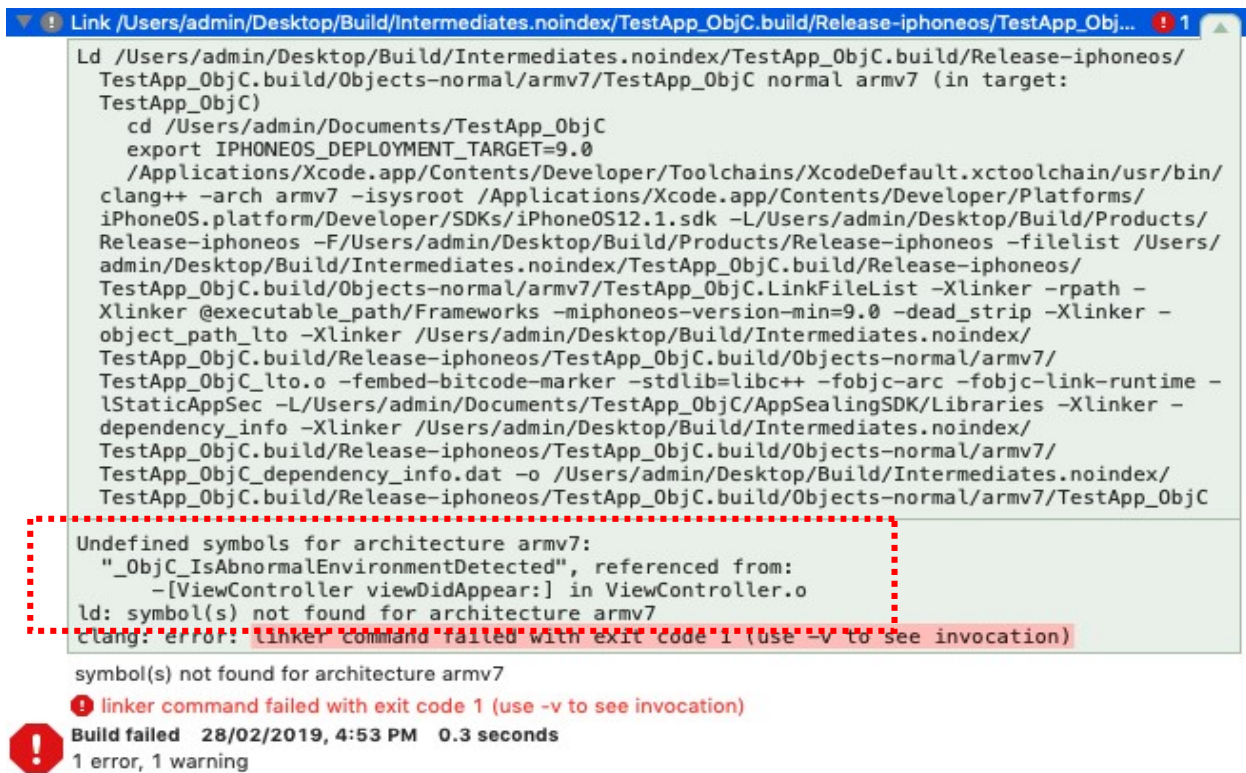


- 4) "Release" 항목을 선택하고 클릭하여 아래 텍스트를 입력하십시오.

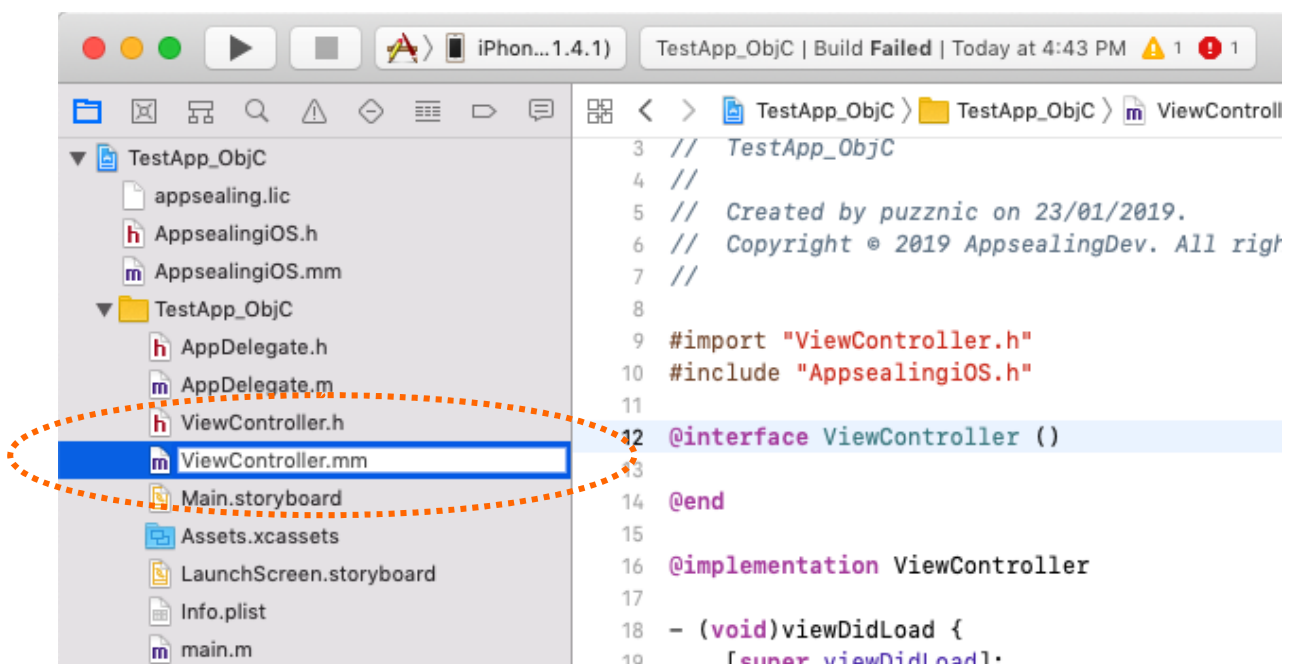
-IStaticAppSec -L\$(PROJECT_DIR)/AppSealingSDK/Libraries



10-5 Xcode Build error (5) **Undefined symbols for architecture arm64:**
"ObjC_IsAbnormalEnvironmentDetected()" (Objective-C 프로젝트)

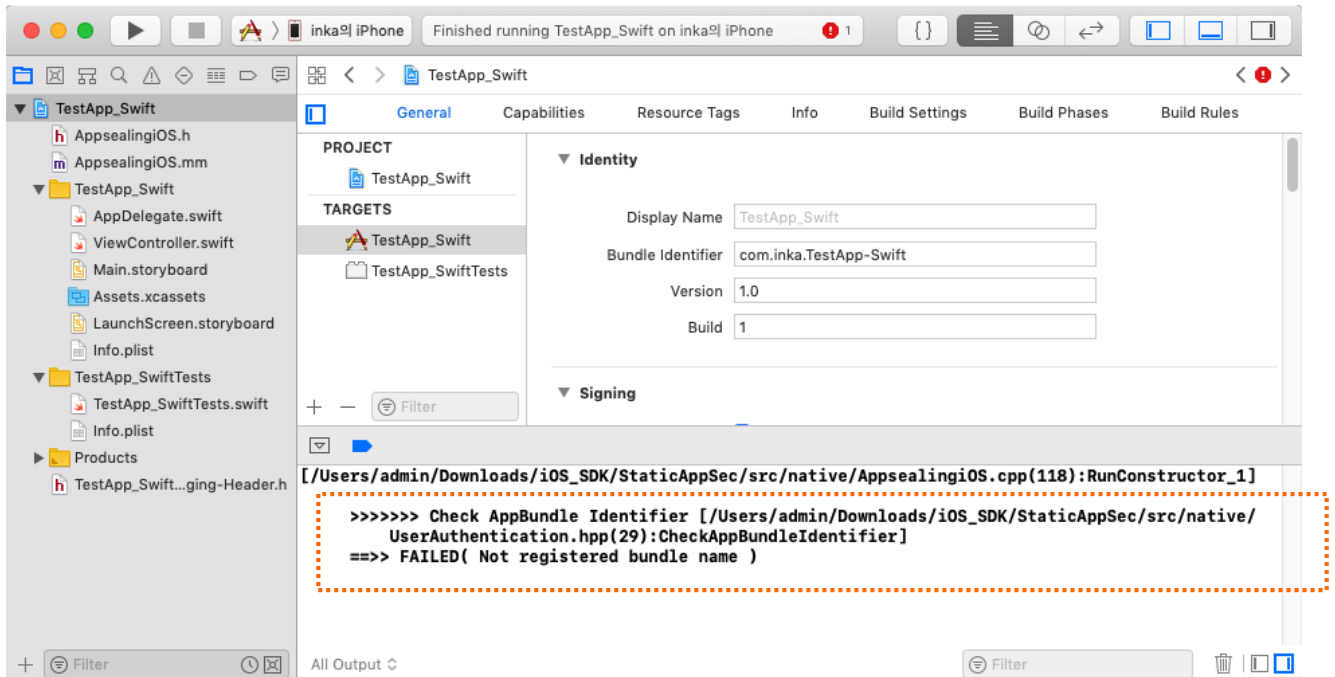


이 링커 오류는 프로젝트가 Objective-C 기반일 때 ViewController 코드의 확장자가 ".m" 인 경우 발생합니다. 파일 확장자를 ".mm"으로 변경하십시오.

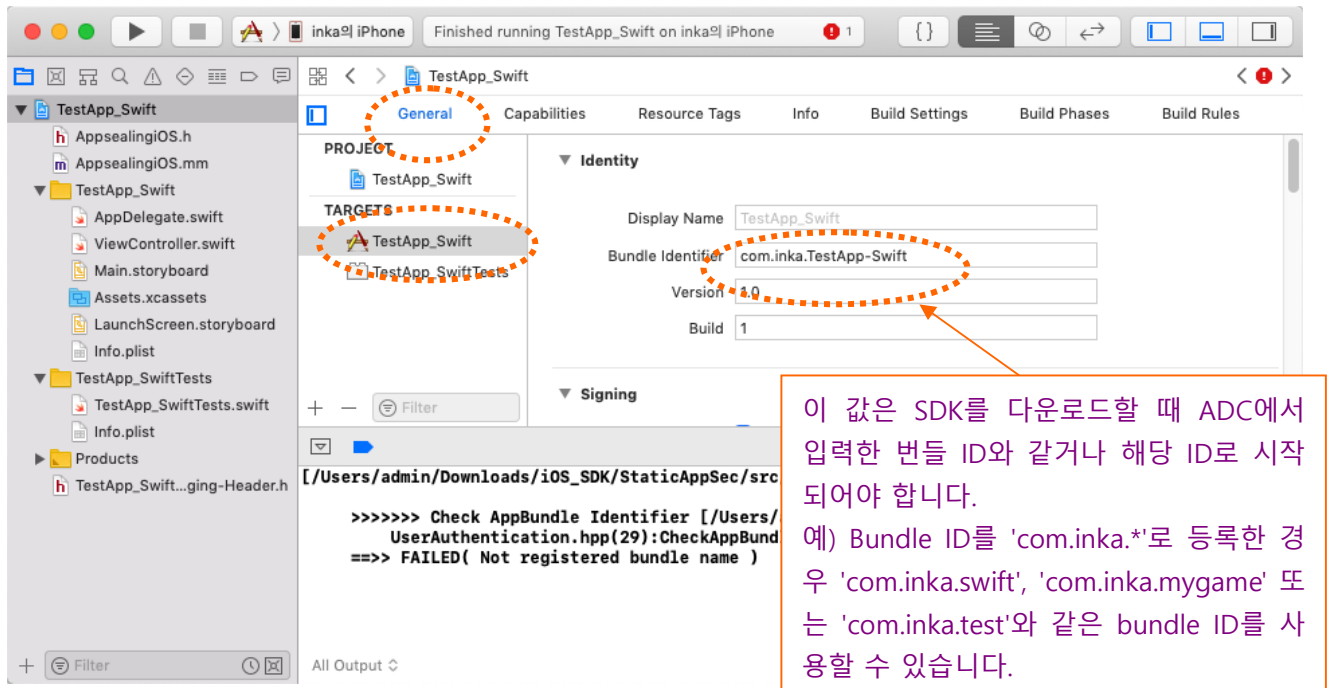


10-6 Execution error (I) 앱이 실행되고 곧 바로 종료됩니다.

기기에서 앱이 실행 직후 종료된 경우 bundle id가 유효한지 실행 로그 메시지를 확인하십시오.

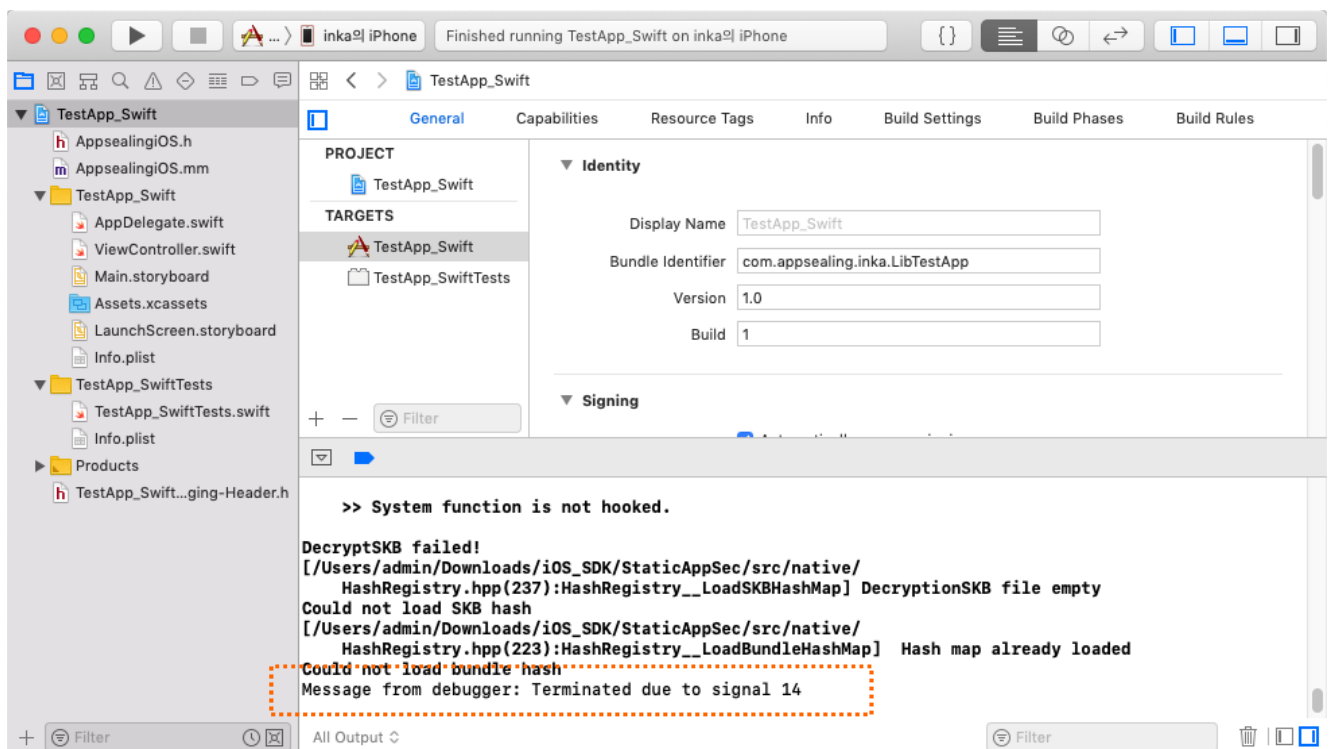


로그 메시지에 "====>>> FAILED(Not registered bundle name)"와 같은 문자열이 포함되어 있으면 현재 프로젝트의 bundle id가 AppSealing SDK를 다운로드하는 동안 AppSealing Developer Center (ADC)에 제대로 등록되어 있지 않다는 의미입니다. ADC에서 SDK를 다운로드할 때 입력한 값과 같은 bundle id를 사용하십시오.



10-7 Execution error (II) 앱이 실행 중 갑자기 종료됩니다.

실행 후 약 20초 후에 갑자기 앱이 종료된 경우, 실행 로그를 확인하여 앱이 릴리스로 실행되었는지 확인할 수 있습니다.

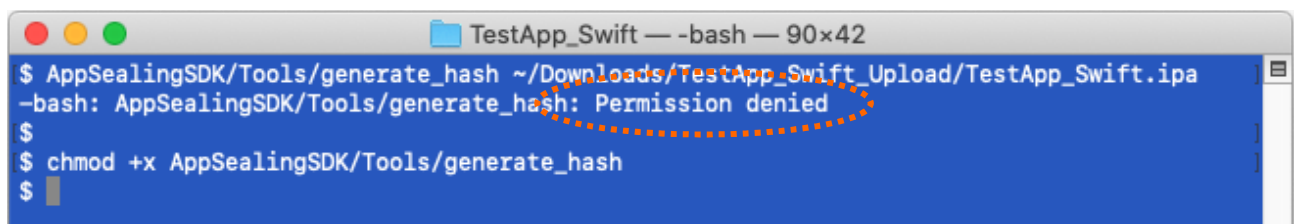


로그 메시지에 "[Message from debugger: Terminated due to signal 14](#)" 와 같은 문자열이 포함되어 있는 경우 앱이 release로 빌드, 실행되었고 AppSealing 보안 로직이 활성화되었음을 알 수 있습니다. Release 모드의 AppSealing 보안 로직은 탈옥된 장치, 실행 파일의 복호화 된 상태, 디버거 연결과 같은 보안 침입을 확인하며, 어떤 문제가 있을 경우 사용자의 조치나 기타 상황에 관계없이 20초 후에 앱을 닫습니다.

Release 모드에서는 AppSealing 로직이 활성화되고 앱이 종료되므로 AppStore 또는 TestFlight에 배포하기 전까지 디버그 모드로 앱을 테스트하십시오. AppSealing 로직은 release 모드로 빌드 시 Xcode 장치에서 앱을 실행하면 디버깅 환경으로 탐지하거나 실행 파일이 비정상적으로 해독되어 있음을 감지하고 20초 후에 앱을 종료합니다.

10-8 무결성/인증서 스냅샷 생성 스크립트가 실행되지 않습니다.

3-5 단계의 무결성/인증서 스냅샷 정보 생성 단계에서 아래와 같이 스크립트 실행이 실패하고 "권한 없음" 오류 메시지가 출력되는 경우가 발생할 수 있습니다.

A terminal window titled 'TestApp_Swift — -bash — 90x42' shows the command 'AppSealingSDK/Tools/generate_hash ~/Downloads/TestApp_Swift_Upload/TestApp_Swift.ipa' being executed. The output is '-bash: AppSealingSDK/Tools/generate_hash: Permission denied'. A red dashed circle highlights the error message. Below the error, the command '\$ chmod +x AppSealingSDK/Tools/generate_hash' is entered and executed, resulting in a new prompt '\$'.

이 경우 아래와 같이 스크립트 파일에 실행 권한을 추가하는 명령을 실행 하고 다시 시도해 보십시오.

```
$ chmod +x AppSealingSDK/Tools/generate_hash
```